

▼ A story of scaling (from algorithms to systems)

▼ Prologue: LLMs everywhere, and a bitter lesson

- 1 ♦ Large, language, models: already everywhere, seemingly able to do everything
- 2 ♦ Large, language, models: what might be behind them?
- 3 ♦ Quantity (large) becomes quality: the magic of exponential scaling
- 4 ♦ The Bitter Lesson
- 5 ♦ Case #1: Deep Blue and chess
- 6 ♦ Case #1: Deep Blue — good ELO by scaling deep search
- 7 ♦ Case #2: AlphaGo
- 8 ♦ Case #2: AlphaGo vs. AlphaGo Zero
- 9 ♦ Case #3: computer vision — learned features > hand-crafted features
- 10 ♦ Connecting the Bitter Lesson to exponential scaling: Moore's Law
- 11 ♦ Wait a minute: why does more compute bring more intelligence?

▼ The learned model, and its scaling law

- 12 ♦ A possible meta method: The large language model
- 13 ♦ Question: how much computation to train on / compute a token?
- 14 ♦ Question: along which dimension do we scale?
- 15 ♦ The Scaling law
- 16 ♦ The Chinchilla Law
- 17 ♦ (Aside) Recall: why must the model be a language model?

▼ Systems are essential to scaling

- 18 ♦ Step one of realizing the scaling law: large-scale digital systems
- 19 ♦ Case study: suppose we build an "all-in-one" app
- 20 ♦ How was it done traditionally?
- 21 ♦ Why LAMP? We need abstractions to simplify development

▼ Scaling storage

- 22 ♦ Problem: scalability of data storage
- 23 ♦ Solution: scale storage with distributed storage
- 24 ♦ Scalability challenge: how to shard?
- 25 ♦ Why naïve sharding cannot scale
- 26 ♦ Solution: consistent hashing
- 27 ♦ Now we come at the performance: Latency & Throughput
- 28 ♦ Performance solution #1: Caching
- 29 ♦ Issue: consistency of caching
- 30 ♦ The consistency model for inferring what can happen
- 31 ♦ Issue: what about faults
- 32 ♦ Fault causes: many sources
- 33 ♦ What can go wrong with faults? Besides out-of-service, there can even be consistency issues!
- 34 ♦ The consistency issue further goes to multi-site case
- 35 ♦ Reliability: from Consistency to Availability
- 36 ♦ Availability is possible: failure vs. partial failure
- 37 ♦ Solution: Replication
- 38 ♦ The challenge of replication: consistency
- 39 ♦ A quick summary: building a scalable, reliable and consistent system
- 40 ♦ The real world: failures are everywhere
- 41 ♦ Large-scale AI services fail too

▼ Scaling performance

- 42 ♦ Parallelism(Concurrency) for a high throughput
- 43 ♦ Sidenote: Parallelism can also reduce latency
- 44 ♦ Example: Parallelism in AI — Training & Inference
- 45 ♦ Training: spread the tasks across GPUs (machines)
- 46 ♦ Inference (or normal request processing in traditional systems): we need to serve many requests at the same time
- 47 ♦ The problem with parallelism: Consistency (the data can still go wrong!)
- 48 ♦ Inter-task example: two transactions, one ticket (Isolation)
- 49 ♦ Intra-task example: synchronous vs. asynchronous training
- 50 ♦ Asynchronous training in practice: lower accuracy, slower convergence
- 51 ♦ Why concurrent consistency is hard: Inter-task Dependency Tracking
- 52 ♦ Another parallelism challenge: Intra-task Workload Partition
- 53 ♦ Cut along k instead: even ignoring memory, the extra work differs
- 54 ♦ Inter-task Partition has challenges too: Load Balance is not that simple
- 55 ♦ Scaling is not that easy, there are walls!
- 56 ♦ Takeaway: Scale the System with Three Easy Pieces — Consistency, Reliability & Performance

Large, language, models: already everywhere, seemingly able to do everything

Everywhere: the **coding assistants** we use every day and the customer service we meet are all backed by LLMs

Able to do (seemingly) everything: an LLM – **not even the strongest one** – did in ten minutes what took me **weeks** as a PhD student in 2015

- e.g., get the RDMA to work



The screenshot shows a terminal window with a dark background. At the top, the text '+1 more · /status' is visible. Below this, there is a large empty space. At the bottom, there is a chat interface with a prompt '› 帮我写一个 RDAM 的 demo' and a response '[Opus 4.8] | wxd'. Below the response, the text 'Context' is followed by a progress bar and '0%'.

```
+1 more · /status
```

```
› 帮我写一个 RDAM 的 demo
```

```
[Opus 4.8] | wxd
```

```
Context ██████████ 0%
```

Large, language, models: what might be behind them?

What is a model? Simple: any AI can be abstracted as a function: $action = F(input)$

- *action* is what to do next; *input* is the current context — for a human, what you see, hear, and feel, etc.

Why use language to encode *action* and *input*?



Why does it take "large" to succeed?

¹ Source: 大模型带来的新范式 新时代 新机会 (talk, in Chinese)

Quantity (large) becomes quality: the magic of exponential scaling



Atomic Habits: Question: if you improve by 1% every day, how much better are you after a year? 37 times better! That is the power of exponential growth

AI surpassing human intelligence may happen in an instant — humans may not even notice

1

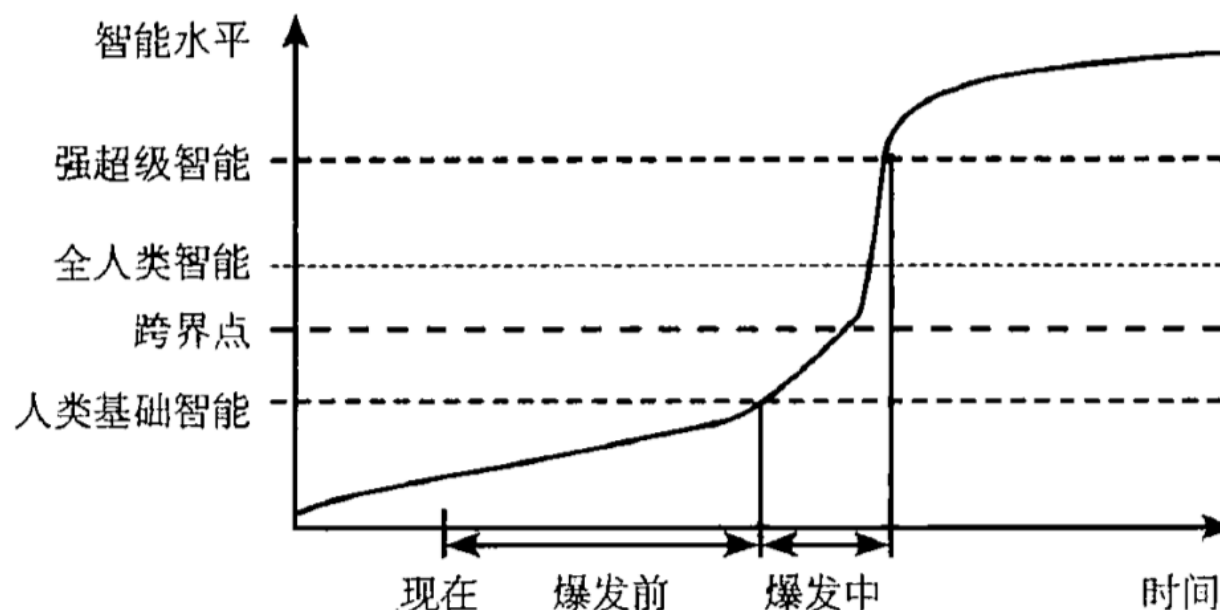


图 4-1 智能爆发图

¹ Figure: *Superintelligence: Paths, Dangers, Strategies*

The Bitter Lesson

2019, Rich Sutton — one of the founders of reinforcement learning:

📖 [The Bitter Lesson · Rich Sutton \(2019\)](#): *The biggest lesson that can be read from 70 years of AI research is that general methods that leverage computation are ultimately the most effective, and by a large margin. The ultimate reason for this is **Moore's law**, or rather its generalization of continued exponentially falling cost per unit of computation. ...*



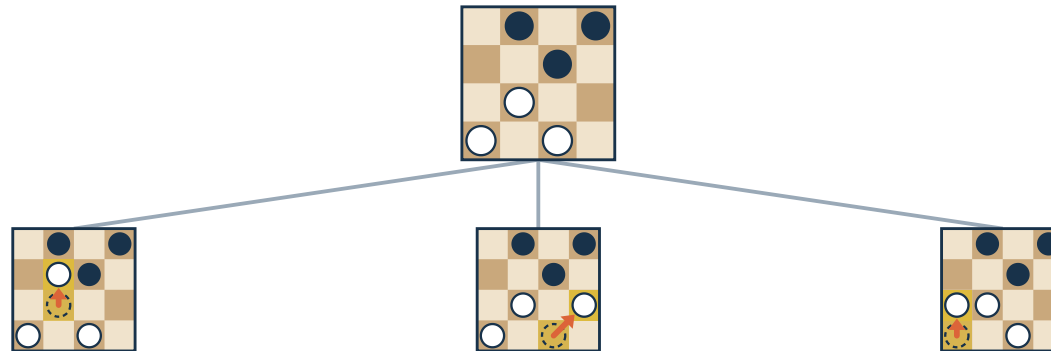
Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

① **our (white) candidate moves**

us ○ white



Case #1: Deep Blue and chess

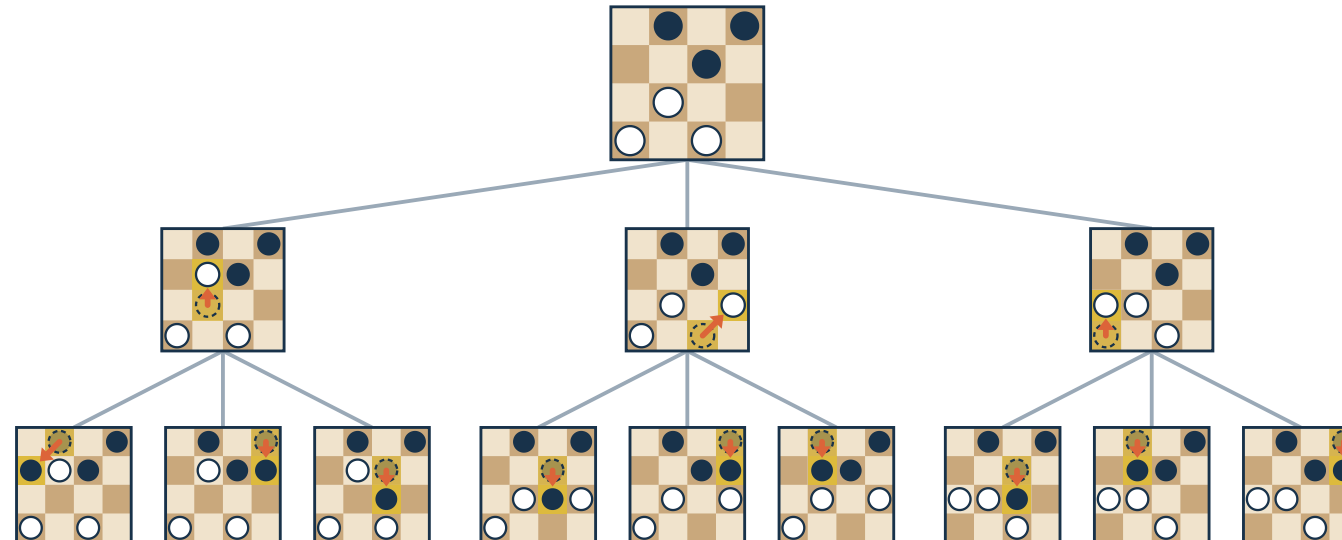
Board-game AI builds on two building blocks:

1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

② the opponent's (black) replies

us ○ white

them ● black



Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

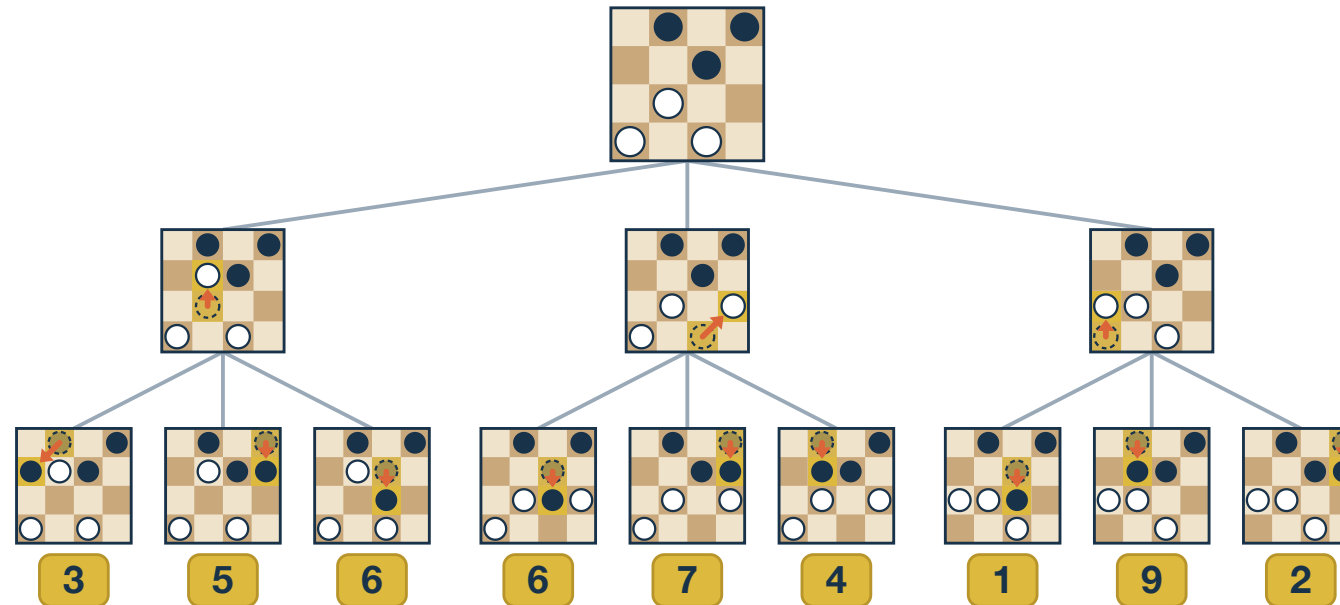
1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

③ **evaluate every leaf**

us ○ white

them ● black

leaves: f



Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

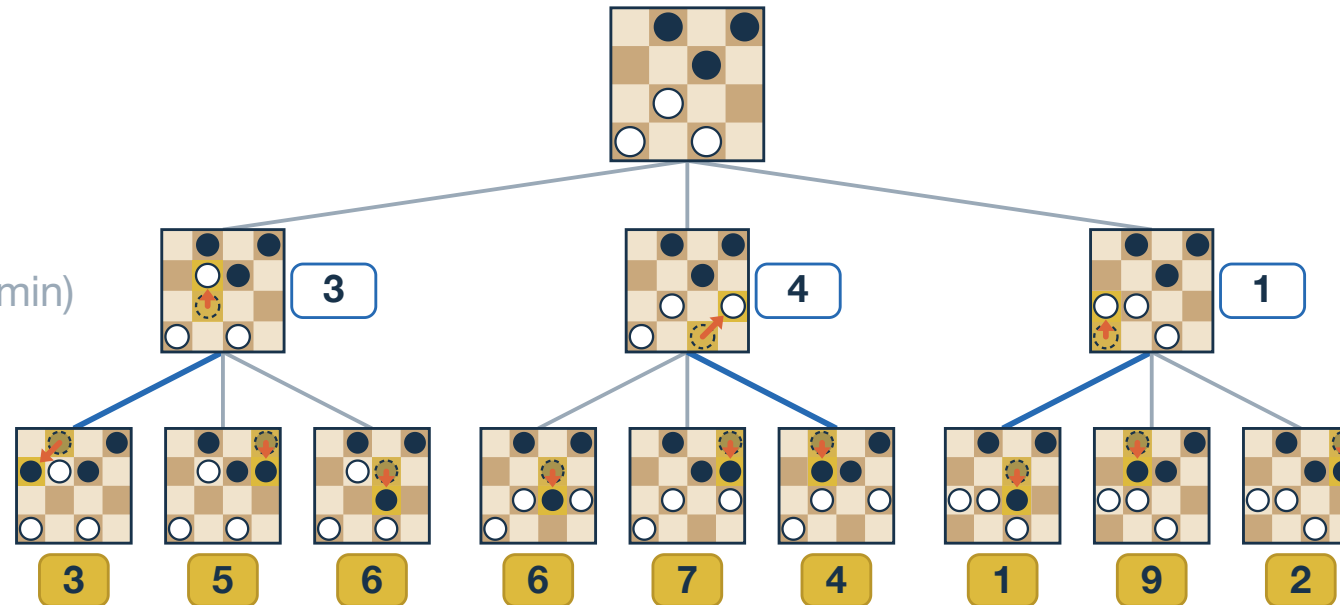
1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

④ the opponent takes min

us ○ white

them ● black (min)

leaves: f



Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

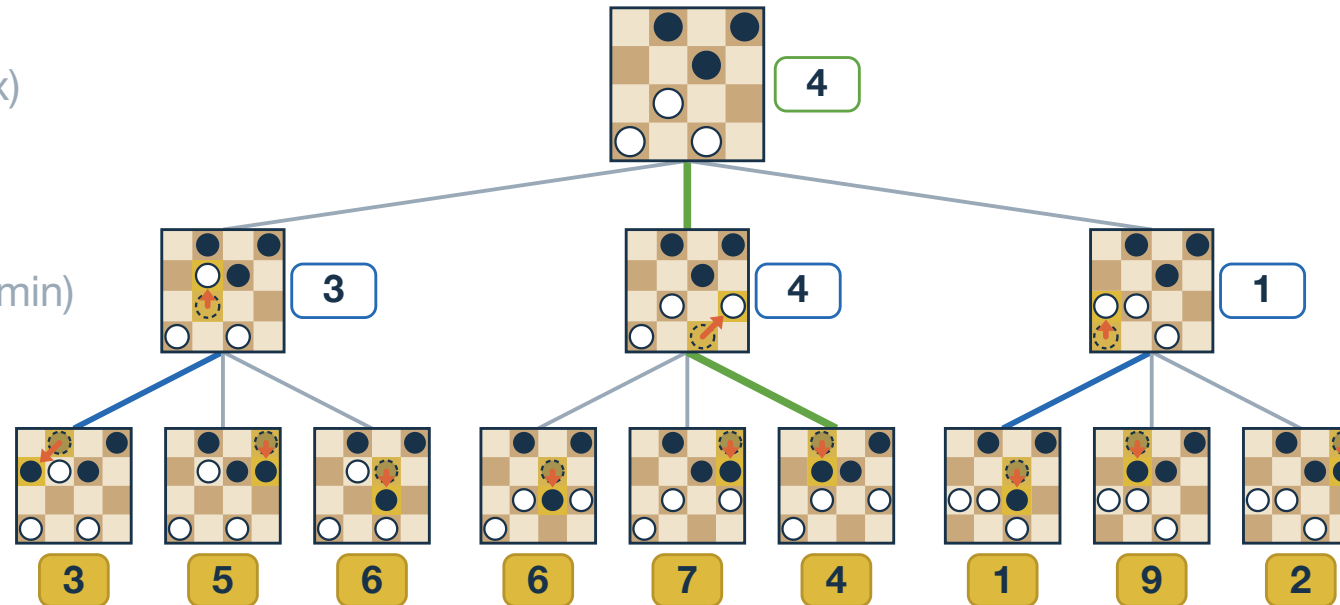
1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

⑤ **we take max**

us ○ white (max)

them ● black (min)

leaves: f



Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

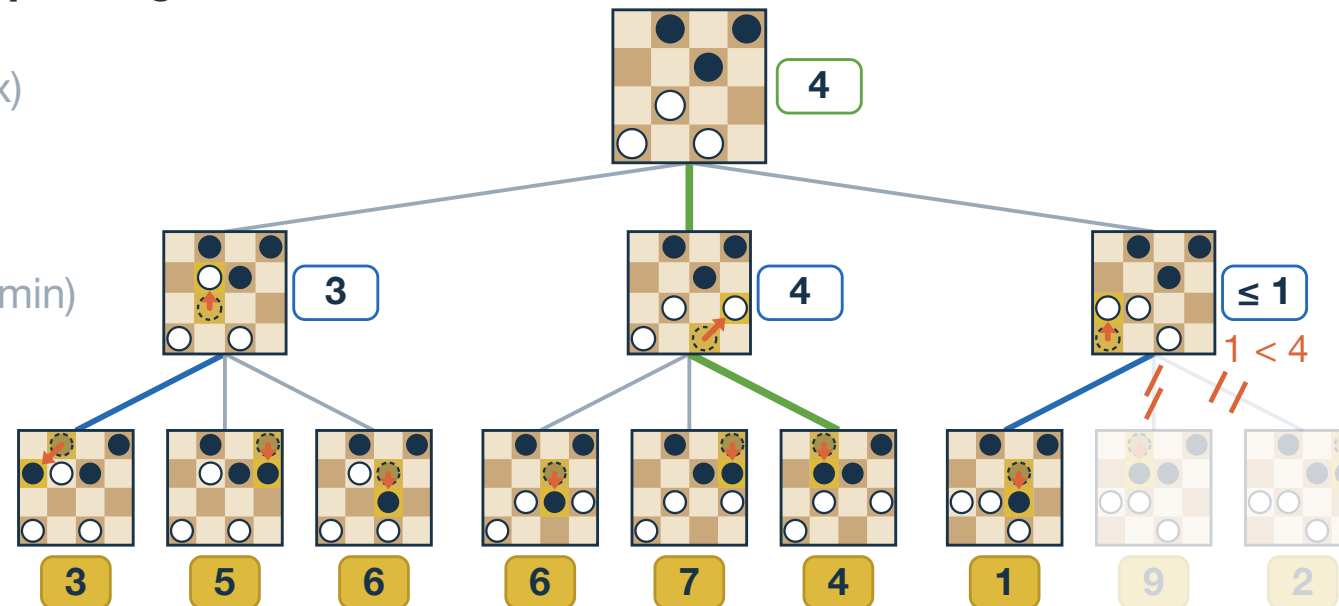
1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

⑥ alpha-beta pruning

us ○ white (max)

them ● black (min)

leaves: f



Case #1: Deep Blue and chess

Board-game AI builds on two building blocks:

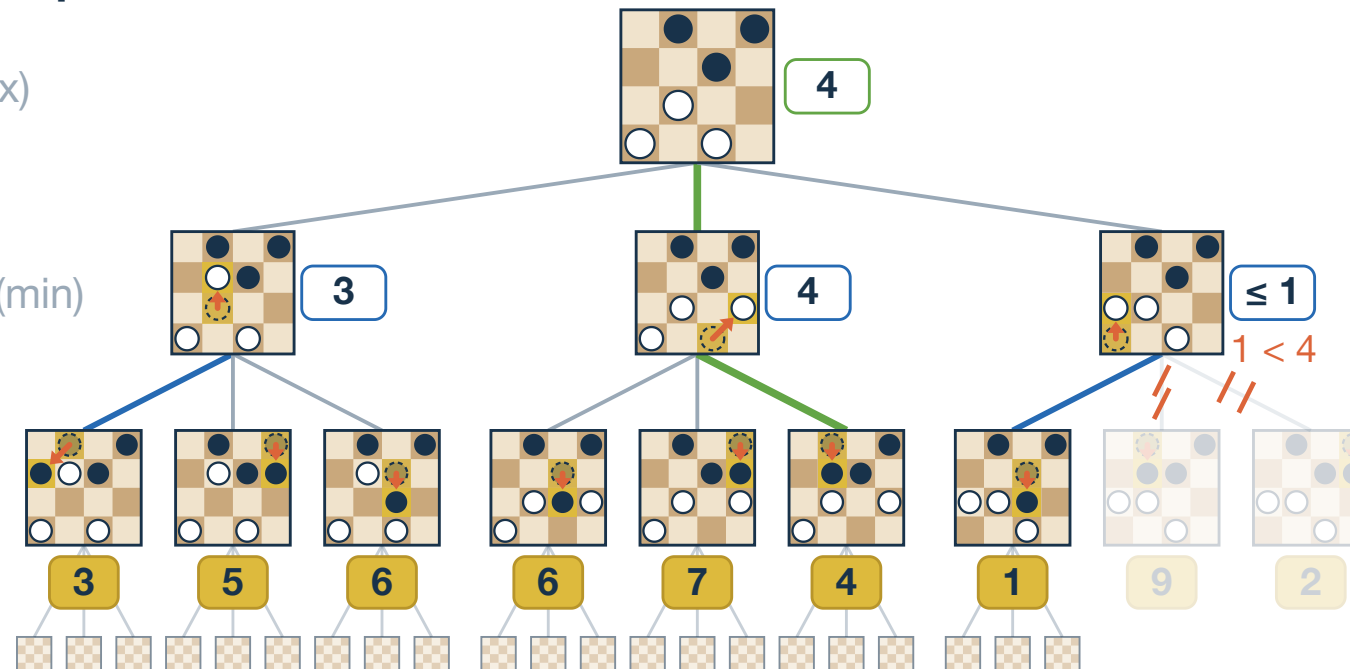
1. **Evaluation function** — score any position without playing to the end (Shannon, 1950);
2. **Search** — **minimax** on the game tree, plus **alpha-beta pruning**, to search the move with the highest score (**human-designed at the DeepBlue time**)

⑦ one level deeper

us ○ white (max)

them ● black (min)

leaves: f



Case #1: Deep Blue — good ELO by scaling deep search

“In computer chess, the methods that defeated the world champion, Kasparov, in 1997, were based on massive, deep search.” — Sutton, The Bitter Lesson



IBM Deep Blue (the machine)

Abstract

Deep Blue is the chess machine that defeated then-reigning World Chess Champion Garry Kasparov in a six-game match in 1997. There were a number of factors that contributed to this success, including:

- a single-chip chess search engine,
- a massively parallel system with multiple levels of parallelism,
- a strong emphasis on search extensions,
- a complex evaluation function, and
- effective use of a Grandmaster game database.

This paper describes the Deep Blue system, and gives some of the rationale that went into the design decisions behind Deep Blue. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Computer chess; Game tree search; Parallel search; Selective search; Search extensions; Evaluation function

Paper abstract: single-chip search engine • massive parallelism • search extensions

¹ Figure: [Deep Blue](#) (James the photographer, CC BY 2.0); abstract from Campbell, Hoane, Hsu, *Deep Blue*, Artificial Intelligence 134 (2002)

Case #2: AlphaGo

Go is **far harder than chess** — the search space is too large:

- a 19×19 board, about **250** moves per position (chess: ~ 35), about **2×10^{170}** legal positions (chess: $\sim 10^{47}$) — **more than atoms in the universe.**¹



*AlphaGo vs. Lee Sedol, March 2016*²

¹ Legal position count: [Tromp, 2016](#)

² Figure: DeepMind × Google

Case #2: AlphaGo vs. AlphaGo Zero

AlphaGo: still search, but **learned score function**

One year later, AlphaGo Zero: **self-play** only,
from random moves

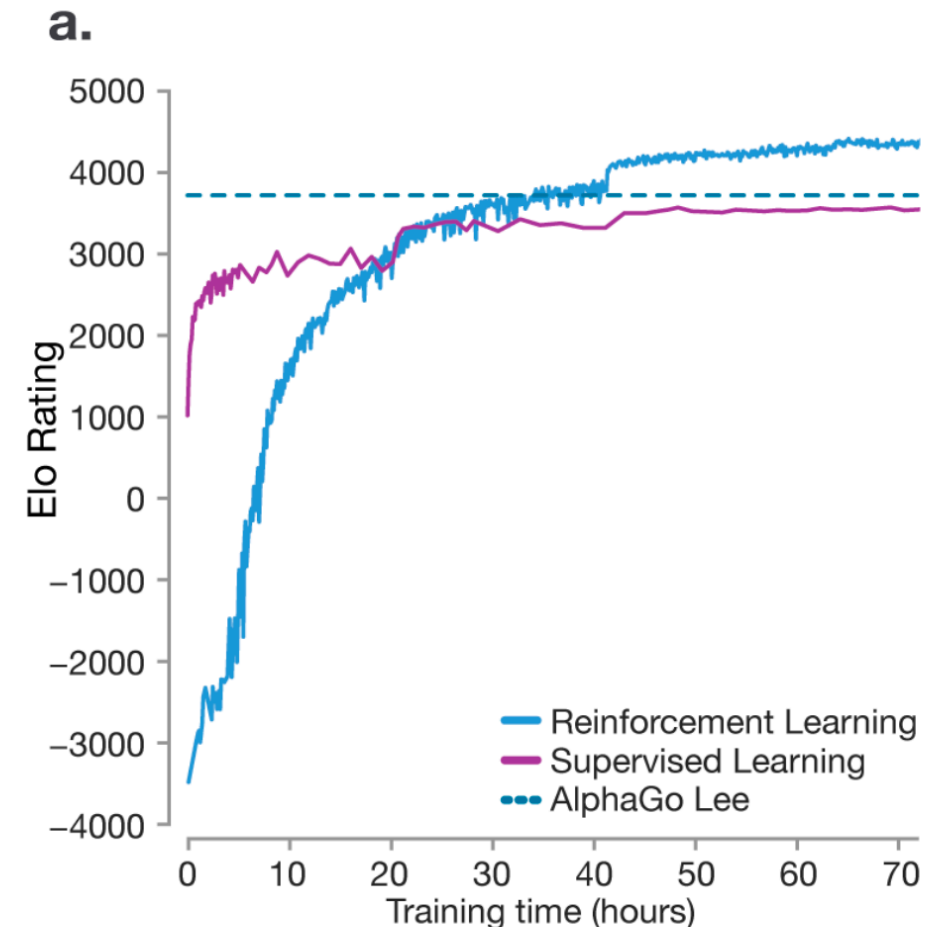
- Right: Elo rating vs. training hours; blue = self-play RL, purple = supervised on human games, dashed = AlphaGo Lee

Mastering the game of Go without human knowledge¹

[David Silver](#) , [Julian Schrittwieser](#), [Karen Simonyan](#), [Ioannis Antonoglou](#), [Aja Huang](#), [Arthur Guez](#), [Thomas Hubert](#), [Lucas Baker](#), [Matthew Lai](#), [Adrian Bolton](#), [Yutian Chen](#), [Timothy Lillicrap](#), [Fan Hui](#), [Laurent Sifre](#), [George van den Driessche](#), [Thore Graepel](#) & [Demis Hassabis](#)

[Nature](#) **550**, 354–359 (2017) | [Cite this article](#)

 [Save article](#)

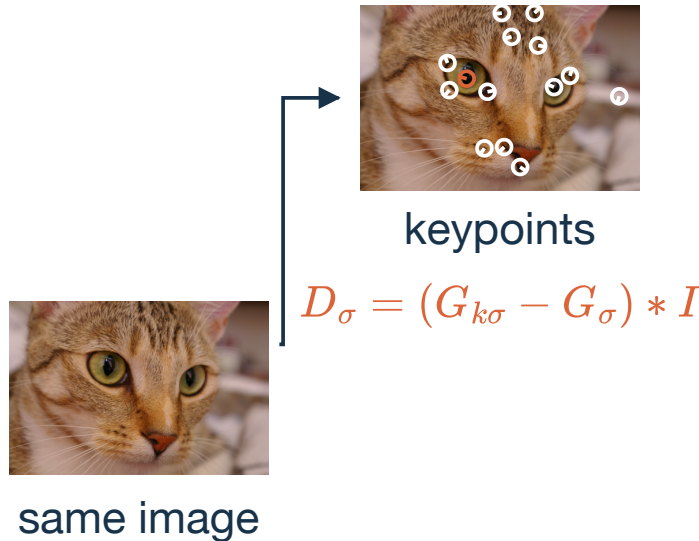


¹ Figure: Silver et al., [Mastering the Game of Go without Human Knowledge](#), *Nature* 550 (2017); right figure is Fig. 3a

Case #3: computer vision — learned features > hand-crafted features

1

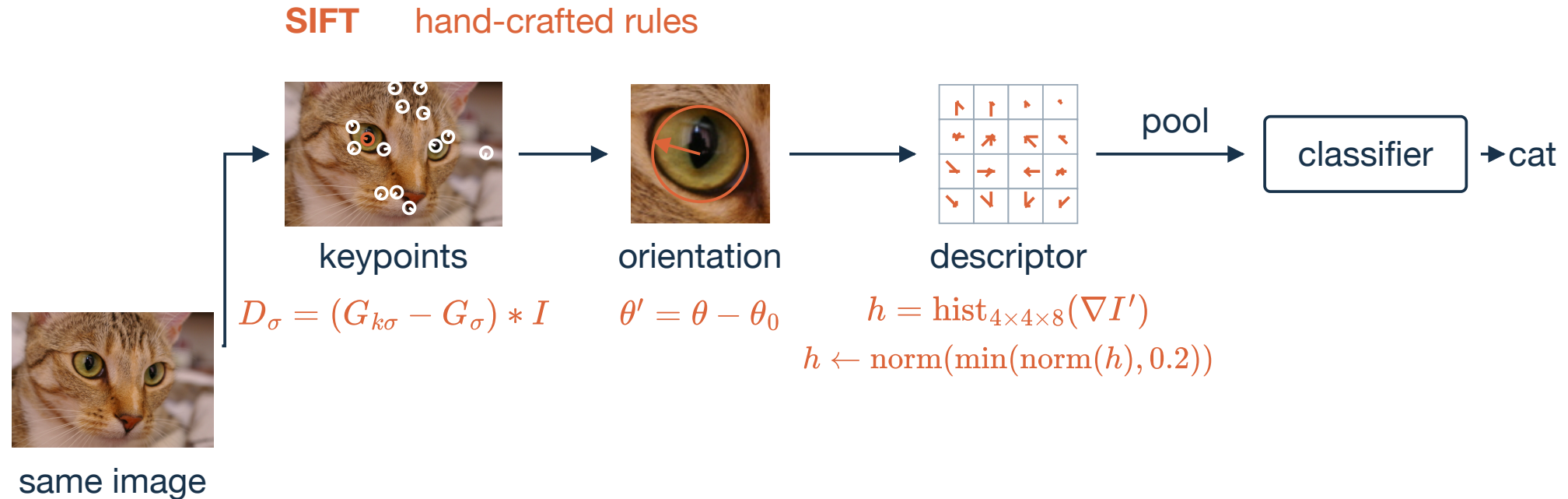
SIFT hand-crafted rules



¹ SIFT: [Lowe, 2004](#), selected stages, compact notation; CNN weights: [CS231n](#). Photo: [S. van der Walt, CC0](#). G: Gaussian; norm: L2 normalization; Θ : network parameters. Paths are schematic.

Case #3: computer vision — learned features > hand-crafted features

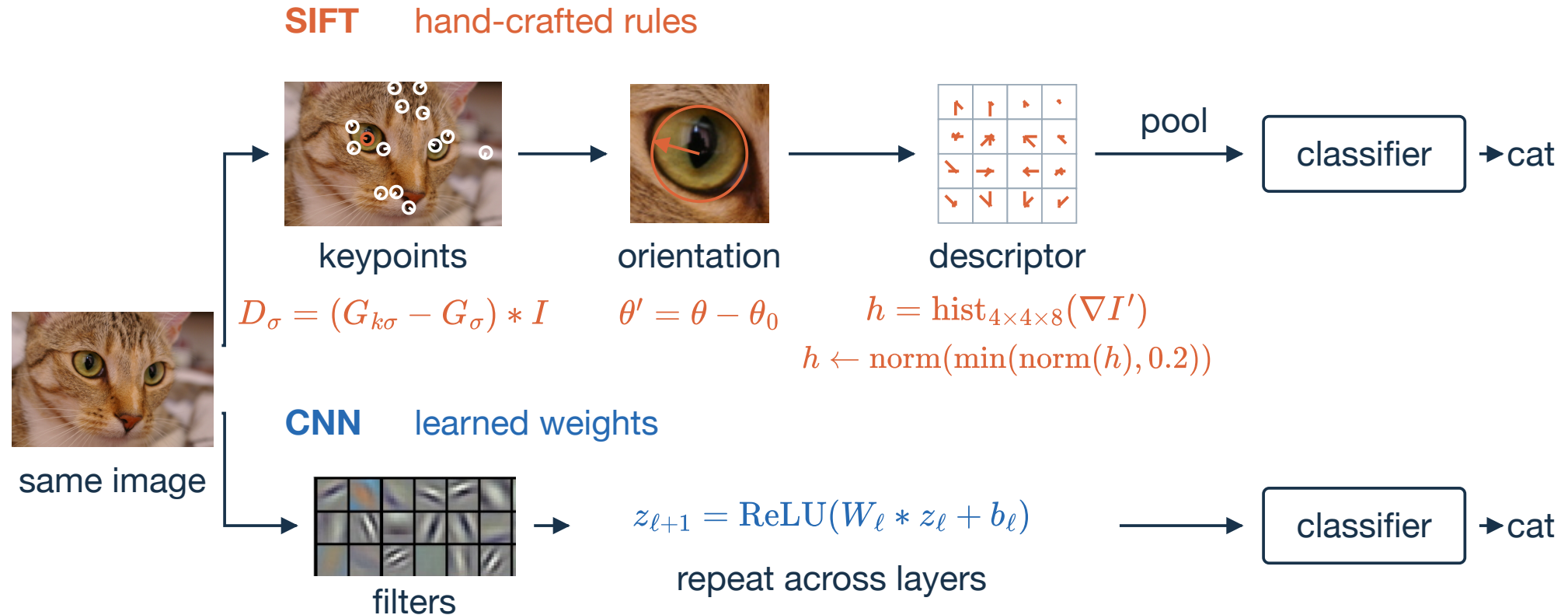
1



¹ SIFT: [Lowe, 2004](#), selected stages, compact notation; CNN weights: [CS231n](#). Photo: [S. van der Walt, CC0](#). G: Gaussian; norm: L2 normalization; Θ : network parameters. Paths are schematic.

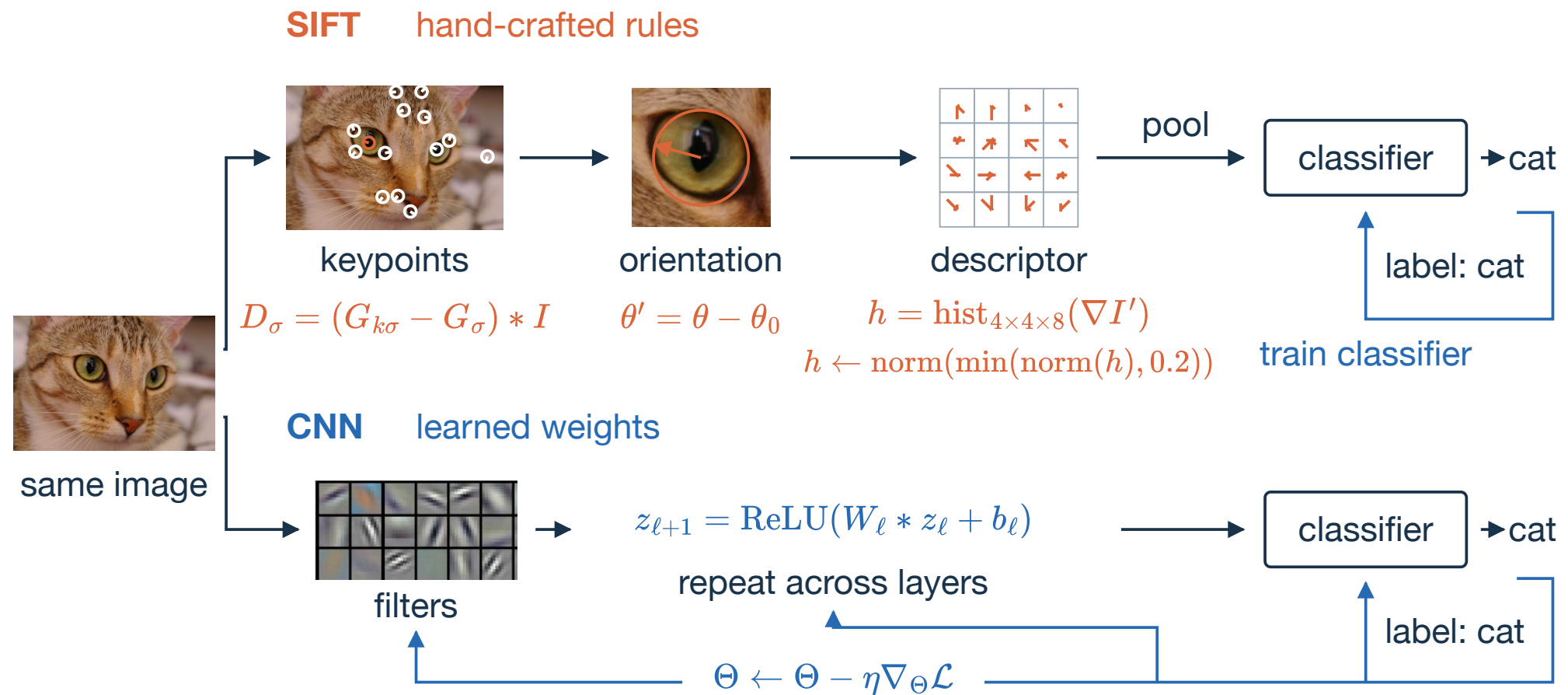
Case #3: computer vision – learned features > hand-crafted features

1



¹ SIFT: [Lowe, 2004](#), selected stages, compact notation; CNN weights: [CS231n](#). Photo: [S. van der Walt, CC0](#). G: Gaussian; norm: L2 normalization; Θ : network parameters. Paths are schematic.

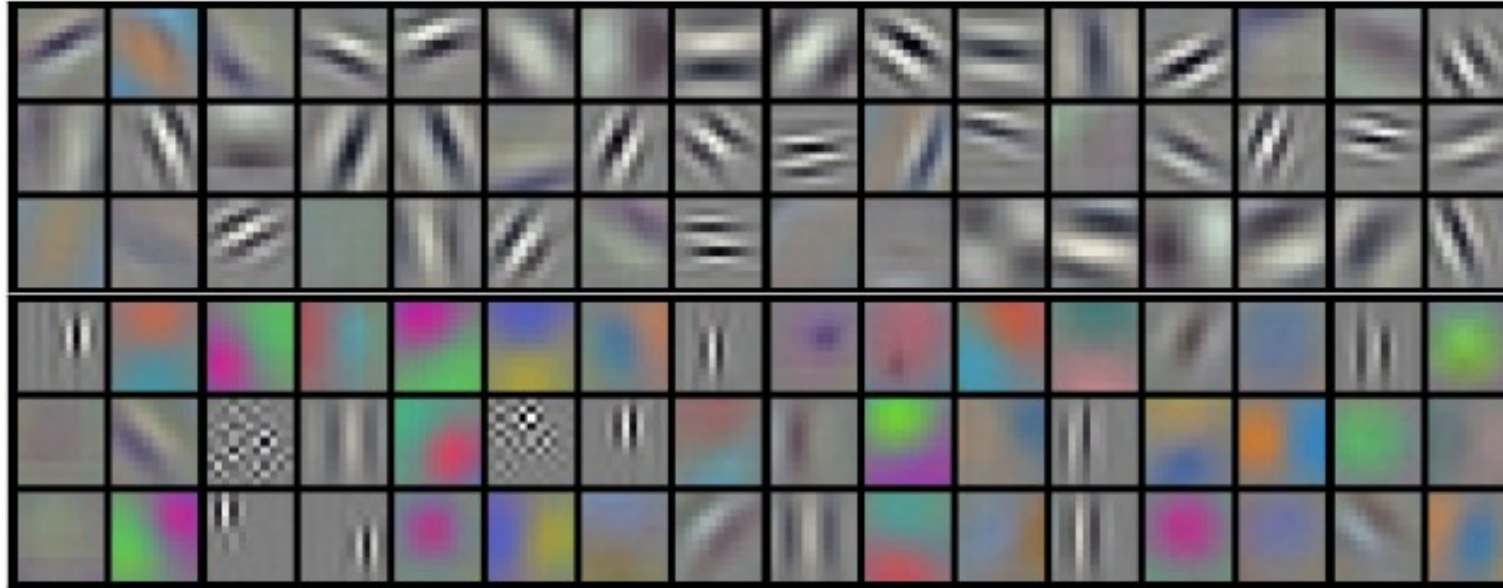
Case #3: computer vision – learned features > hand-crafted features



SIFT: more hand-crafted rules. CNN: repeated operators with **learned weights**.

¹ SIFT: [Lowe, 2004](#), selected stages, compact notation; CNN weights: [CS231n](#). Photo: [S. van der Walt, CC0](#). G: Gaussian; norm: L2 normalization; Θ : network parameters. Paths are schematic.

Case #3: computer vision — learned features > hand-crafted features



AlexNet's first layer after training: edge and color filters

People chose the network structure. **Training learned these filter weights.**

2012 ImageNet top-5 error: **26.2%** (runner-up) vs. **15.3%** (AlexNet).¹

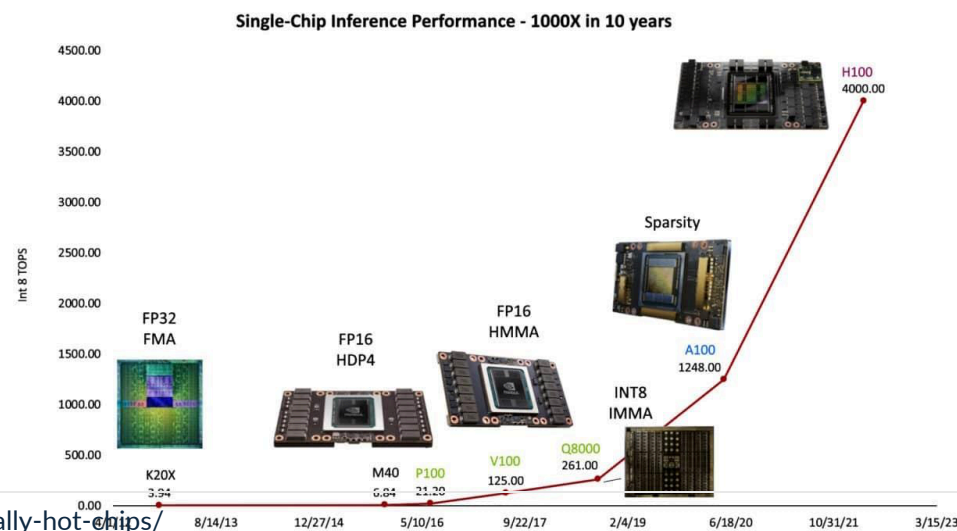
¹ Krizhevsky et al., [ImageNet Classification with Deep Convolutional Neural Networks](#), 2012, abstract and Fig. 3.

Connecting the Bitter Lesson to exponential scaling: Moore's Law

📖 Recap The Bitter Lesson: The ultimate reason for this is **Moore's law**, or rather its generalization of continued exponentially falling cost per unit of computation.* ...

We keep hearing that **Moore's Law (compute grows exponentially over time) is dead** — but is it?

- At the scale of a single chip: yes
- At the scale of a "distributed system": not yet! (though it is non-trivial, and is getting harder)



¹ Source: <https://blogs.nvidia.com/blog/huangs-law-daily-hot-chips/>

Wait a minute: why does more compute bring more intelligence?

This reasoning has a hidden assumption: we have found **a method whose intelligence grows with compute**

- The Bitter Lesson calls such a method a **meta-method**



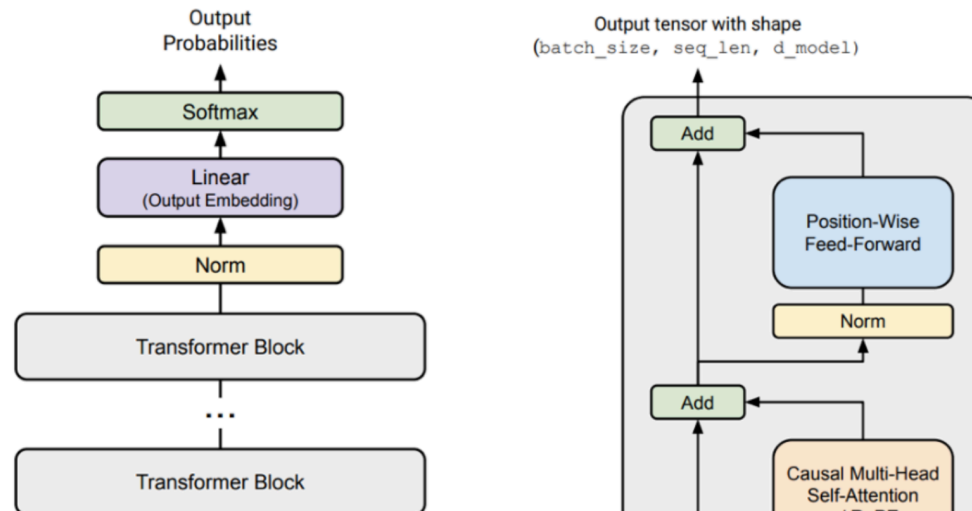
The Bitter Lesson: One thing that should be learned from the bitter lesson is the great power of general purpose methods, of methods that continue to scale with increased computation even as the available computation becomes very great. The two methods that seem to scale arbitrarily in this way are **search** and **learning**.

The second general point to be learned from the bitter lesson is that the actual contents of minds are tremendously, irredeemably complex; we should stop trying to find simple ways to think about the contents of minds ... instead we should build in only the **meta-methods** that can find and capture this arbitrary complexity. ... We want AI agents that **can discover like we can, not which contain what we have discovered**. Building in our discoveries only makes it harder to see how the discovering process can be done.

A possible meta method: The large language model

From a systems view, a model, it is: a set of parameters + a fixed forward computation

- Input: a sequence of token ids
- Output: at each position, scores for "which token comes next" — the logits



```
logits = model(tokens)           # inference: run one forward pass
next_token = logits[:, -1].argmax(dim=-1)
```

Question: how much computation to train on / compute a token?

A simple empirical formula: $\text{FLOP} = 6 \times B \times W \times S$ (we will see why it is later lectures, or you can just deduce yourself with the model :)

- **B**: how many requests one step computes (learns) over
- **W**: the model's parameter count
- **S**: the average length of a request
- **6**: forward takes 2 operations, backward takes 4

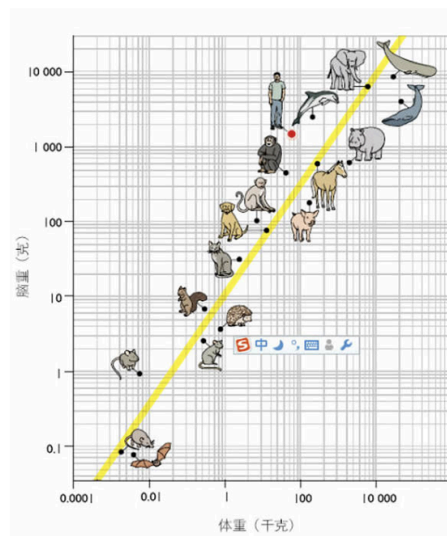
 Assume "intelligence" ≈ "computing more": Then we can link "intelligence" to "compute"! Moore's Law lets us compute more every year — does that mean intelligence climbs year by year?

Question: along which dimension do we scale?

B X W X S gives us three dimensions to scale

- B: scale the amount of training data (e.g., the model reads more and more books)
- W: scale the model size (the model gets bigger and bigger)
- S: scale the input length (the model's context gets longer and longer)

1



Scale W

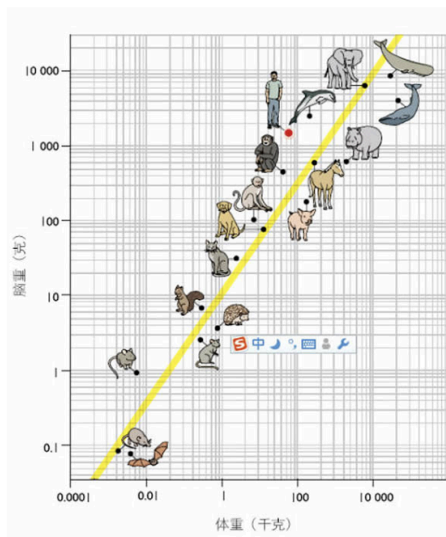
¹ Sources: [brain vs. body mass figure](#); Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS 2022.

Question: along which dimension do we scale?

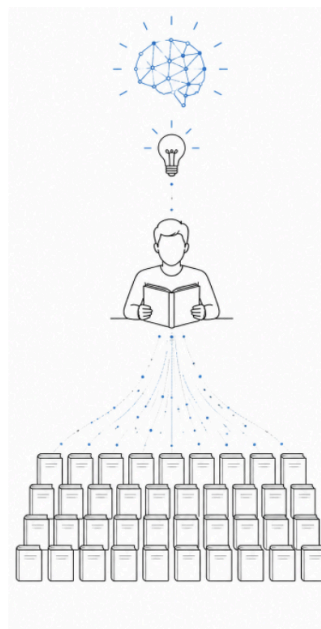
B X W X S gives us three dimensions to scale

- B: scale the amount of training data (e.g., the model reads more and more books)
- W: scale the model size (the model gets bigger and bigger)
- S: scale the input length (the model's context gets longer and longer)

1



Scale W



Scale B

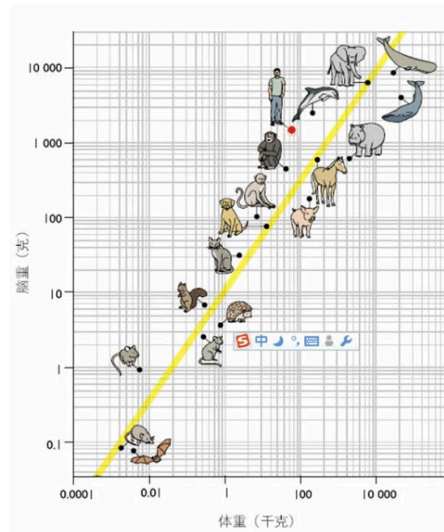
¹ Sources: [brain vs. body mass figure](#); Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS 2022.

Question: along which dimension do we scale?

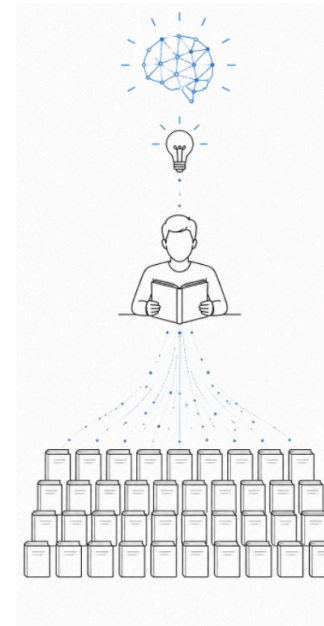
B X W X S gives us three dimensions to scale

- B: scale the amount of training data (e.g., the model reads more and more books)
- W: scale the model size (the model gets bigger and bigger)
- S: scale the input length (the model's context gets longer and longer)

1



Scale W



Scale B

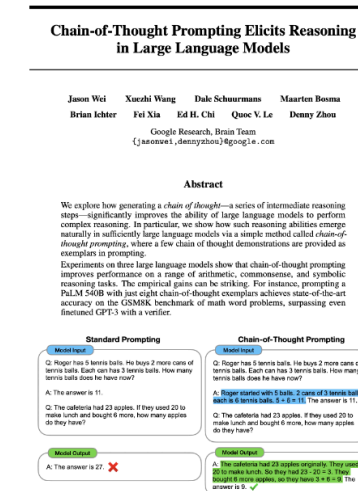
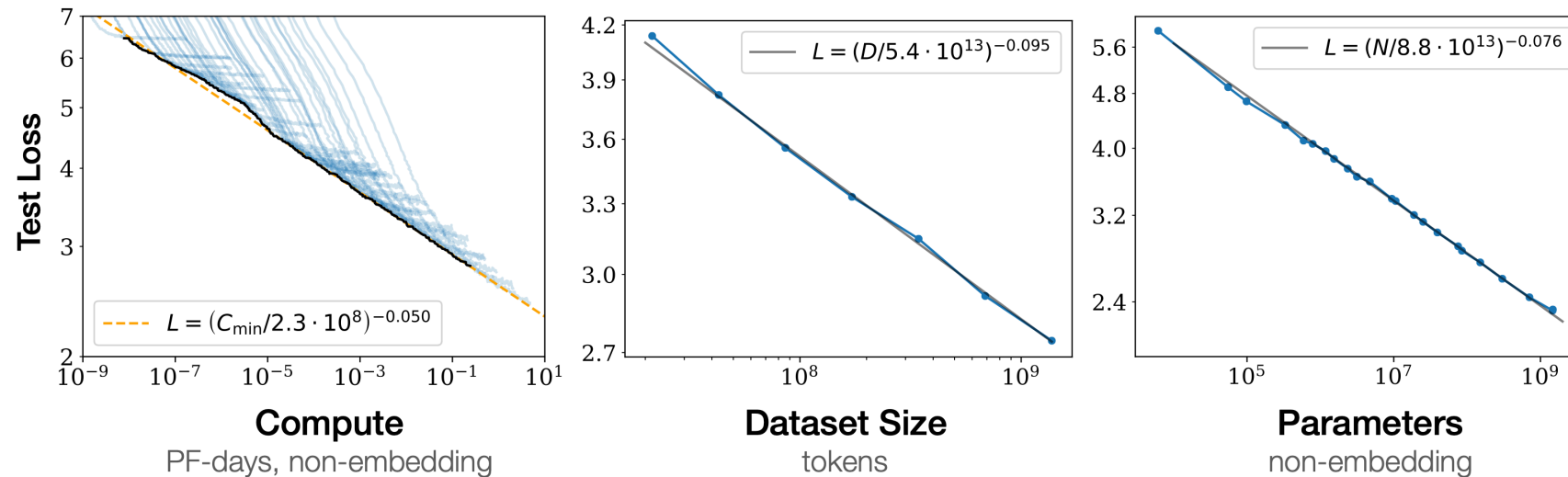


Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

¹ Sources: [brain vs. body mass figure](#); Wei et al., *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*, NeurIPS 2022.

The Scaling law

Scaling Laws for Neural Language Models: how loss scales can be predicted by a power law!



And this law was measured on **Transformers**: the Transformer seems to have found its own scaling law

The Chinchilla Law

Problem: the scaling law says scaling works, but not which direction one training run should scale

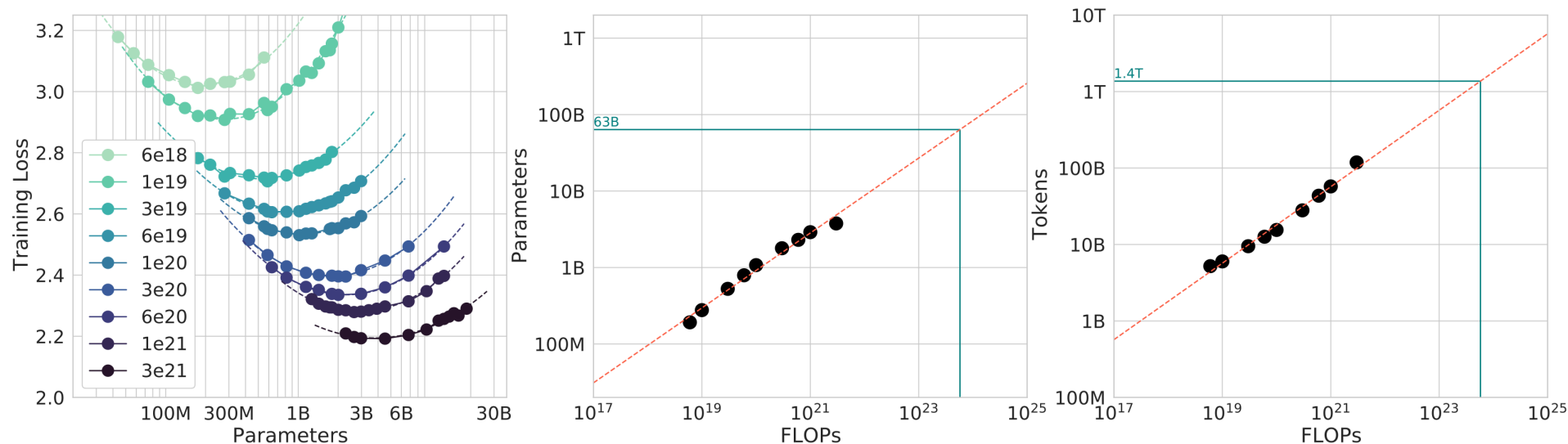
- Compute grows year over year; but within one release cycle (a few months), the compute budget barely grows

The model trainer must answer: given a compute budget, should I spend it on B, or others?

- Ideal: use a small scale data to predict at a larger scale

The Chinchilla Law

Training Compute-Optimal Large Language Models: fix the FLOPs, vary the model size, and find the valley



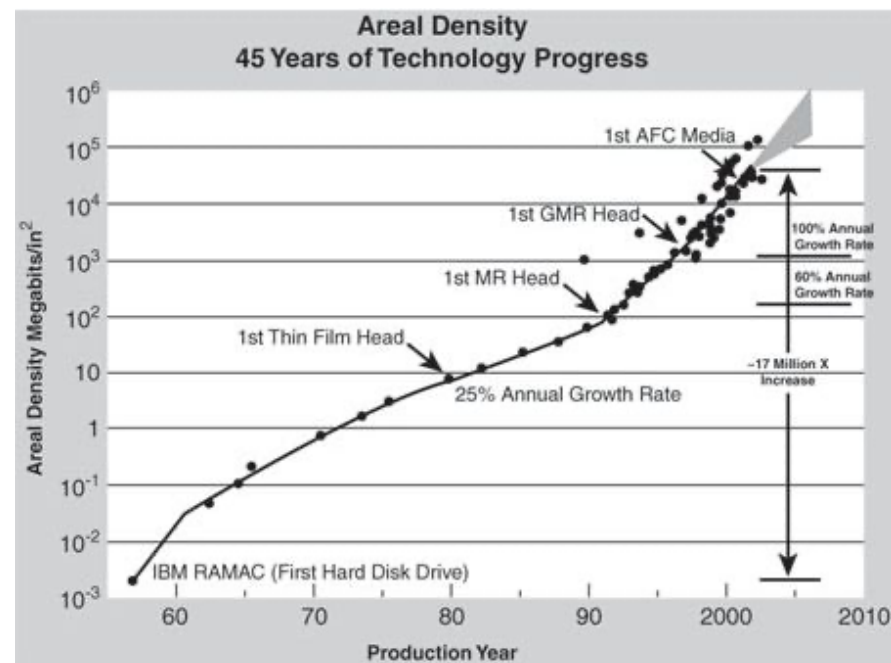
$\hat{L}(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$: too few parameters costs one term, too few tokens costs the other

(Aside) Recall: why must the model be a language model?

We have abundant training data (B) for language (at least by now)

- General data: the Internet
- Domain data: code (e.g., GitHub)

Growing storage capacity lets us store enough language knowledge (not as trivial as it sounds — we will see later)¹



¹ Source: <https://www.primaryio.com/hard-disk-trends/>

Step one of realizing the scaling law: large-scale digital systems

AI has so much training data not because the AI era suddenly produced it, but because "digital services" accumulated it over decades

- A **digital service**: carry out what "people" want to do in a digital way + store it in **computing systems**.
- Afterward, we can **enjoy the More's law**!

Scaling a digital system boils down to doing three pieces well:

1. **Consistency**: is the data/computing results **correct**?
2. **Reliability**: can things **operate under failures**?
3. **Performance**: is it fast enough?

Case study: suppose we build an "all-in-one" app

The app offers the following:

- Users can socialize (WeChat/QQ/Slack-like)
- Users can shop + trade (Taobao/Amazon-like)
- We train + deploy our own model to help users shop + handle social tasks



Think: What would the system behind this app look like?

How was it done traditionally?

The LAMP architecture:

Operating system:

- L is **L**inux (in OS class)

Serving the requests: web server

- A is **A**ppache, Nginx (in ICS)

Serving the data: file system & DB

- M is **M**ysql & inode file system (in CSE)

Displaying the page: HTML

- P is **P**HP (in Web class)

> 帮我用 LAMP 的方式搭建一个网站

• Skill(superpowers:brainstorming)
 | Successfully loaded skill

- 使用 superpowers:brainstorming 来帮你规划 LAMP 网站的需求和架构。

让我先了解一下你的需求。我看到你在新项目目录中，所以我们从头开始。

首先，请告诉我：你想用 LAMP 搭建什么类型的网站？

比如说，是：

- 个人博客或内容管理网站
- 企业门户网站
- 电商平台
- 社区论坛
- 其他特定的应用

这会帮助我理解你的基本需求。

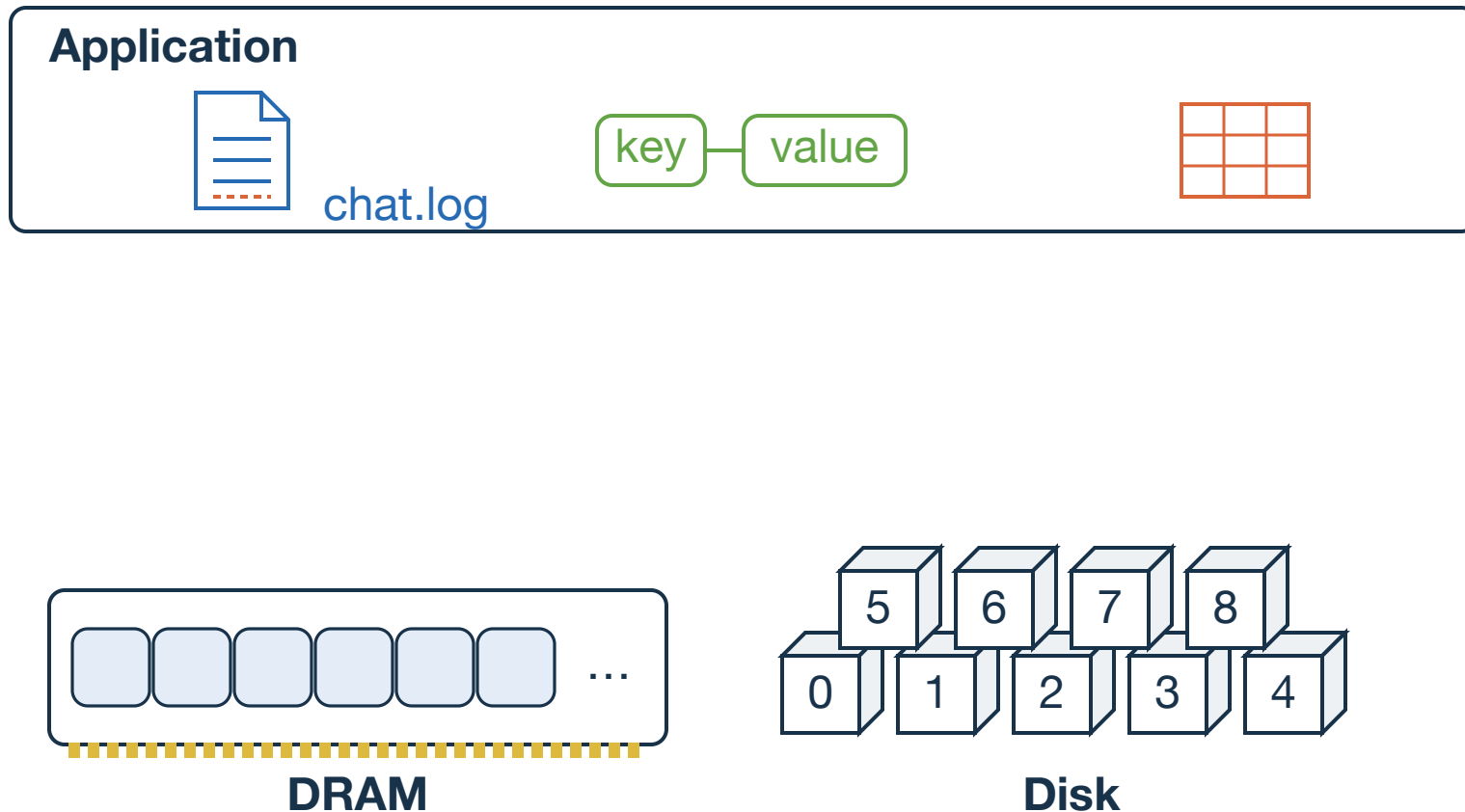
* Churned for 12s

> █

Why LAMP? We need abstractions to simplify development

What the hardware gives: DRAM is a **raw byte array**, a disk is **a pile of blocks**

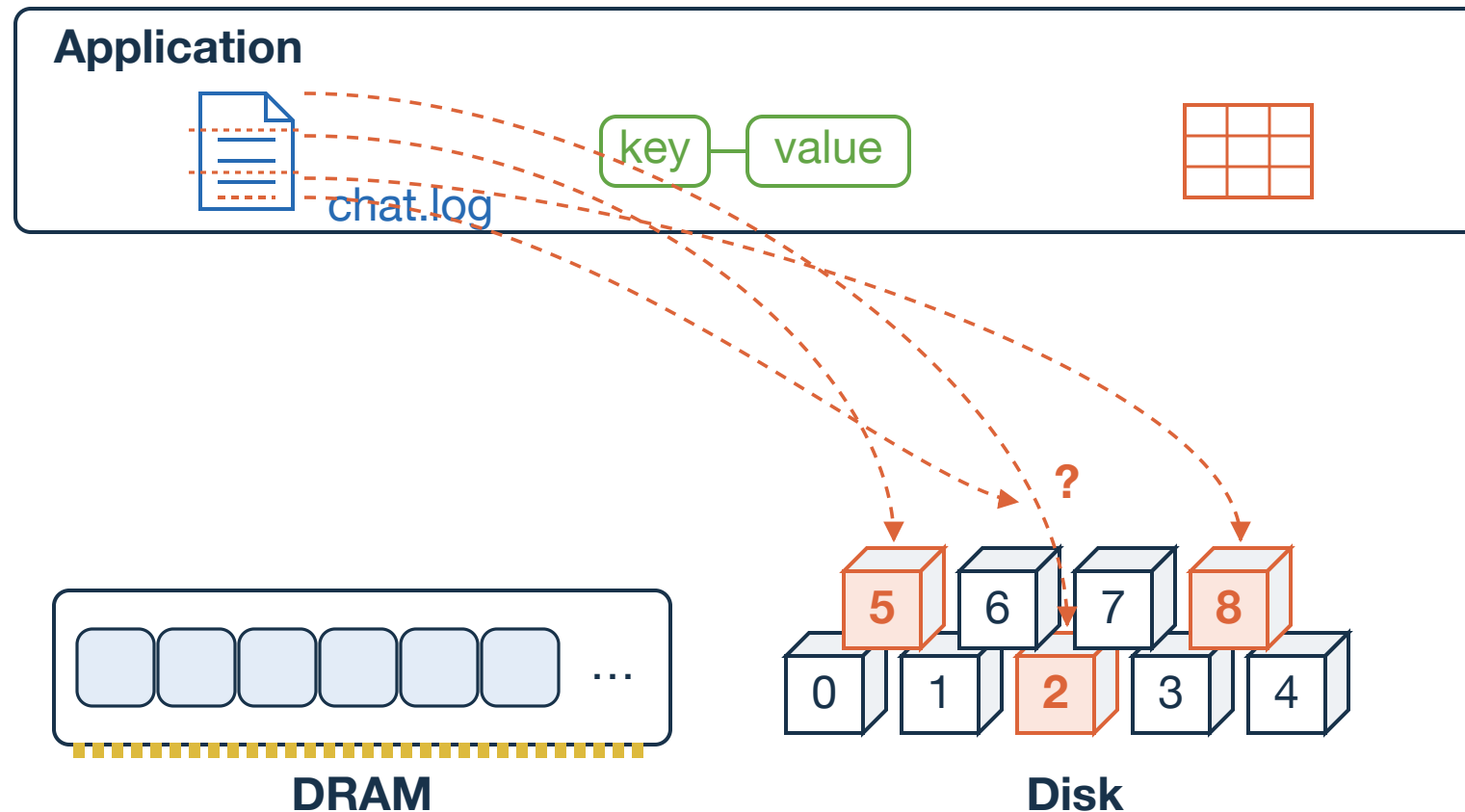
- What the app has: a growing file `chat.log`, key-value pairs, tables



Why LAMP? We need abstractions to simplify development

Problem #1: the shapes do not match

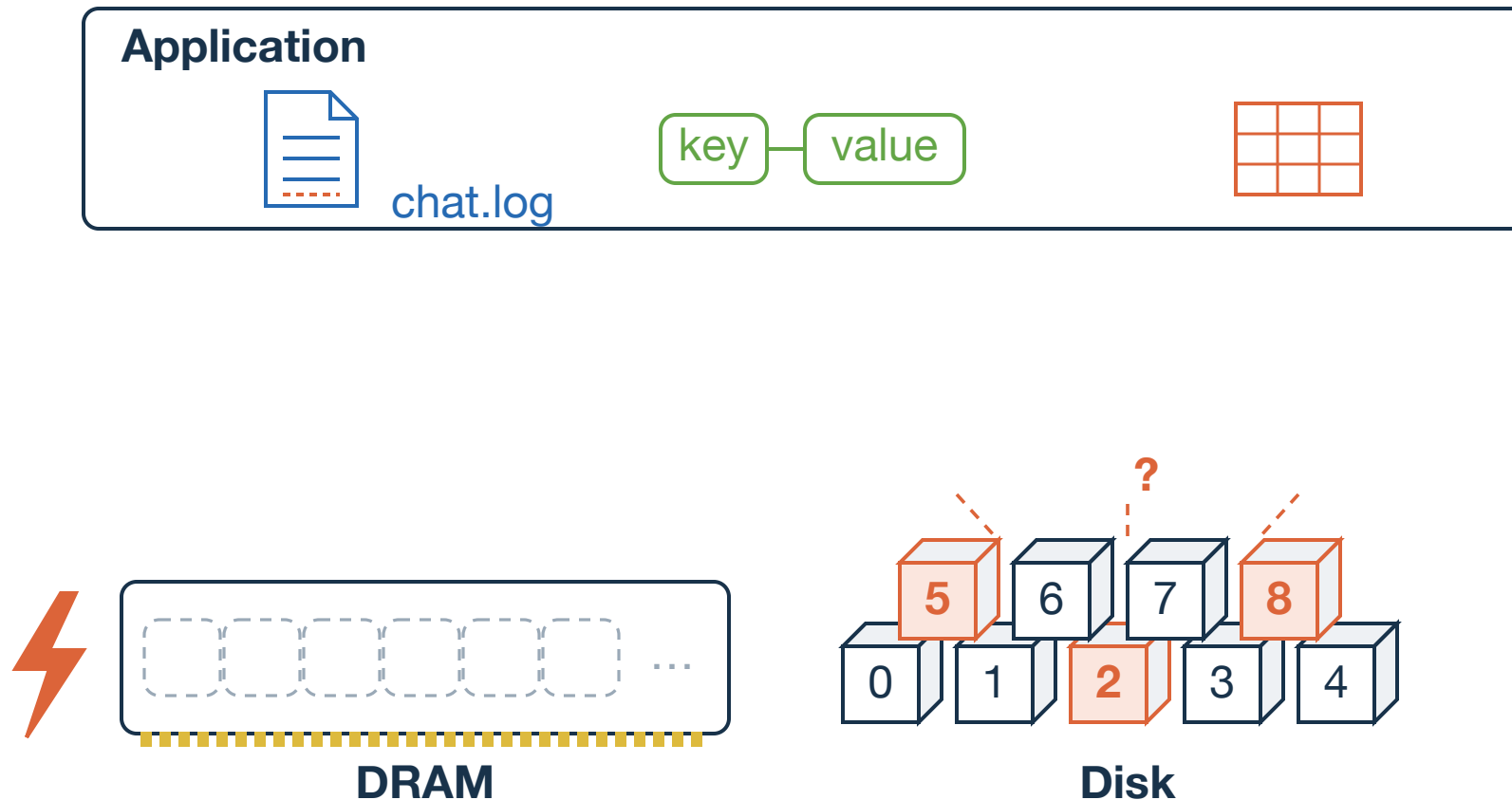
- A file is stored in fixed-size blocks: which blocks? Where does the byte go?



Why LAMP? We need abstractions to simplify development

Problem #2: persistence

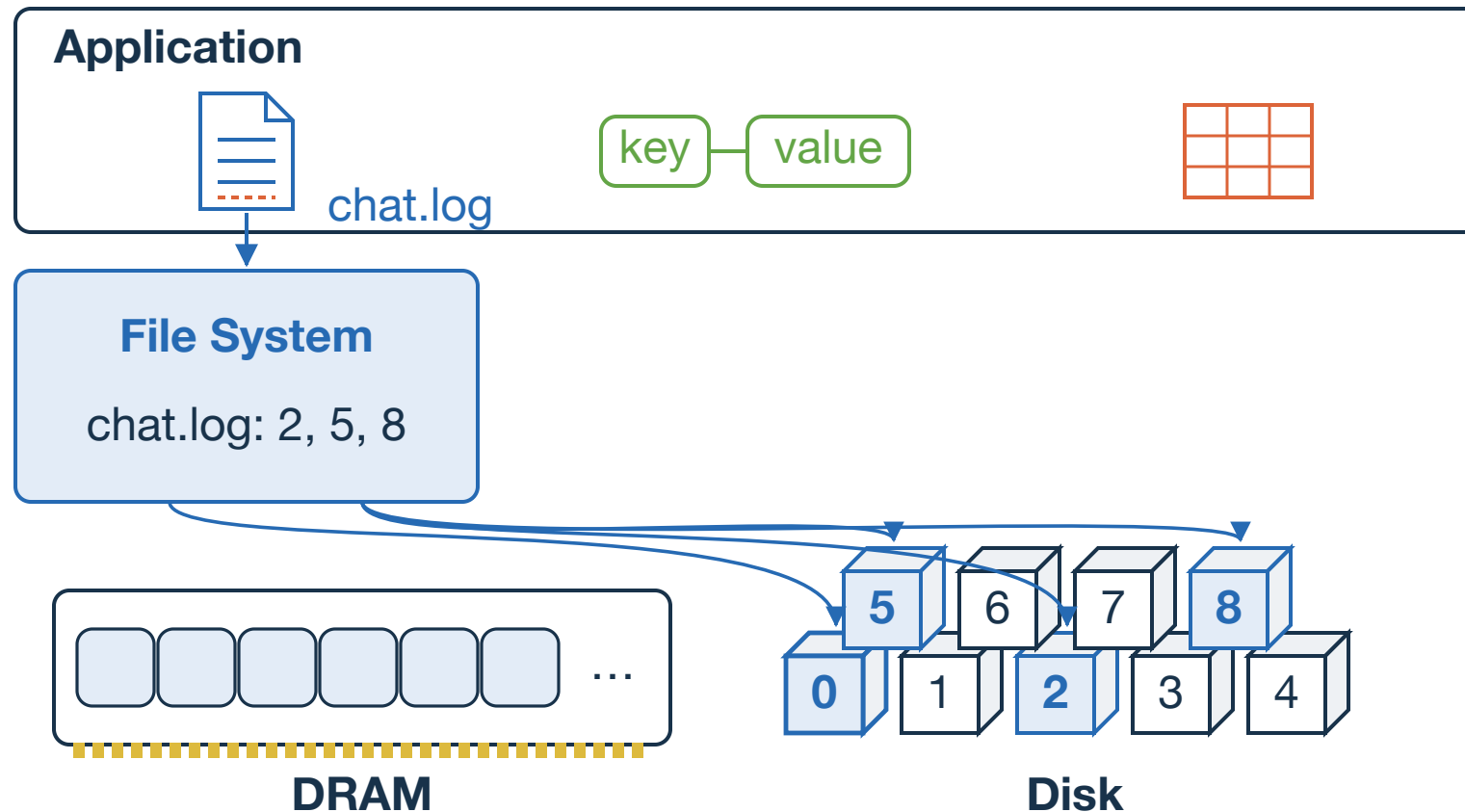
- Power off: DRAM is gone; the blocks remain, but which of them are `chat.log`?



Why LAMP? We need abstractions to simplify development

Solution: build a layer in between, the **file system**

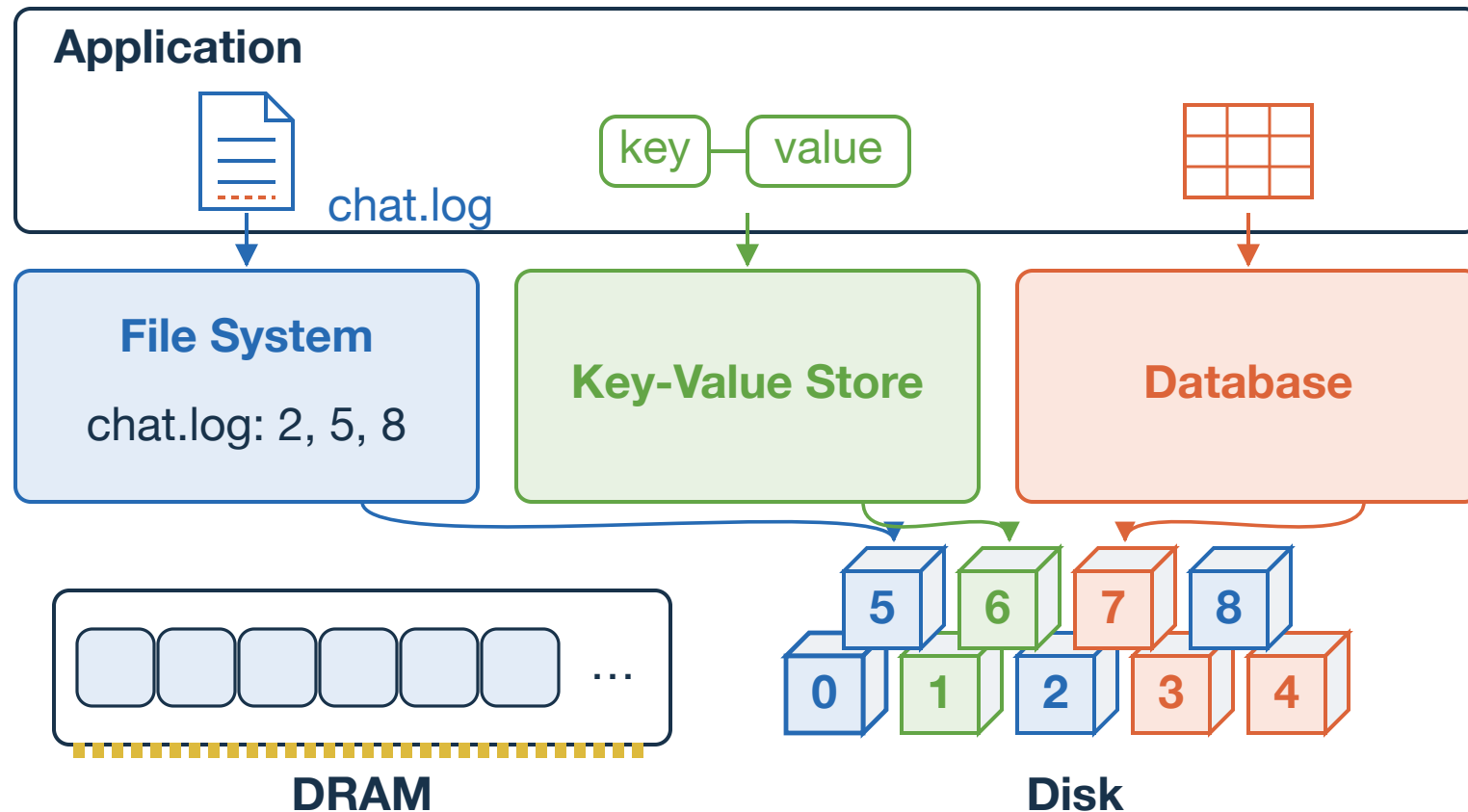
- It keeps name → blocks for every file, and with many more features



Why LAMP? We need abstractions to simplify development

So we need "file systems", "key-value stores", databases, etc.

- Each provides a useful interface for the targeted applications



Problem: scalability of data storage

Facebook photos, 2010: ~1B uploads/week $\approx$ 60 TB/week $\rightarrow$ new photo data ~8.6 TB/day

A single 40 TB disk is filled in:

- ~4.7 days, or ~1.6 days with 3 $\times$ replication¹

And the number only grows: by 2012 it was >350M uploads/day — ~2.5 $\times$ of 2010

2

Every day people upload more than 350 million photos to Facebook as of December 2012 and view many more in their News Feeds and on their friends' Timelines. Facebook stores these photos on Haystack machines that are optimized to store photos. But there is also a deep and distributed photo-serving stack with many layers of caches that delivers photos to people so they can view them.

The LAMP architecture cannot possibly support storage at this scale

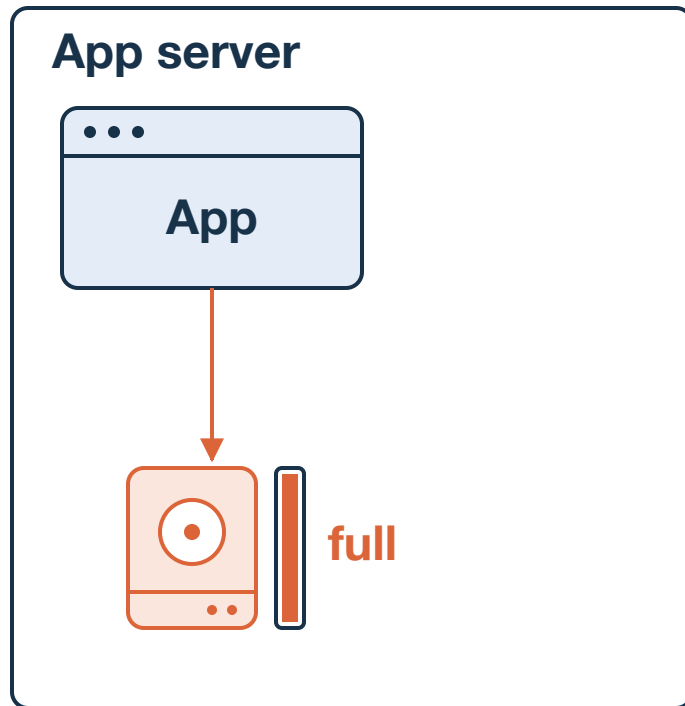
¹ Source: https://www.usenix.org/legacy/event/osdi10/tech/full_papers/Beaver.pdf

² Source: <https://engineering.fb.com/2014/02/27/web/an-analysis-of-facebook-photo-caching-2/>

Solution: scale storage with distributed storage

The app server has one disk, and it can be filled within days

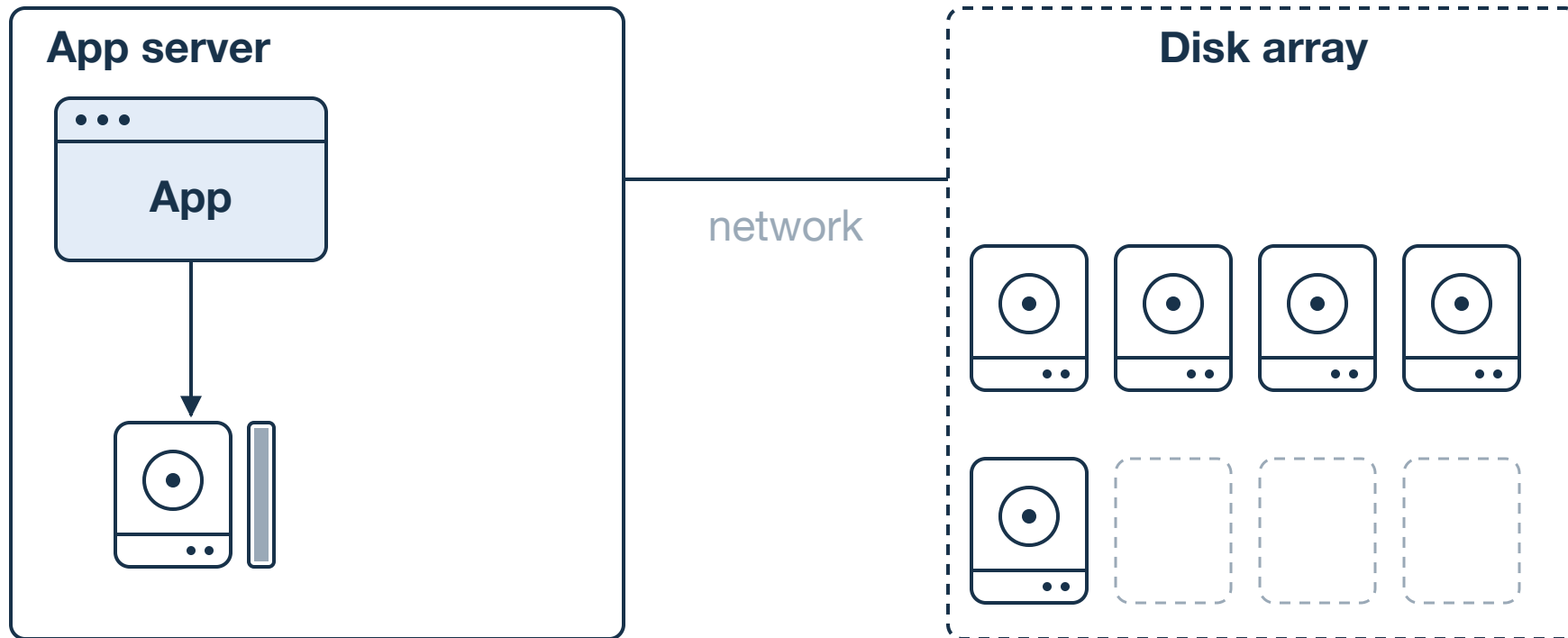
- Therefore, we need a new architecture that **scales** with the data



Solution: scale storage with distributed storage

Disaggregation: Connect many disks together over the network: a **disk array**

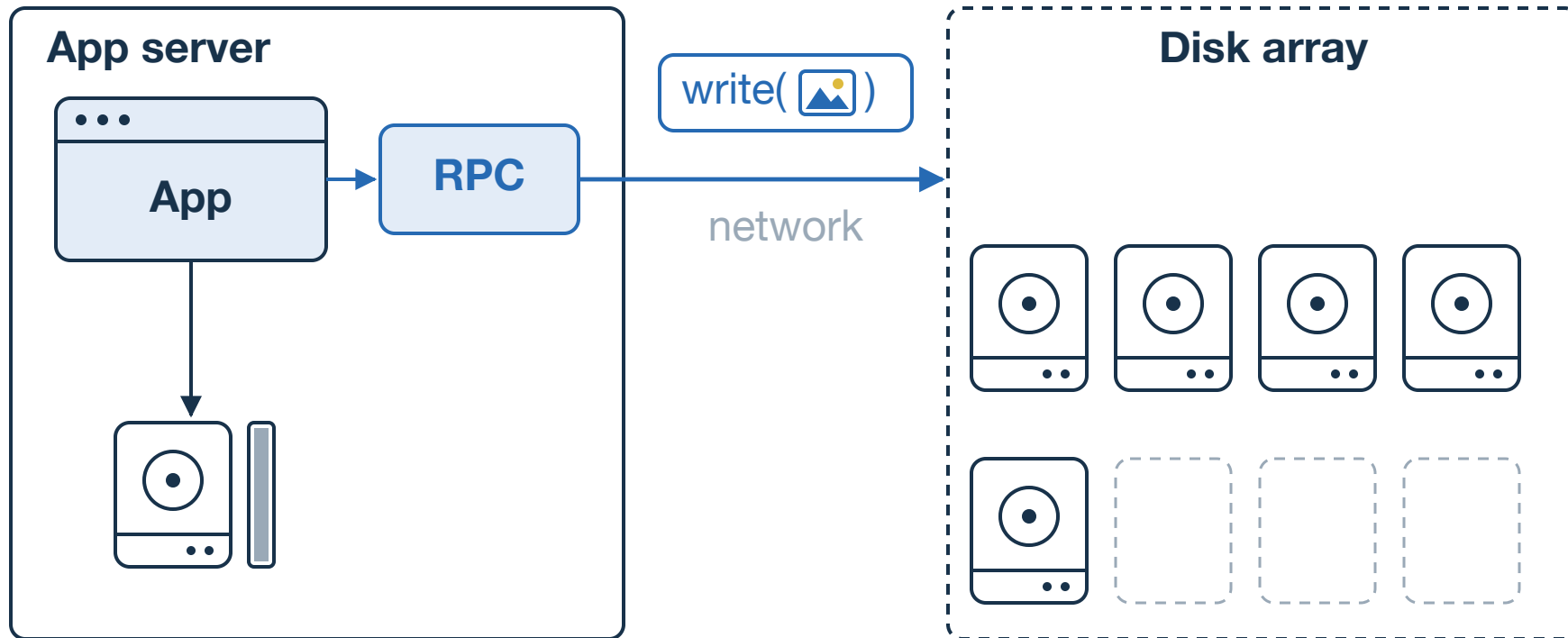
- The disks sit on other machines, and can add as demand increases



Solution: scale storage with distributed storage

The app accesses the array via **RPC** (Remote Procedure Call)

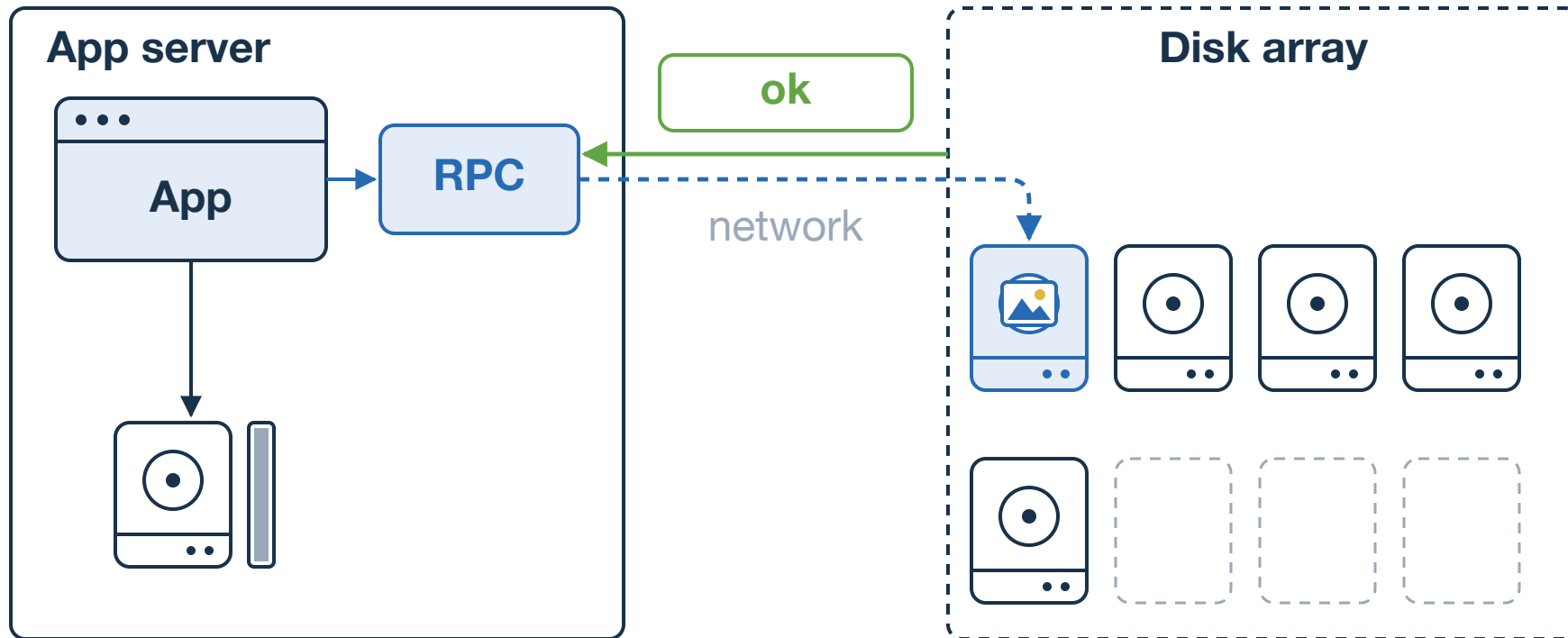
- `write(photo)` looks like a local call, but travels over the network



Solution: scale storage with distributed storage

The array stores the photo on one of its disks and replies

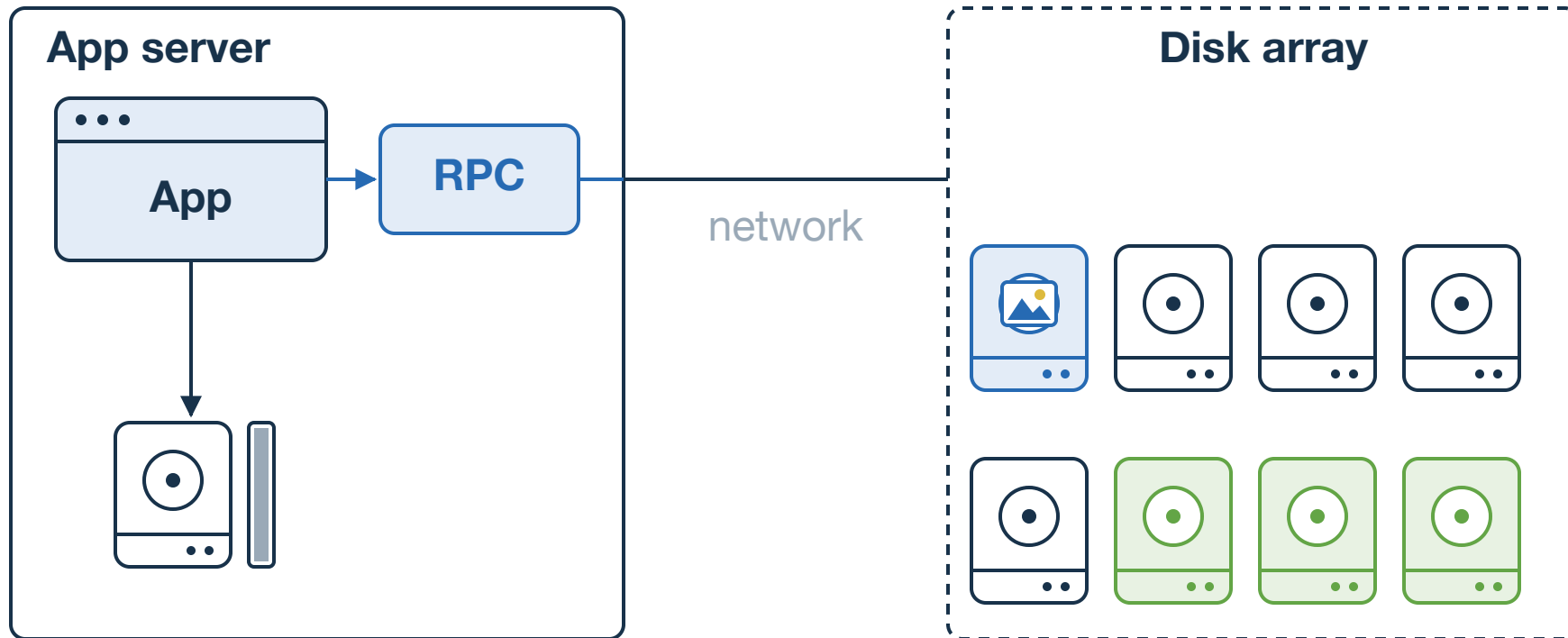
- To the app, the array is just one very large disk



Solution: scale storage with distributed storage

In theory, as long as there is room, capacity can grow without bound

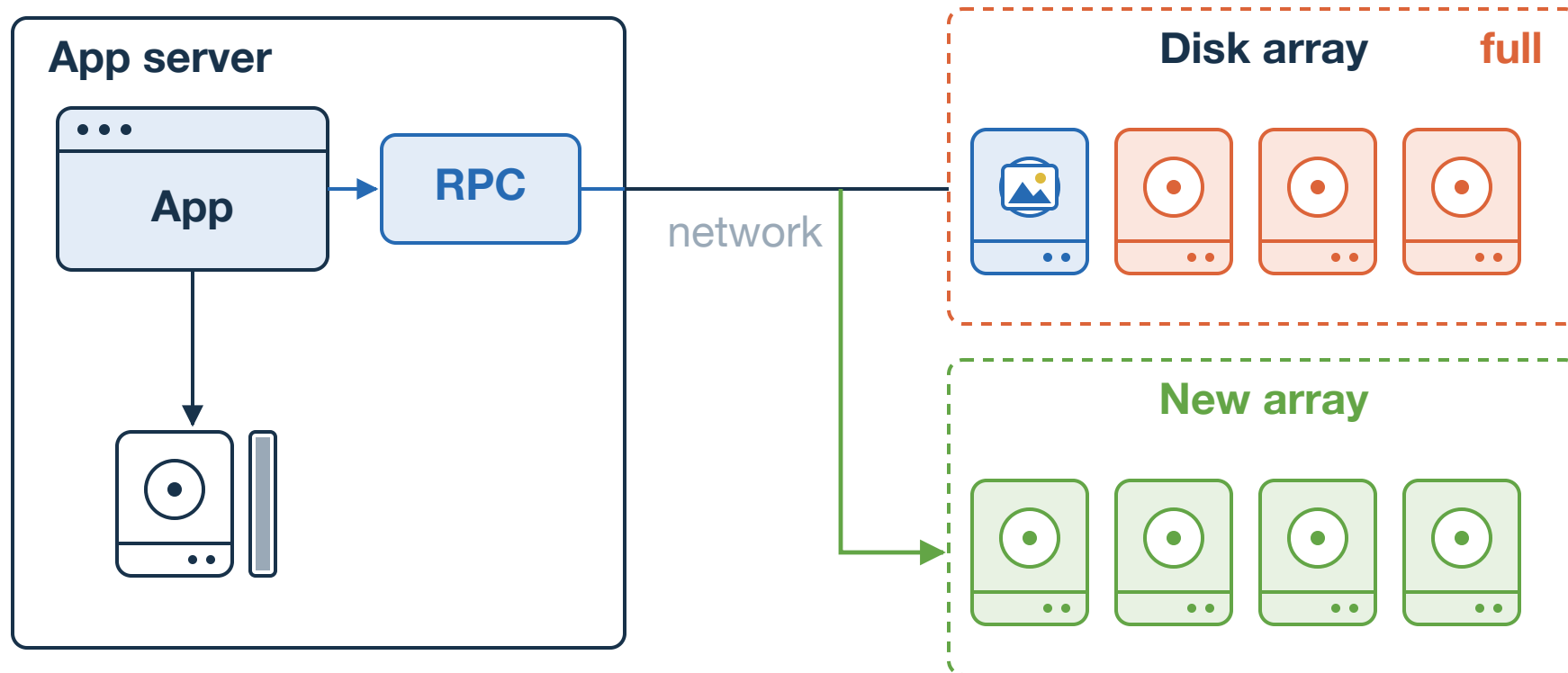
- Full again? Add more disks to the array



Solution: scale storage with distributed storage

Even a full array is not the end: **add a new array**

- Capacity keeps growing, and the app still sees **one big disk**



Scalability challenge: how to shard?

Sharding: given a piece of data (identified by a **key**), which disk do we look on? Some simple (classic) schemes:

- hashing: compute `hash(key) % N`; the result says which disk it goes to

What is wrong with these schemes? Consider **hashing under scaling** (e.g., adding more data):

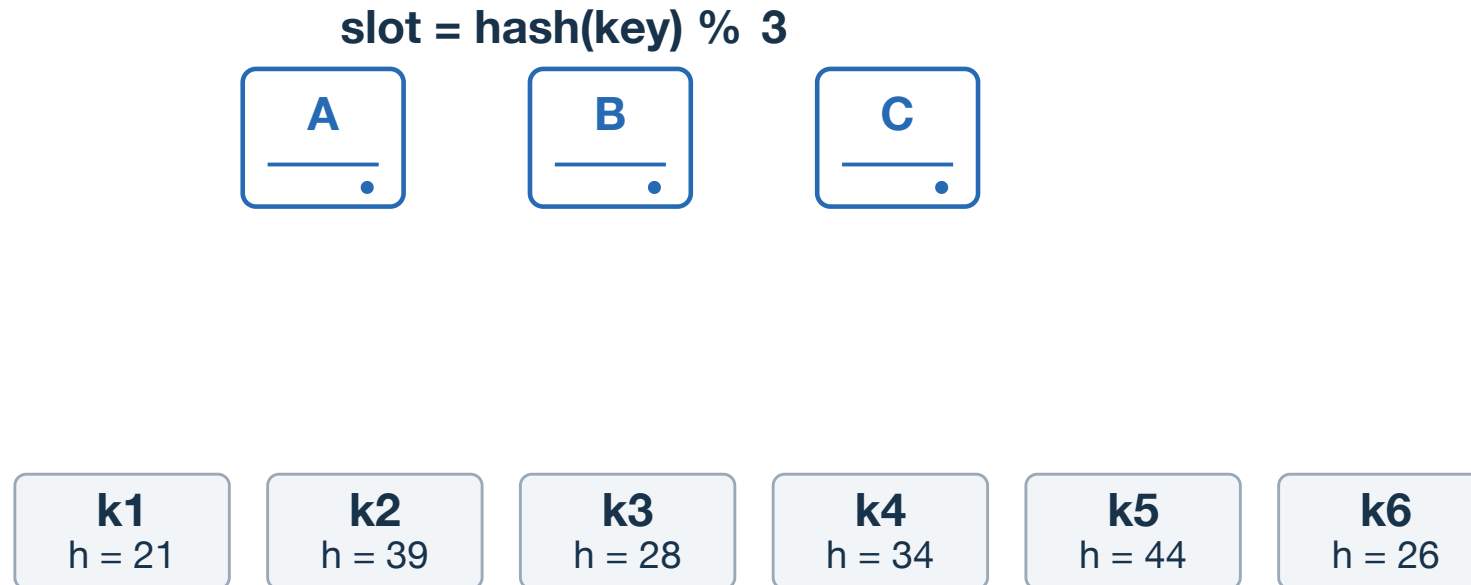
- Suppose we have N disks and D pieces of data, and pick the disk via `hash(key) % N`

What happens if we grow the disks to $N + 1$?

Why naive sharding cannot scale

Six keys on three disks with `slot = hash(key) % N`; then a fourth disk arrives

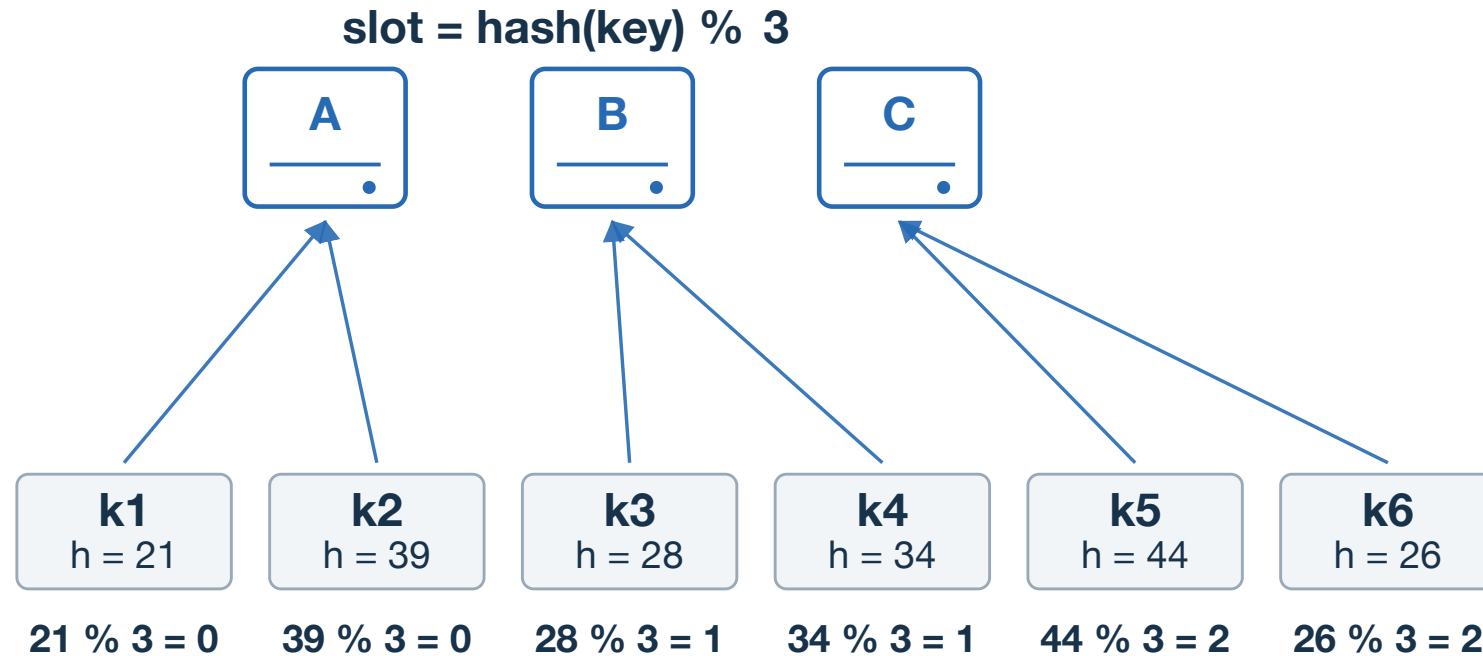
- Growing N changes the divisor, so almost every key gets a different remainder
- A key whose remainder changed has to be physically moved to another disk



Why naive sharding cannot scale

Six keys on three disks with `slot = hash(key) % N`; then a fourth disk arrives

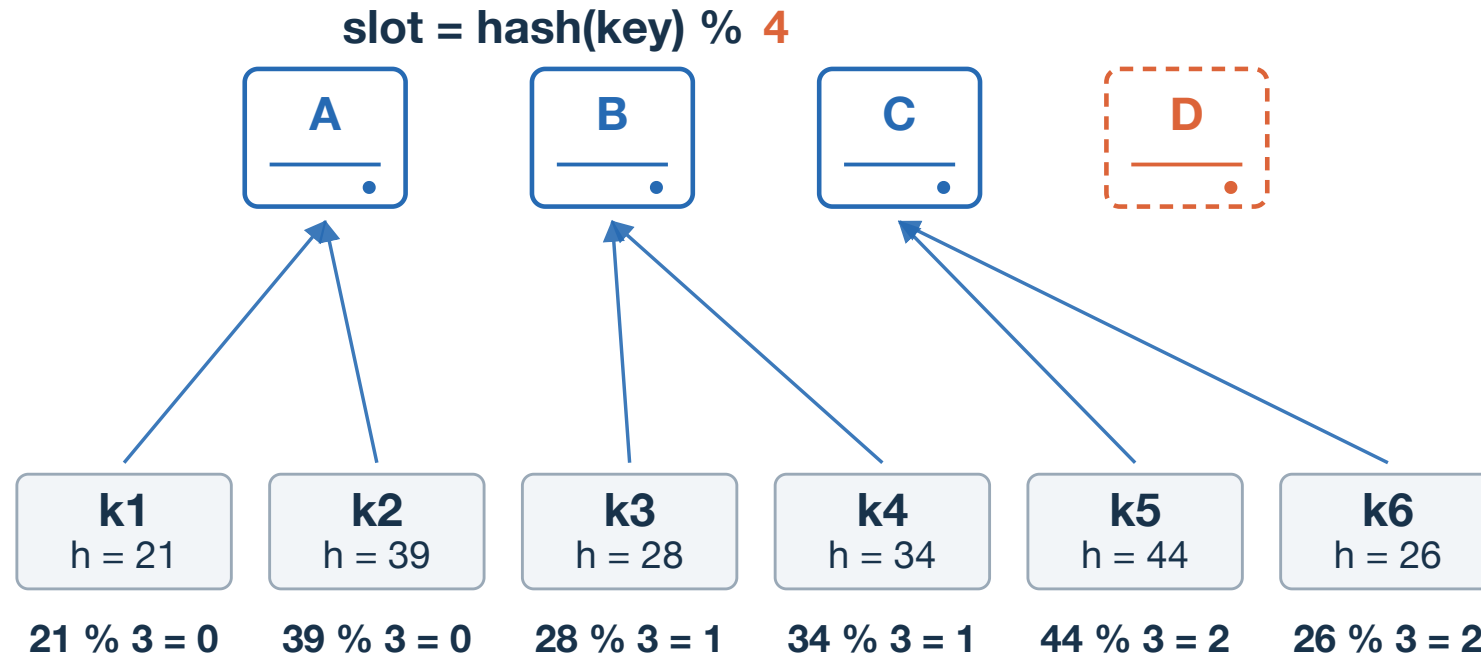
- Growing N changes the divisor, so almost every key gets a different remainder
- A key whose remainder changed has to be physically moved to another disk



Why naive sharding cannot scale

Six keys on three disks with `slot = hash(key) % N`; then a fourth disk arrives

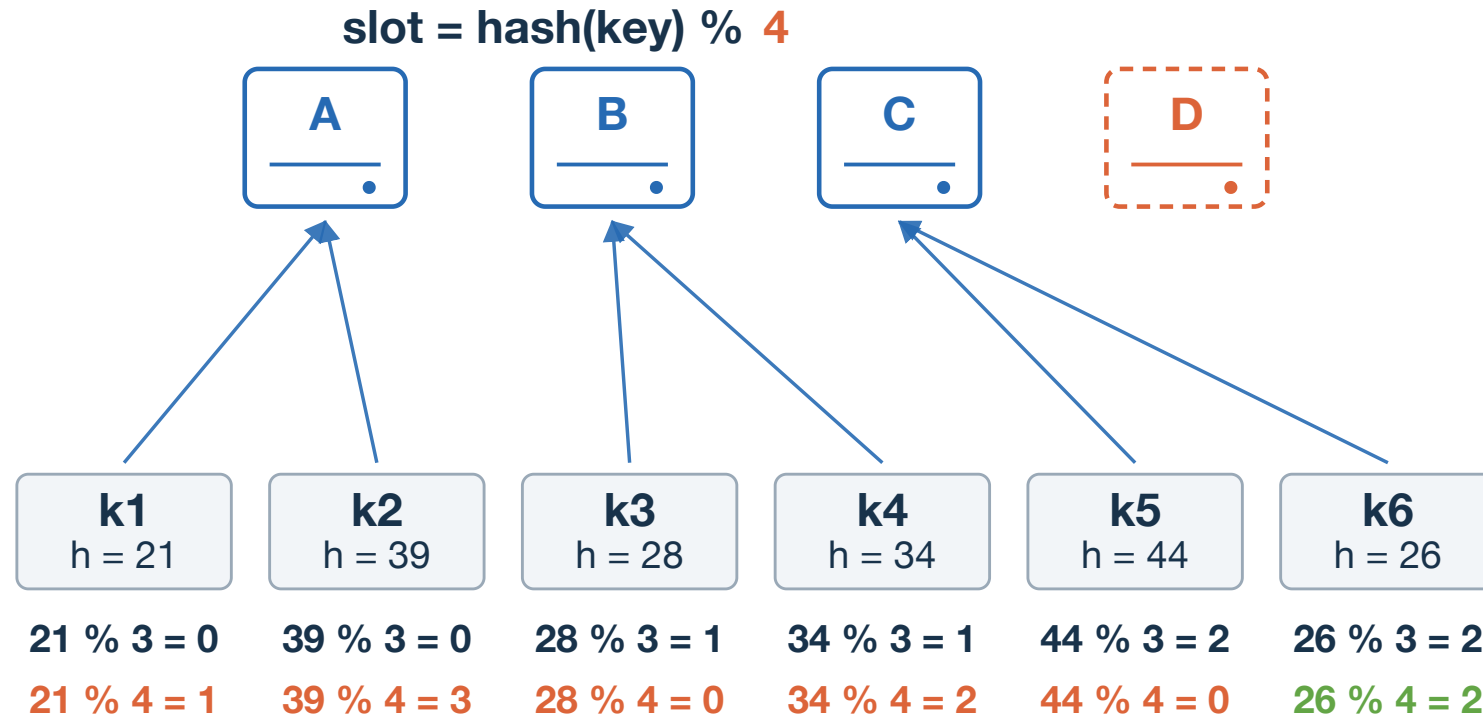
- Growing N changes the divisor, so almost every key gets a different remainder
- A key whose remainder changed has to be physically moved to another disk



Why naive sharding cannot scale

Six keys on three disks with `slot = hash(key) % N`; then a fourth disk arrives

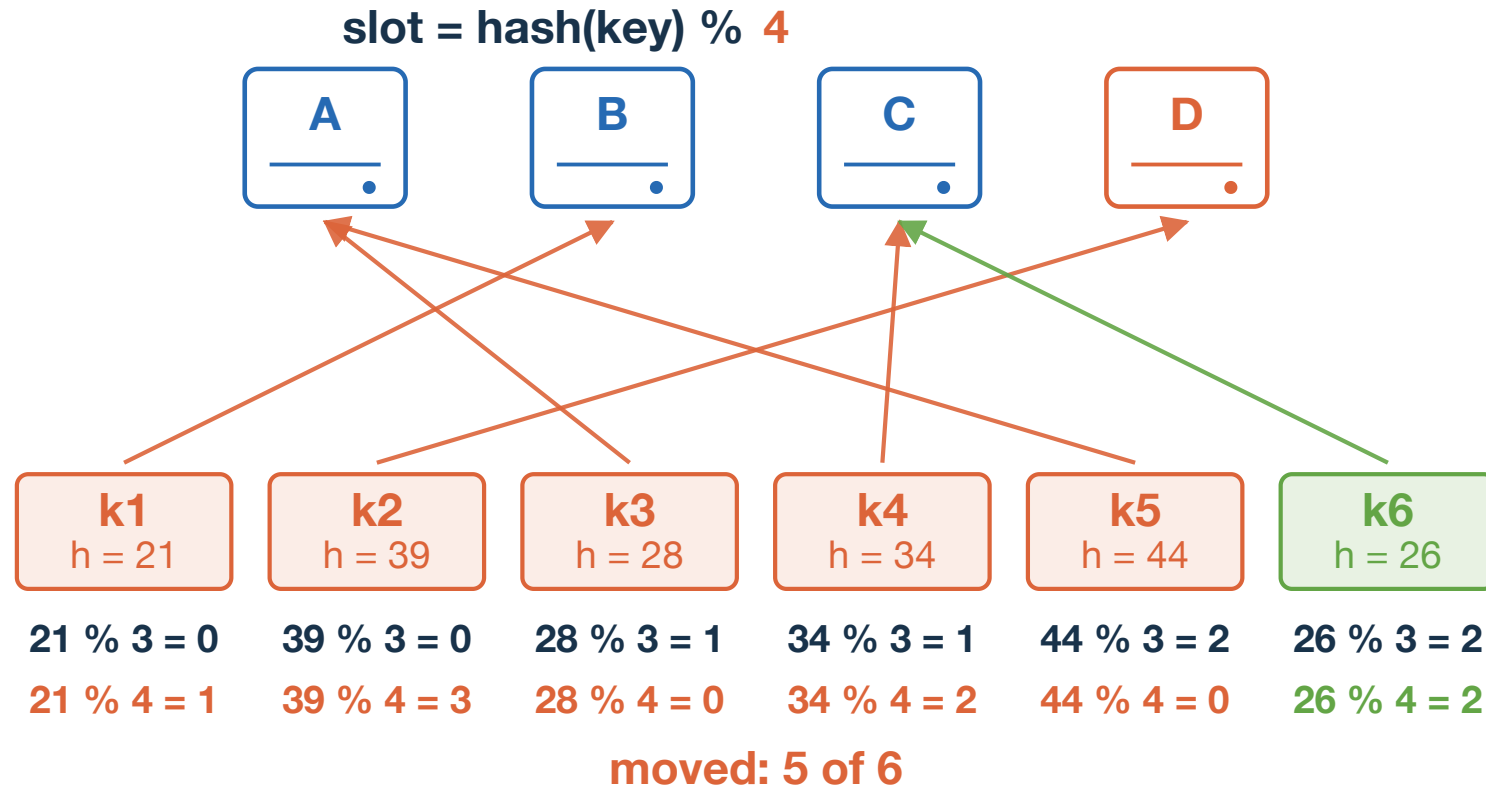
- Growing N changes the divisor, so almost every key gets a different remainder
- A key whose remainder changed has to be physically moved to another disk



Why naive sharding cannot scale

Six keys on three disks with `slot = hash(key) % N`; then a fourth disk arrives

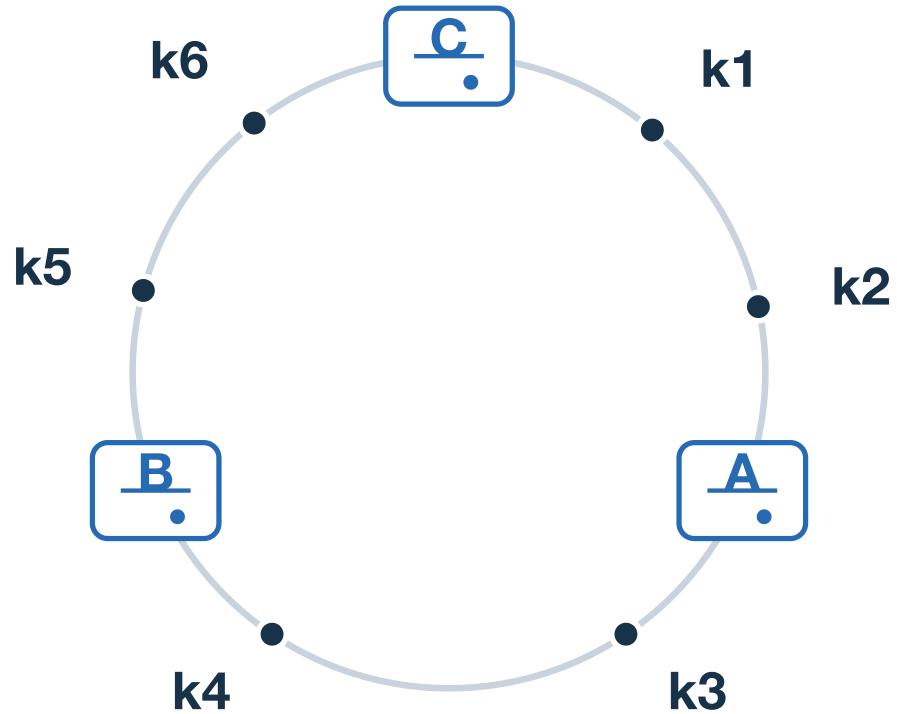
- Growing N changes the divisor, so almost every key gets a different remainder
- A key whose remainder changed has to be physically moved to another disk



Solution: consistent hashing

Consistent hashing: the hash space as a ring

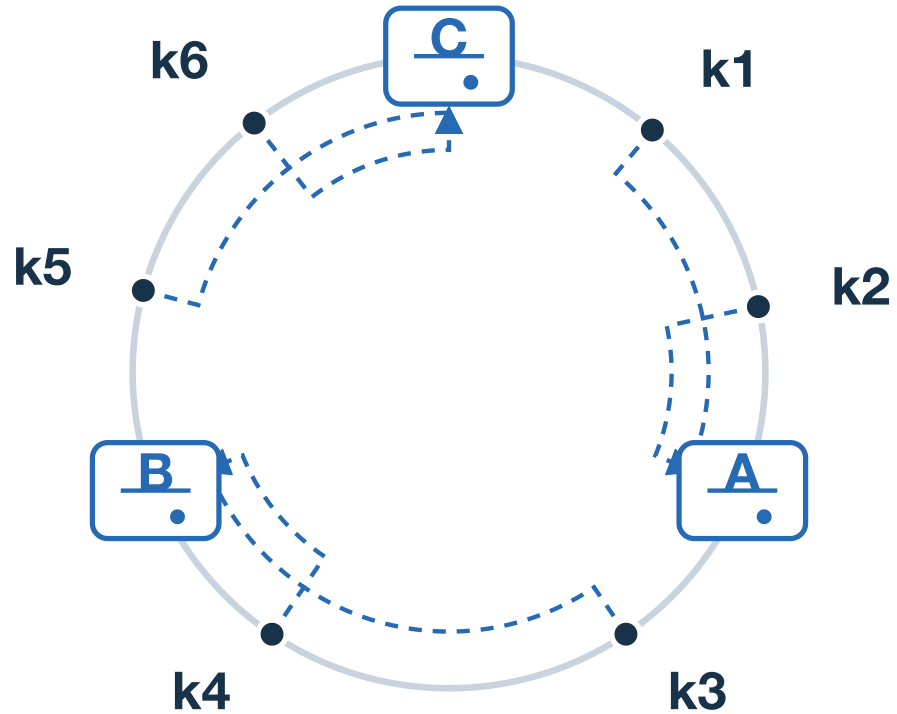
- A key belongs to the first disk clockwise from it



Solution: consistent hashing

Consistent hashing: the hash space as a ring

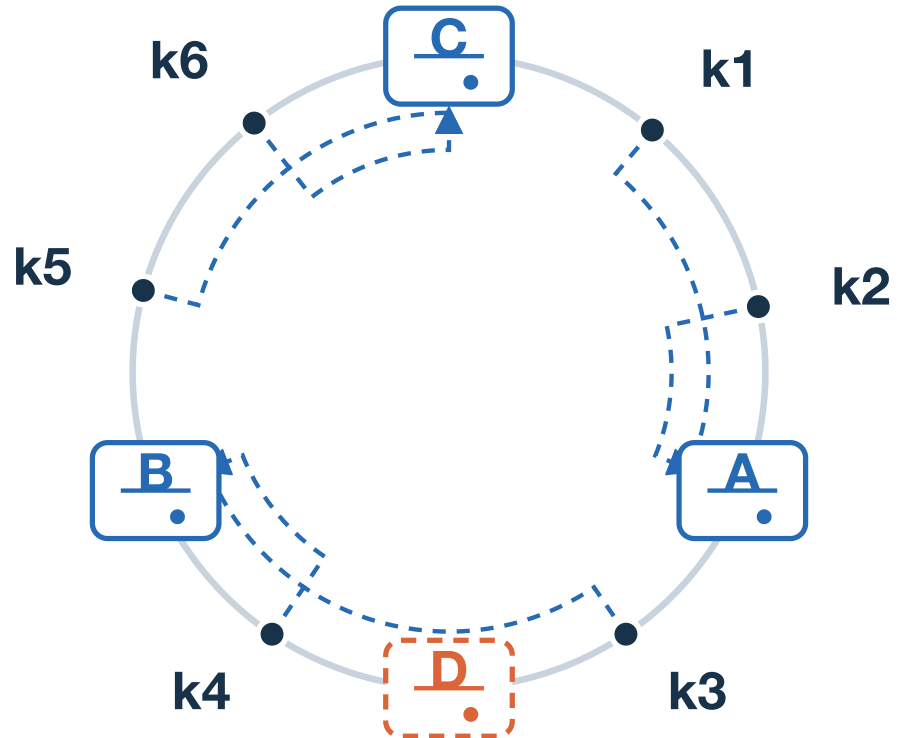
- A key belongs to the first disk clockwise from it



Solution: consistent hashing

Consistent hashing: the hash space as a ring

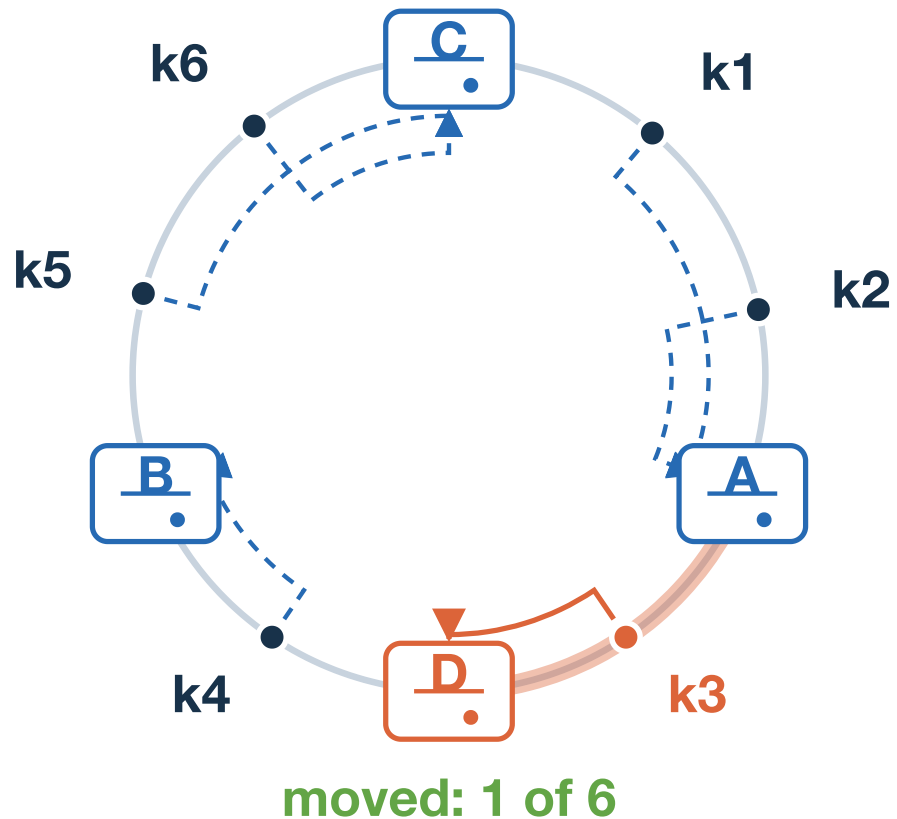
- A key belongs to the first disk clockwise from it



Solution: consistent hashing

Consistent hashing: the hash space as a ring

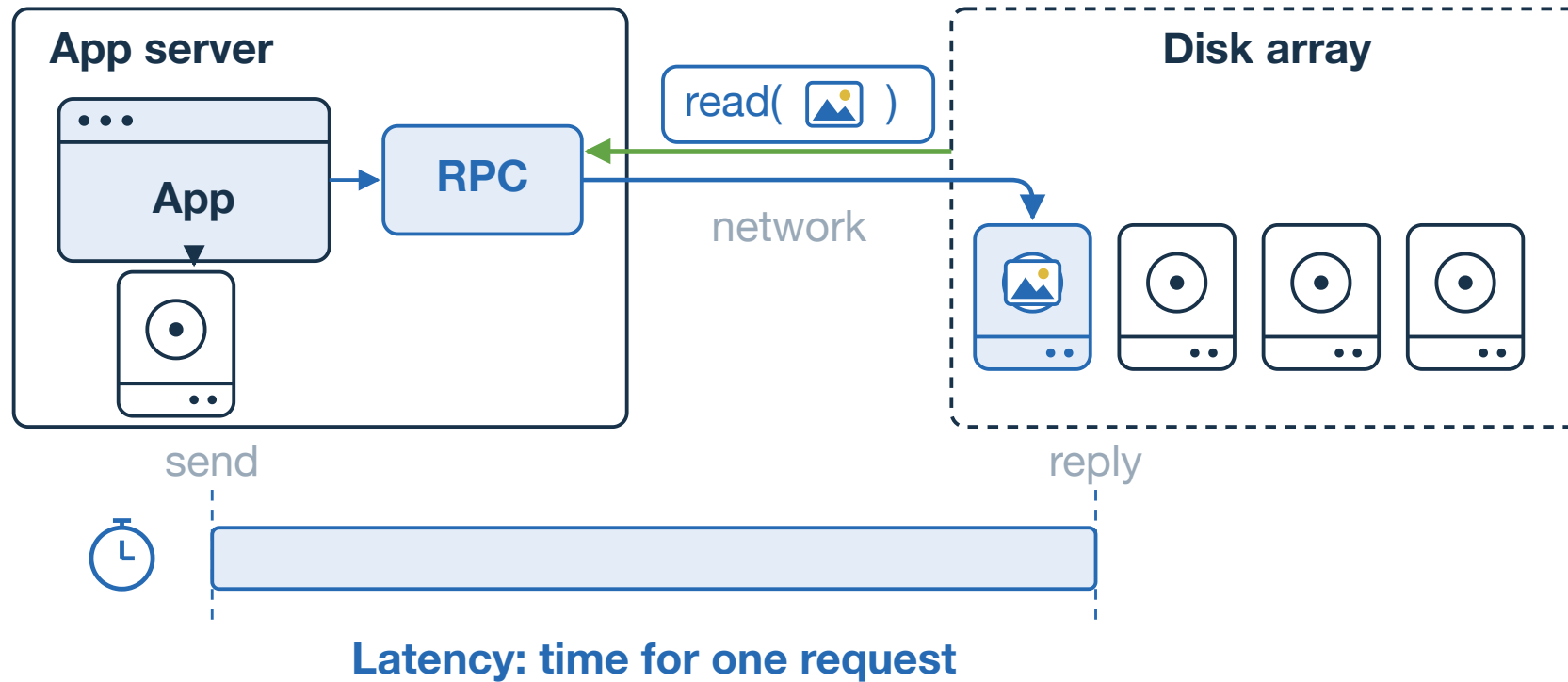
- A key belongs to the first disk clockwise from it



Now we come at the **performance**: Latency & Throughput

Distributed storage solved the capacity problem; but what is the cost? Performance

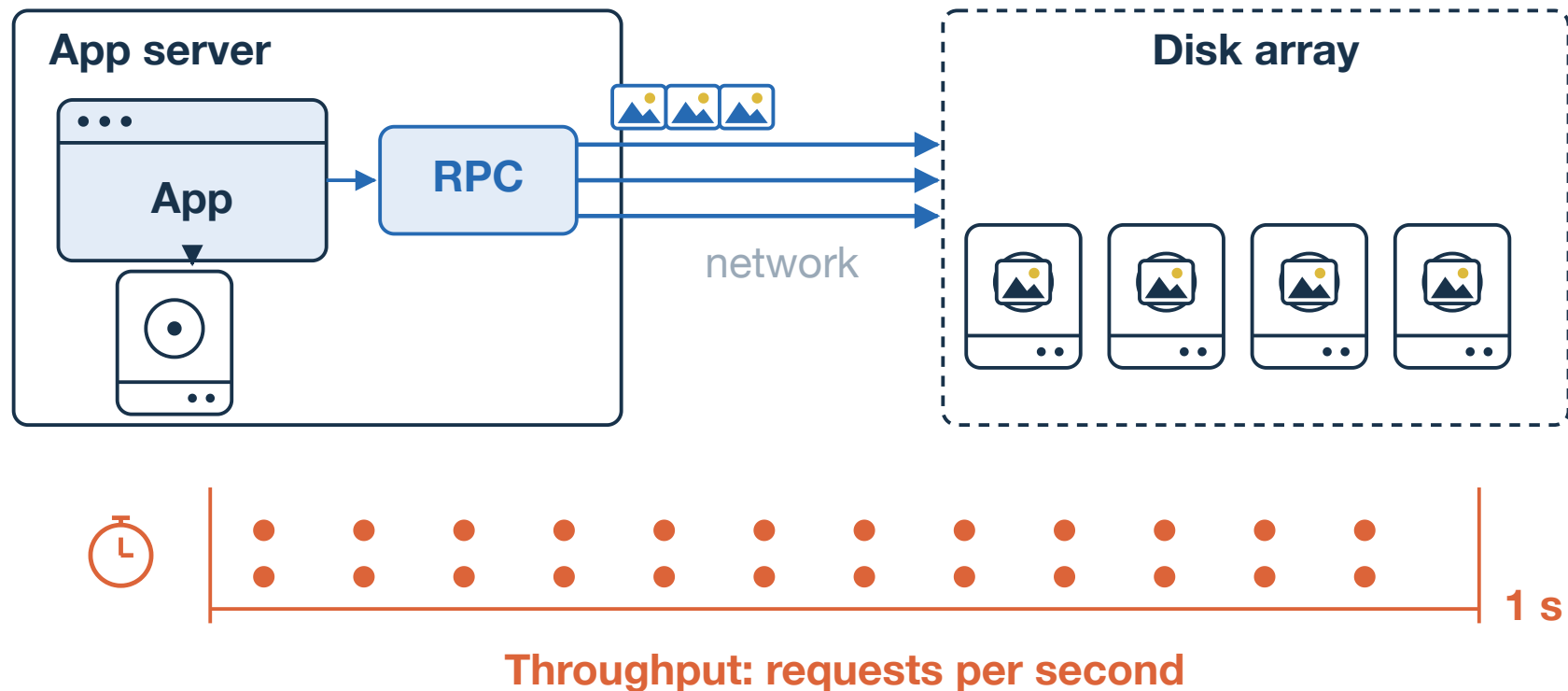
- **Latency**: how fast can one request finish, from send to reply?



Now we come at the **performance**: Latency & Throughput

Performance has two basic metrics

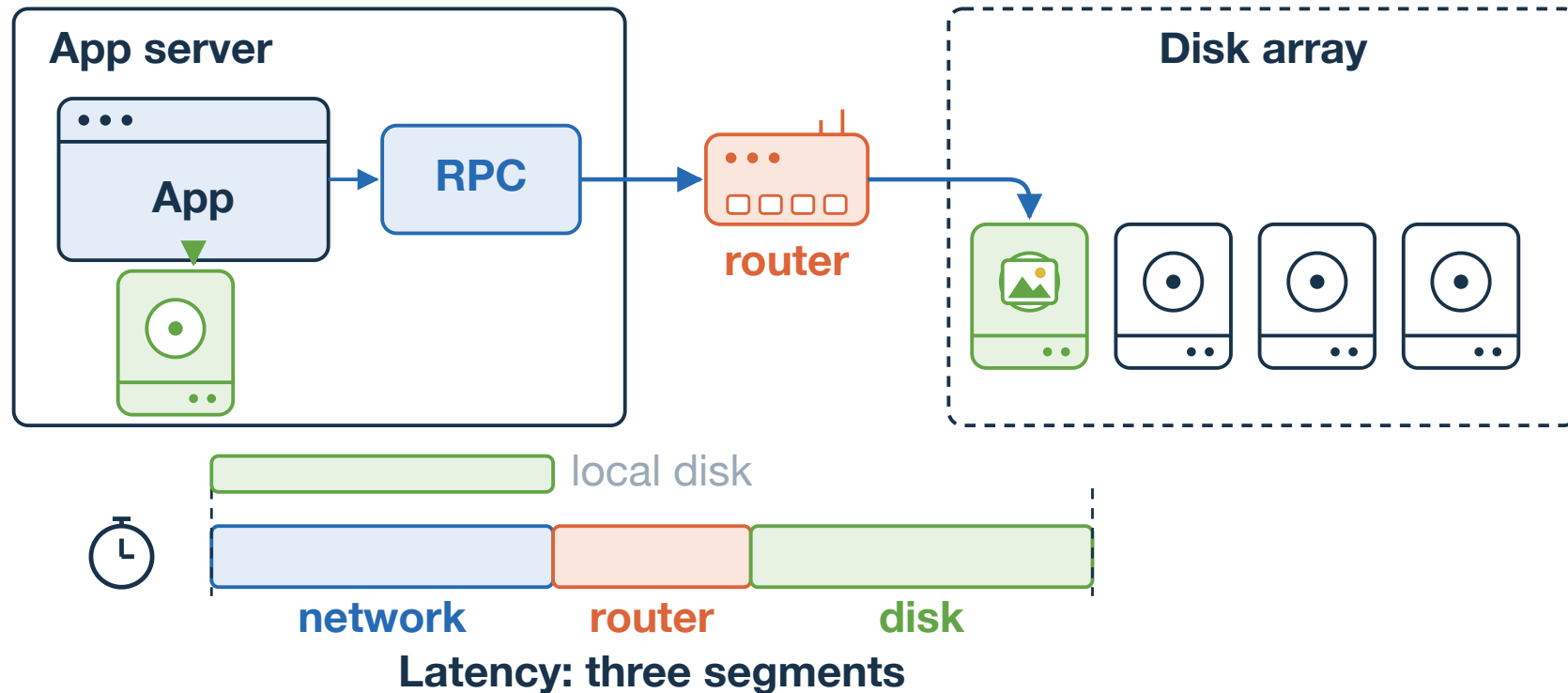
- Latency: how fast can one request finish?
- **Throughput**: how many requests per second can the system (e.g., a website) handle?



Now we come at the **performance: Latency & Throughput**

Latency: the shorter the better (down to a threshold, i.e., the service level agreement)

- For our networked disks, one disk access becomes three segments: network + router + disk



Performance solution #1: Caching

Start with latency. Latency is sometimes the harder one, because it has "physical limits"

1

1	Latency Comparison Numbers (~2012)				
2	-----				
3	L1 cache reference	0.5 ns			
4	Branch mispredict	5 ns			
5	L2 cache reference	7 ns		14x L1 cache	
6	Mutex lock/unlock	25 ns			
7	Main memory reference	100 ns		20x L2 cache, 200x L1 cache	
8	Compress 1K bytes with Zippy	3,000 ns	3 us		
9	Send 1K bytes over 1 Gbps network	10,000 ns	10 us		
10	Read 4K randomly from SSD*	150,000 ns	150 us	~1GB/sec SSD	
11	Read 1 MB sequentially from memory	250,000 ns	250 us		
12	Round trip within same datacenter	500,000 ns	500 us		
13	Read 1 MB sequentially from SSD*	1,000,000 ns	1,000 us	1 ms	~1GB/sec SSD, 4X memory
14	Disk seek	10,000,000 ns	10,000 us	10 ms	20x datacenter roundtrip
15	Local LLM, generate 1 token	15,000,000 ns	15,000 us	15 ms	Small model on consumer GPU (2026)
16	Read 1 MB sequentially from disk	20,000,000 ns	20,000 us	20 ms	80x memory, 20X SSD
17	Frontier LLM, generate 1 token	20,000,000 ns	20,000 us	20 ms	Hosted model output (2026)
18	Local LLM, time to first token	75,000,000 ns	75,000 us	75 ms	Small model, short prompt (2026)
19	Local LLM (CPU), generate 1 token	100,000,000 ns	100,000 us	100 ms	Small model, no GPU (2026)
20	Send packet CA->Netherlands->CA	150,000,000 ns	150,000 us	150 ms	
21	Fast LLM, time to first token	250,000,000 ns	250,000 us	250 ms	Specialized inference hardware (2026)
22	Frontier LLM, time to first token			1,000 ms	Short prompt, no cache (2026)
23	Frontier LLM, short response			3,000 ms	~100 output tokens (2026)
24	Frontier LLM, long context prefill			10,000 ms	~100K input tokens, no cache (2026)
25	Frontier LLM, reasoning response			30,000 ms	Single call with thinking (2026)

Caching: put the data in the **a faster tier**

- What about the fast tier's limited capacity? Rely on the **skewness** of data access

¹ Source: <https://gist.github.com/jboner/2841832>

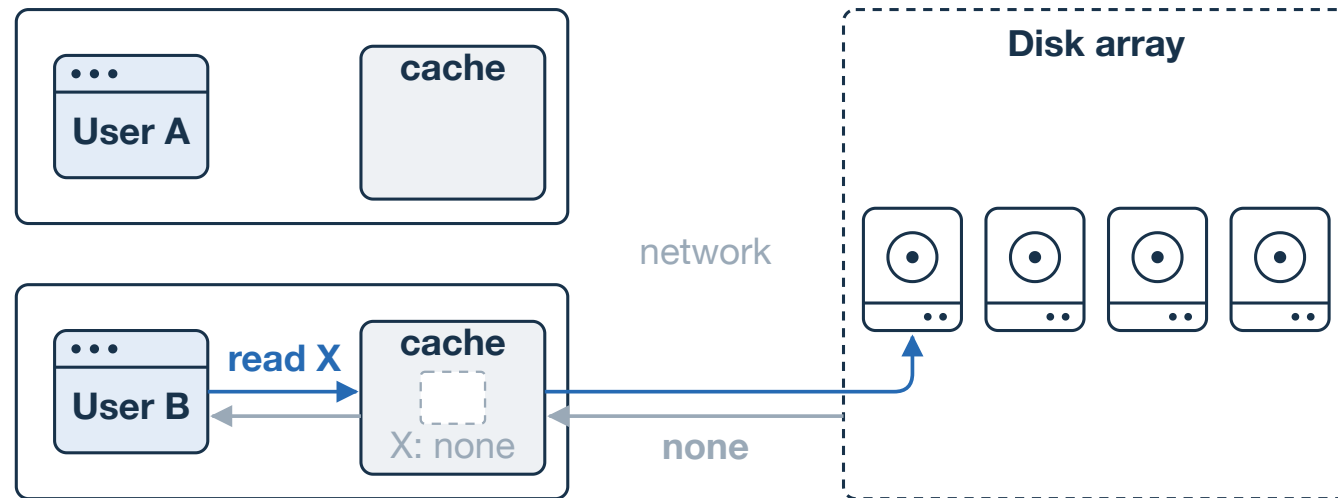
Issue: consistency of caching

So what is the cost of adding a cache?

- Consistency! The data read back does not match what the user expects

Example: suppose there are two users, each caching disk data in memory

- Different users (A and B) view different data



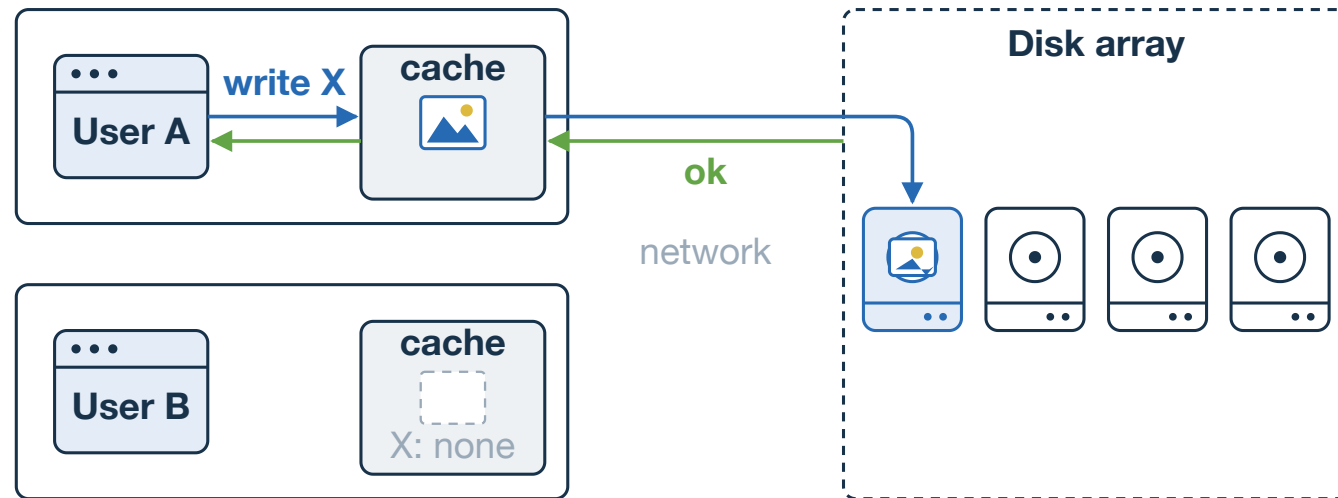
Issue: consistency of caching

So what is the cost of adding a cache?

- Consistency! The data read back does not match what the user expects

Example: suppose there are two users, each caching disk data in memory

- Different users (A and B) view different data



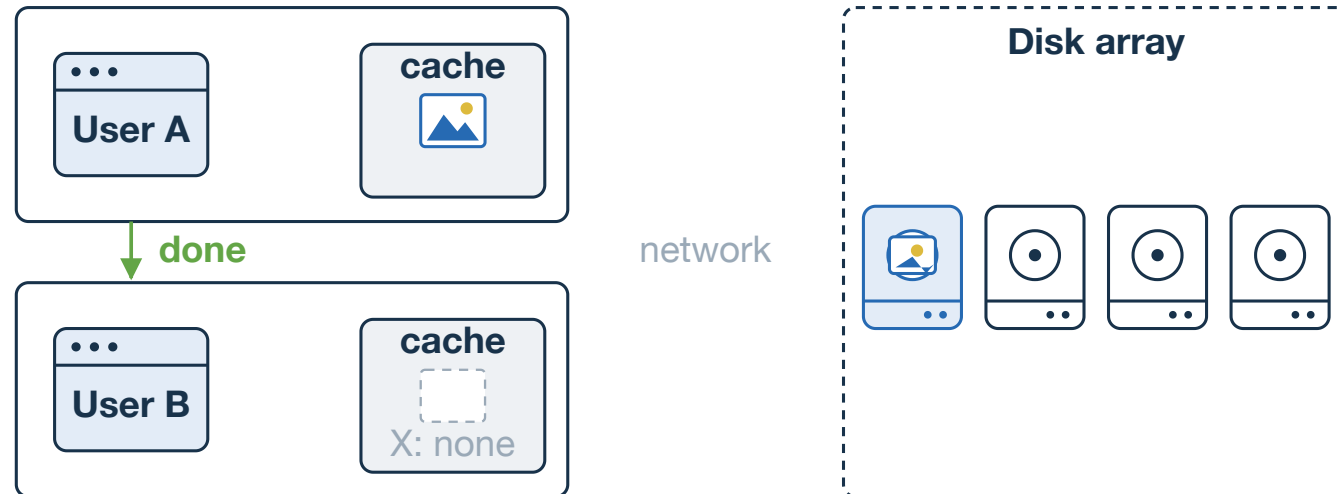
Issue: **consistency** of caching

So what is the cost of adding a cache?

- Consistency! The **data** read back does **not match what the user expects**

Example: suppose there are two users, each caching disk data in memory

- Different users (A and B) view different data



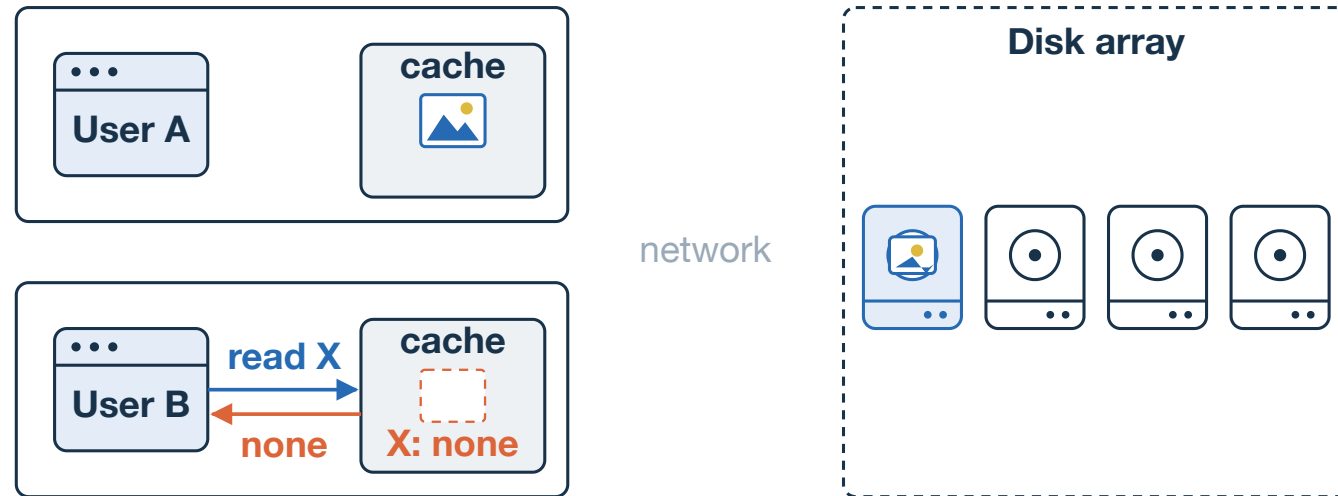
Issue: consistency of caching

So what is the cost of adding a cache?

- Consistency! The data read back does not match what the user expects

Example: suppose there are two users, each caching disk data in memory

- Different users (A and B) view different data



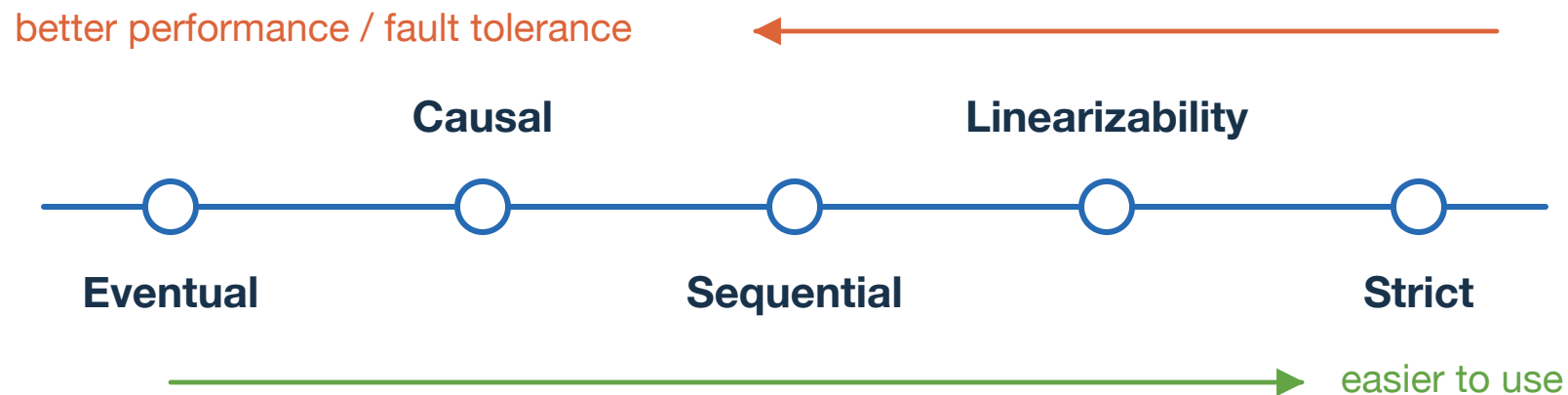
- User B goes to read X, and finds it does not exist! B's cache still holds the old answer

The consistency model for inferring what can happen

Once we scale to a distributed setting, naively scaling performance with caching makes results inconsistent

However, always achieving the strongest consistency may lose performance (trade-off)

Therefore, we need a consistency model to simplify users infer about the correctness¹



¹ Many other consistency models sit between the ones below.

Issue: what about faults

Faults: failed system components lead to system failures if not covered

Scale amplifies "low-probability faults" into the system's normal state:

- A single machine can look very reliable
- But once the cluster is large enough, node failures become a problem you handle every day¹



Scale turns rare faults into daily events:

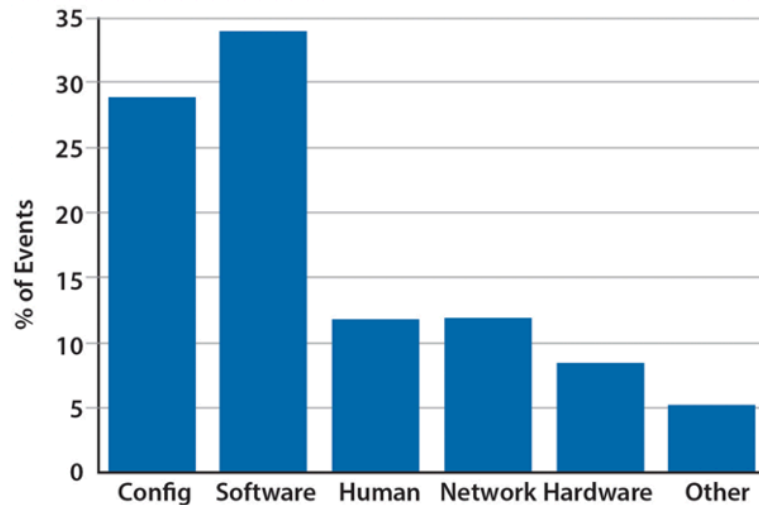
“Suppose a cluster has ultra-reliable server nodes with a stellar mean time between failures (MTBF) of 30 years (10,000 days)—a cluster of 10,000 servers will see an average of one server failure per day.”

¹ Source: The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines

Fault causes: many sources

Common causes go beyond hardware:

- **Operation errors**: configuration, deployment, manual mistakes
- **Software errors**: bugs, crashes, resource leaks
- **Infrastructure (hardware) errors**: hardware, network, power, facilities
- **External** events: natural disasters, regional accidents¹



¹ Source: The Datacenter as a Computer: An Introduction to the Design of Warehouse-Scale Machines

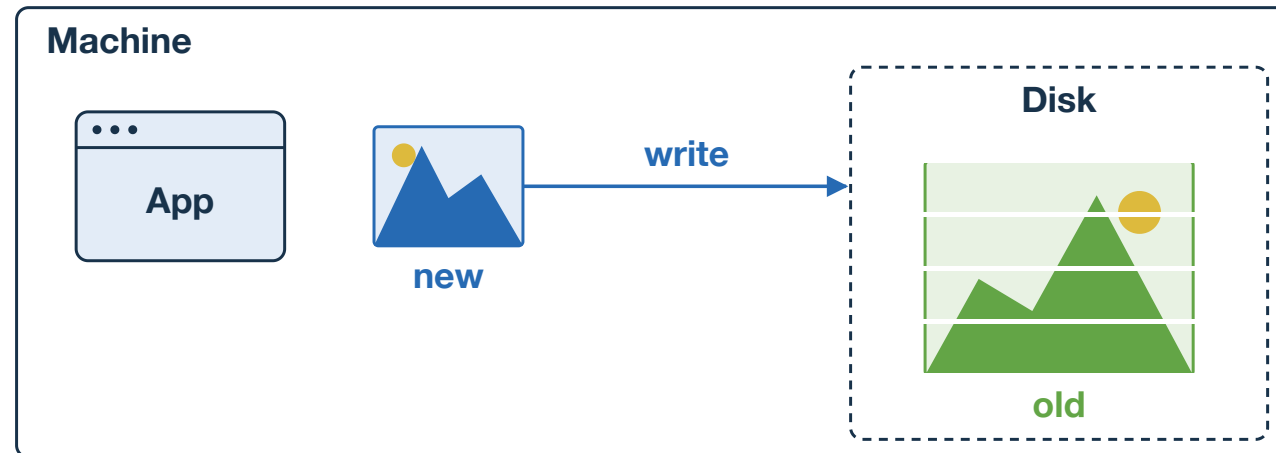
What can go wrong with faults? Besides out-of-service, there can even be consistency issues!

The most direct issue: **data is gone** (never go to the disk), so we need durability

- i.e., the data goes to disk before reporting success to the users

But have you ever wondered: once data hits the disk, is it necessarily correct?

- E.g., I write a whole file but only half of it lands on disk – is the data still match user's expectations?



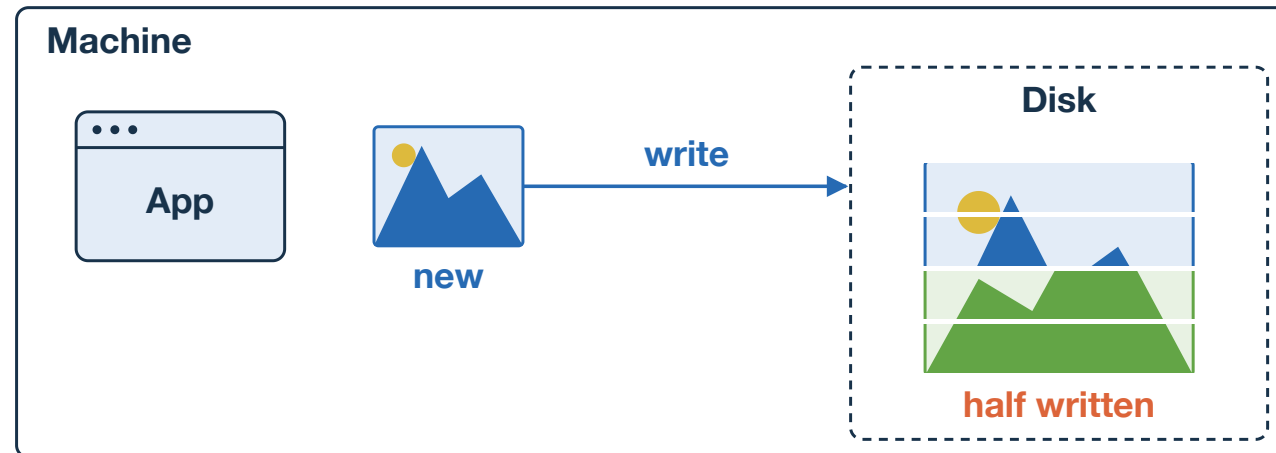
What can go wrong with faults? Besides out-of-service, there can even be consistency issues!

The most direct issue: **data is gone** (never go to the disk), so we need durability

- i.e., the data goes to disk before reporting success to the users

But have you ever wondered: once data hits the disk, is it necessarily correct?

- E.g., I write a whole file but only half of it lands on disk – is the data still match user's expectations?



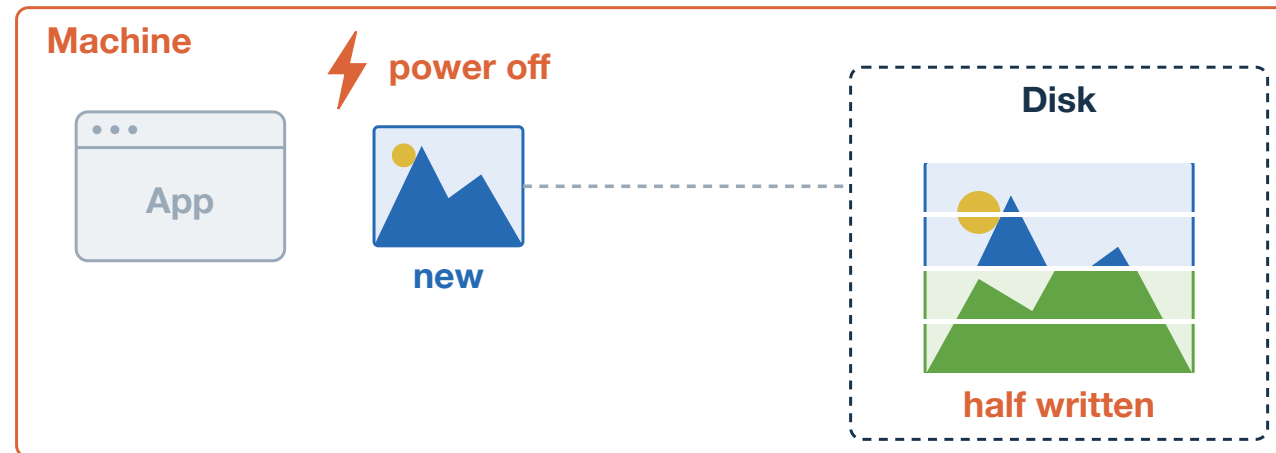
What can go wrong with faults? Besides out-of-service, there can even be consistency issues!

The most direct issue: **data is gone** (never go to the disk), so we need durability

- i.e., the data goes to disk before reporting success to the users

But have you ever wondered: once data hits the disk, is it necessarily correct?

- E.g., I write a whole file but only half of it lands on disk — is the data still match user's expectations?



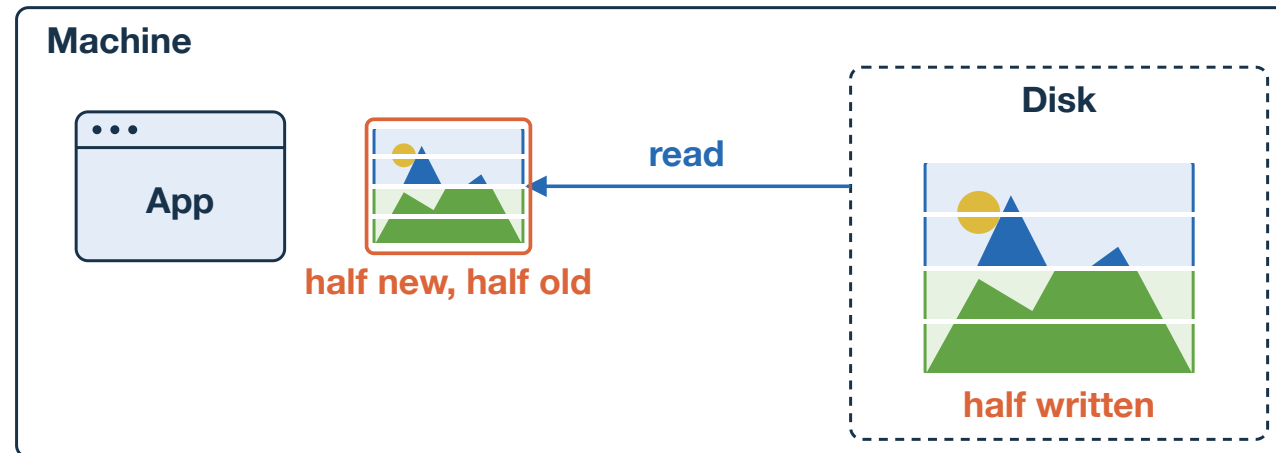
What can go wrong with faults? Besides out-of-service, there can even be consistency issues!

The most direct issue: **data is gone** (never go to the disk), so we need durability

- i.e., the data goes to disk before reporting success to the users

But have you ever wondered: once data hits the disk, is it necessarily correct?

- E.g., I write a whole file but only half of it lands on disk — is the data still match user's expectations?

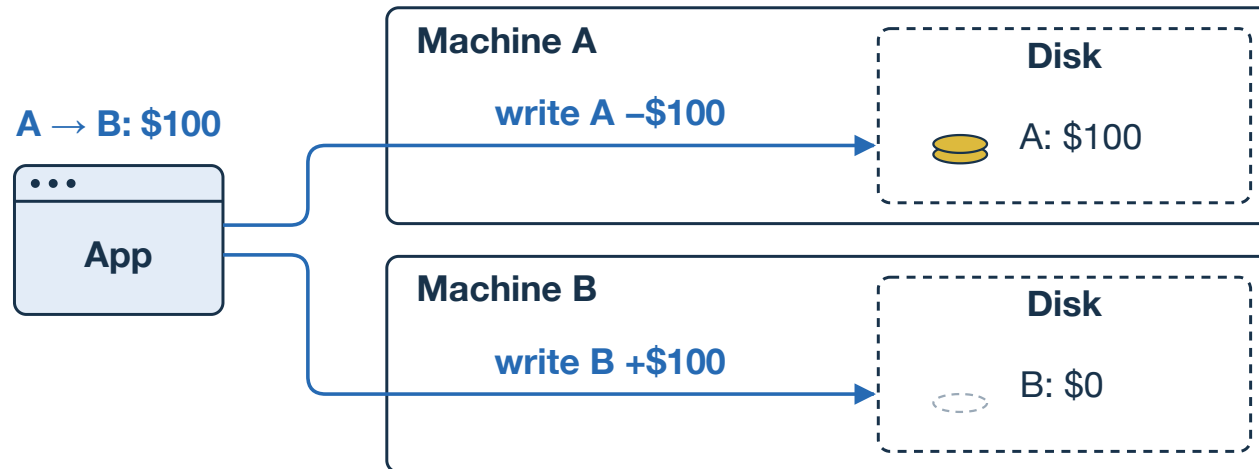


To make data reliable we need systems methods: **all-or-nothing atomicity**

The consistency issue further goes to **multi-site** case

Single-machine faults can lead to multi-machine storage failure because **data is shared**

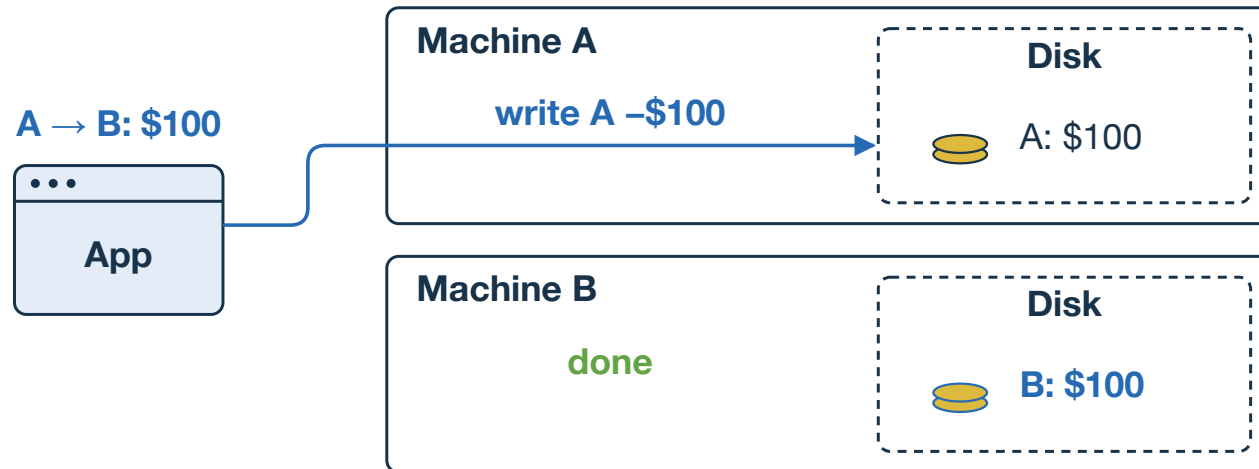
- E.g., transfer \$100 from A to B: B's update hits its disk, then the machine storing A crashes



The consistency issue further goes to **multi-site** case

Single-machine faults can lead to multi-machine storage failure because **data is shared**

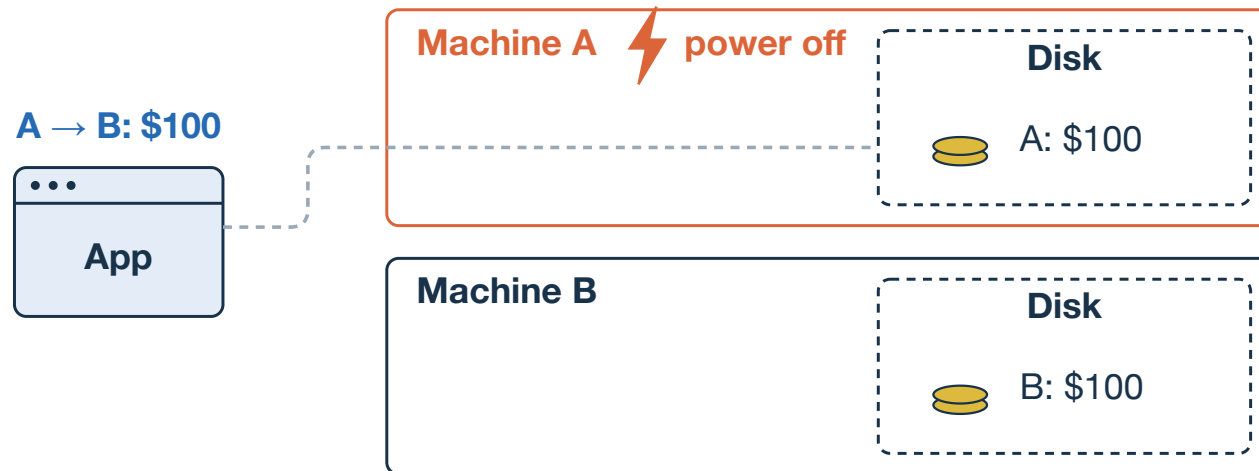
- E.g., transfer \$100 from A to B: B's update hits its disk, then the machine storing A crashes



The consistency issue further goes to **multi-site** case

Single-machine faults can lead to multi-machine storage failure because **data is shared**

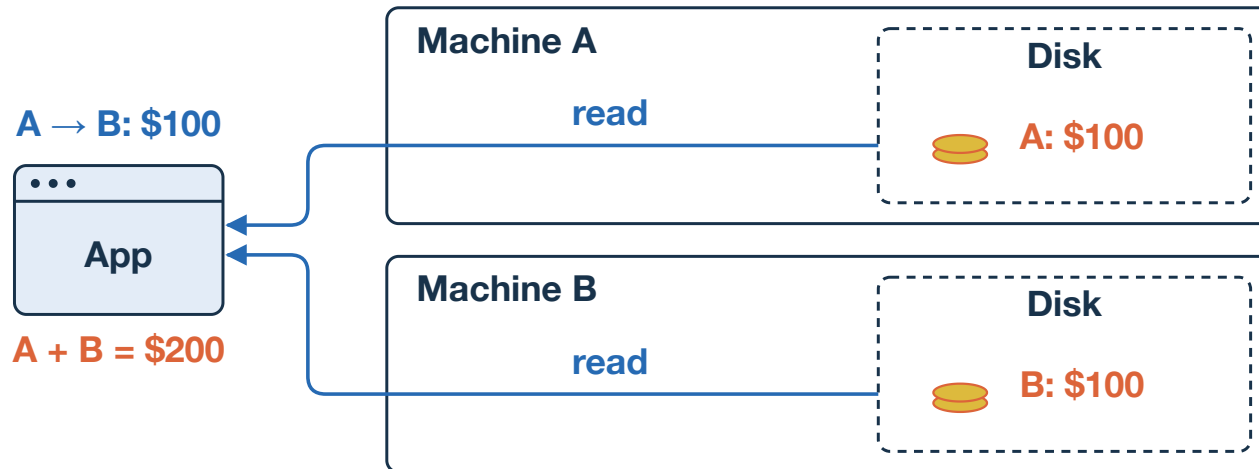
- E.g., transfer \$100 from A to B: B's update hits its disk, then the machine storing A crashes



The consistency issue further goes to **multi-site** case

Single-machine faults can lead to multi-machine storage failure because **data is shared**

- E.g., transfer \$100 from A to B: B's update hits its disk, then the machine storing A crashes



To make distributed data reliable we need systems methods: in systems terms, **multi-site atomicity**

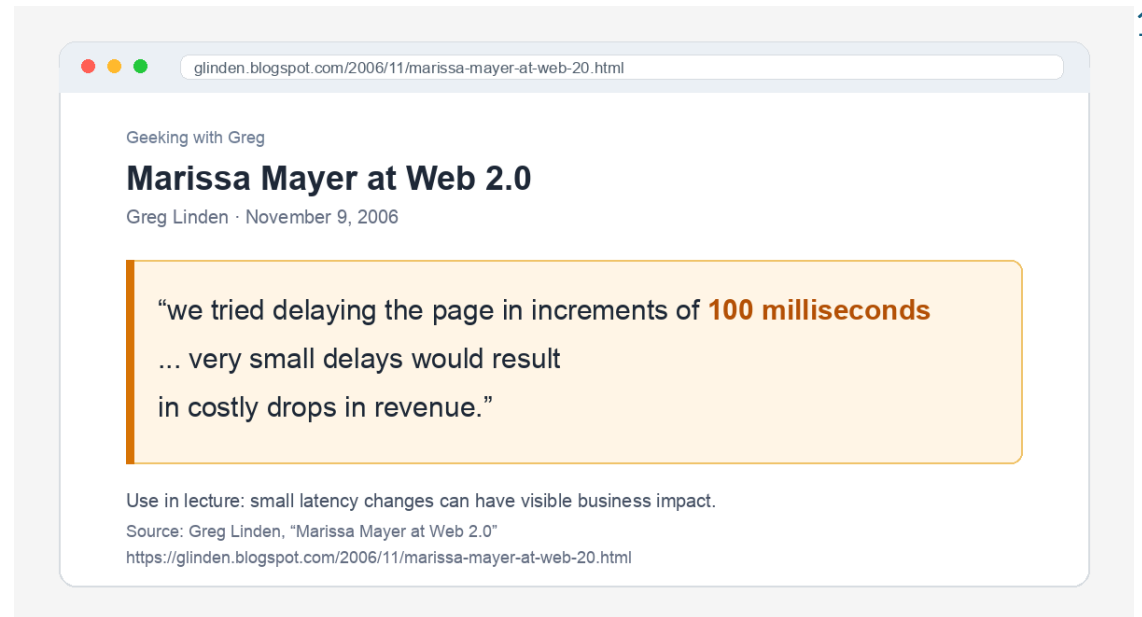
Reliability: from Consistency to Availability

Atomicity only guarantees the data is correct, not that the data/service is **available**

- E.g., If the data is sharded on two nodes, and if one fails, then I cannot read all the data

Availability: the service must **respond within bounded time (usually seconds)**

- Amazon's study: if checkout responds a bit slower, users may just not buy



¹ Source: <https://glinden.blogspot.com/2006/11/marissa-mayer-at-web-20.html>

Availability is possible: failure vs. **partial** failure

As we scale, the chance that something fails grows — but **the chance that everything fails also degrades**

- If one node is unavailable with probability p , k **independent nodes** are all unavailable with probability p^k
- E.g., $p \approx 0.001/\text{hour} \rightarrow 3 \text{ replicas all unavailable} \approx 10^{-9}/\text{hour}$



The real magnitude of p :

An estimate from a public Google cluster trace: 12,500 machines, 8,771 failure-like machine events in 29 days.

That works out to $p \approx 0.001$ per machine per hour; i.e., at the scale of 1,000 machines, about 1 failure per hour.

If 3 replicas fail roughly independently, the probability all fail is $p^3 \approx 10^{-9} / \text{hour}$.¹

So a failure is almost always a **"partial failure"**: part of the system is down, but most of it is alive

¹ Source: DC-Prophet: Predicting Catastrophic Machine Failures in DataCenters, <https://arxiv.org/abs/1709.06537>

Solution: Replication

Faults are **a fact**; they cannot be eliminated — so we can only **hide them**

How to hide them? Replication:

- Replication in **space**: store multiple copies of the data, deploy multiple copies of the service instances, etc.
- Replication in **time**: retry

The two almost always go together



The challenge of replication: consistency

Replication is somewhat like caching: it turns one copy of the data into many

- So inconsistency appears again: e.g., two replicas of the same data disagree!

How to keep them consistent with availability? We need "distributed consensus protocols", such as Raft and Paxos

- They are among the cores of distributed systems

A quick summary: building a scalable, reliable and consistent system

Scalable means that the system is possibly distributed

- Classic definition: "a collection of components on different networked computers that work together to achieve a common goal"

For a distributed system, ideal properties are

- the data is correct (consistency)
- the data is never lost, and always accessible (atomicity, durability, and availability)

None of these is easy once failures — and partial failures — are present



What is a distributed system?:

You know you have a distributed system when the crash of a computer you've never heard of stops you from getting any work done.

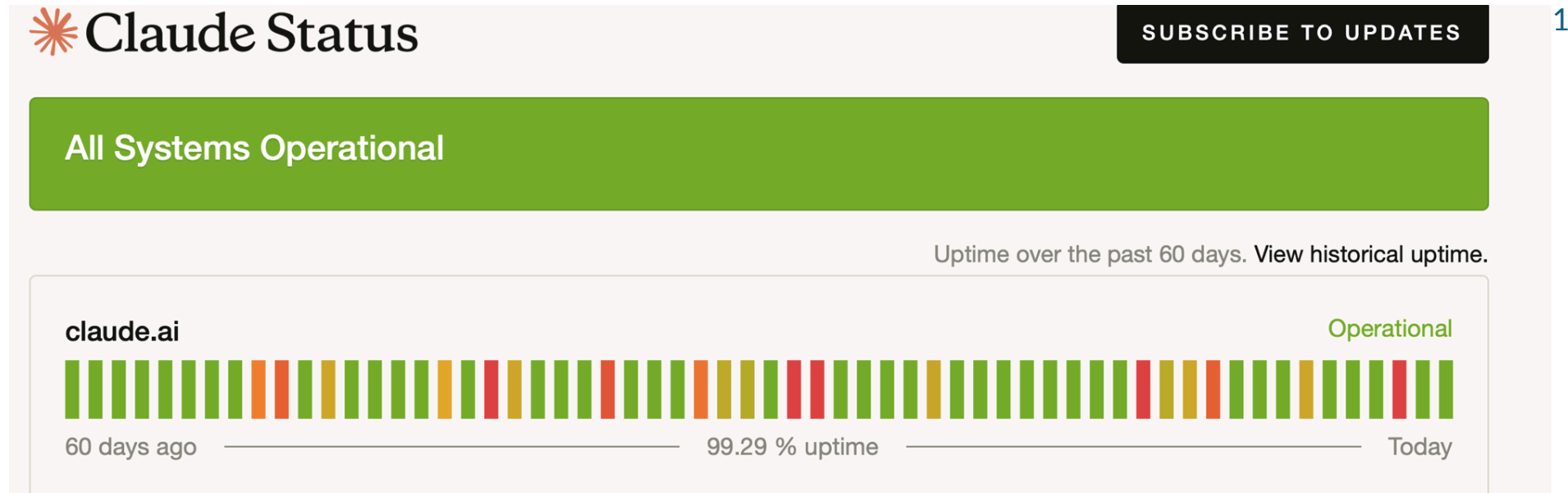
— Leslie Lamport

The real world: failures are everywhere



Large-scale AI services fail too

For example, claude has a status paging shown the service quality



99.29% over 60 days $\approx$ 10 hours of not being healthy

¹ Source: status.claude.com, this is a snapshot

So far: latency, consistency, reliability.

But performance has two metrics.

What about throughput?

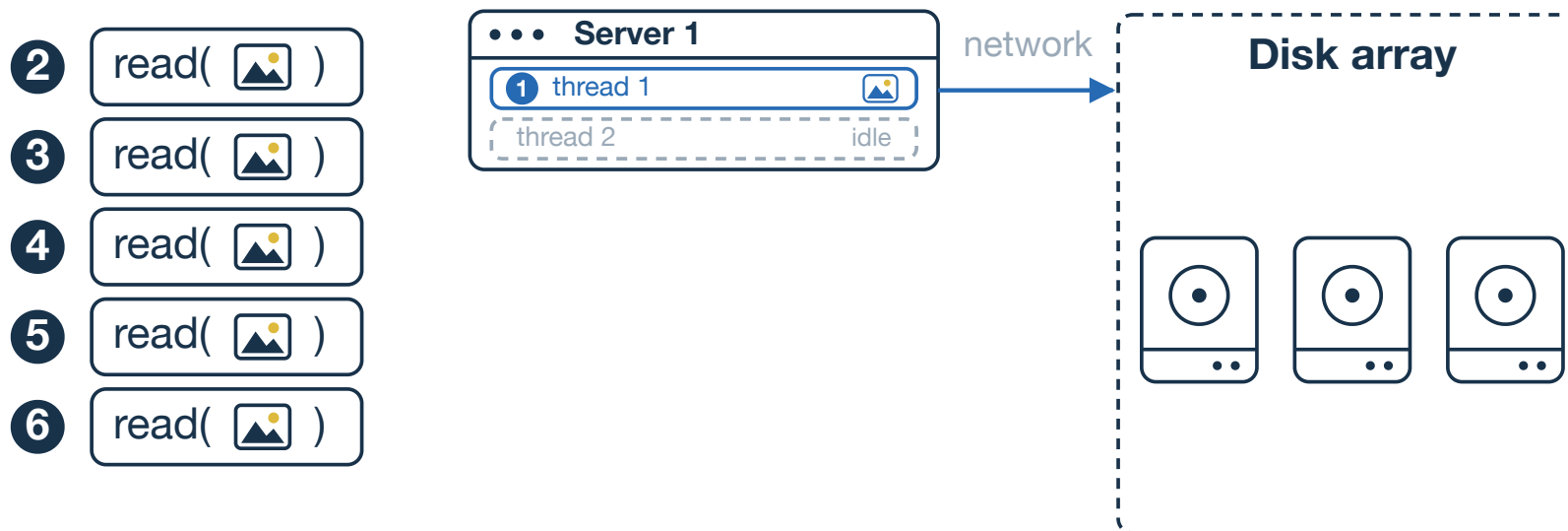
Parallelism(Concurrency) for a high throughput

Recap: performance has two core objectives, latency and throughput

- Latency: how long one task takes to finish
- Throughput: how many tasks a system finishes per second

An intuitive way to scale throughput is parallelism: serve many requests at the same time

- **Scale up** (Scaling within one or a few machine), **Scale out** (Scaling across machines)



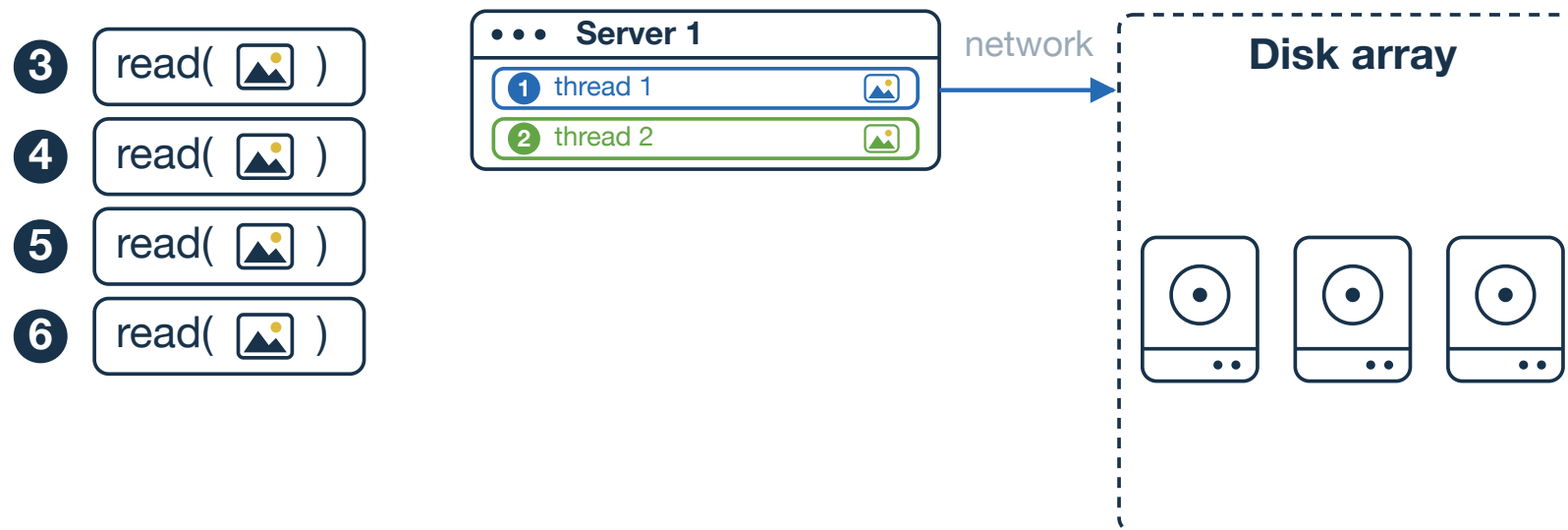
Parallelism(Concurrency) for a high throughput

Recap: performance has two core objectives, latency and throughput

- Latency: how long one task takes to finish
- Throughput: how many tasks a system finishes per second

An intuitive way to scale throughput is parallelism: serve many requests at the same time

- **Scale up** (Scaling within one or a few machine), **Scale out** (Scaling across machines)



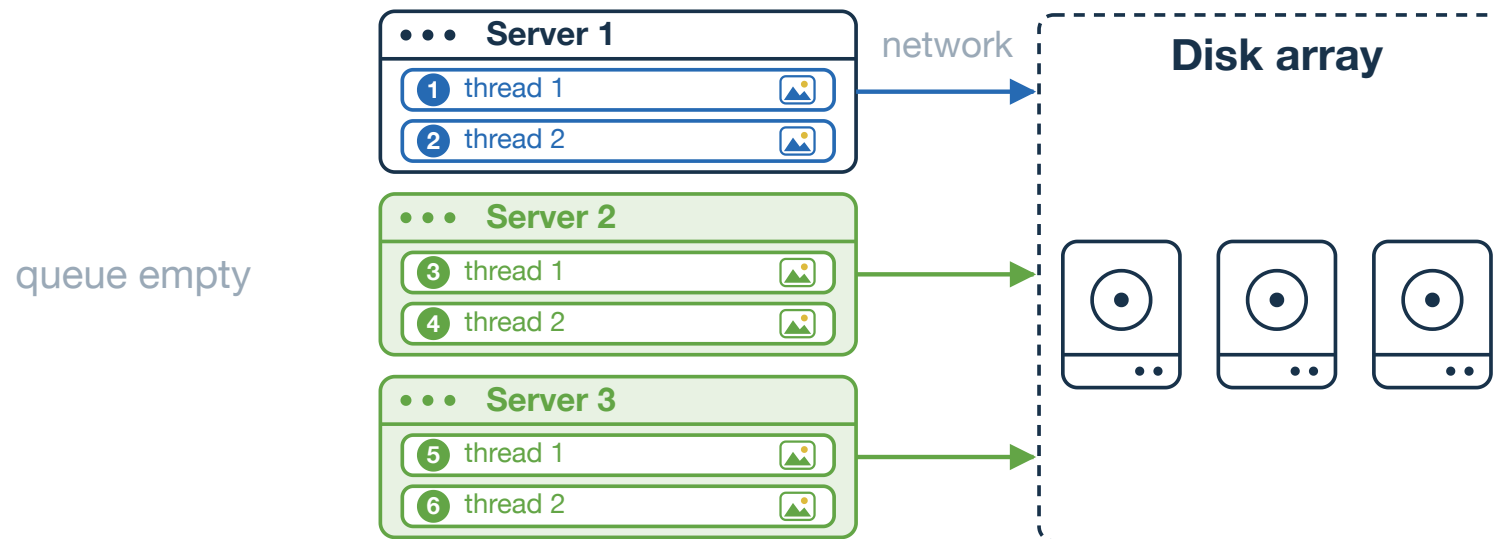
Parallelism(Concurrency) for a high throughput

Recap: performance has two core objectives, latency and throughput

- Latency: how long one task takes to finish
- Throughput: how many tasks a system finishes per second

An intuitive way to scale throughput is parallelism: serve many requests at the same time

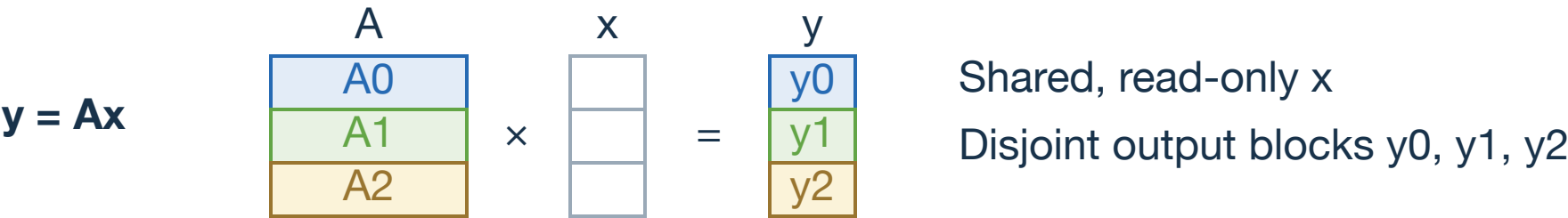
- **Scale up** (Scaling within one or a few machine), **Scale out** (Scaling across machines)



Sidenote: Parallelism can also reduce latency

Because one task can be split into multiple sub-tasks, e.g., split $y = Ax$ by rows.

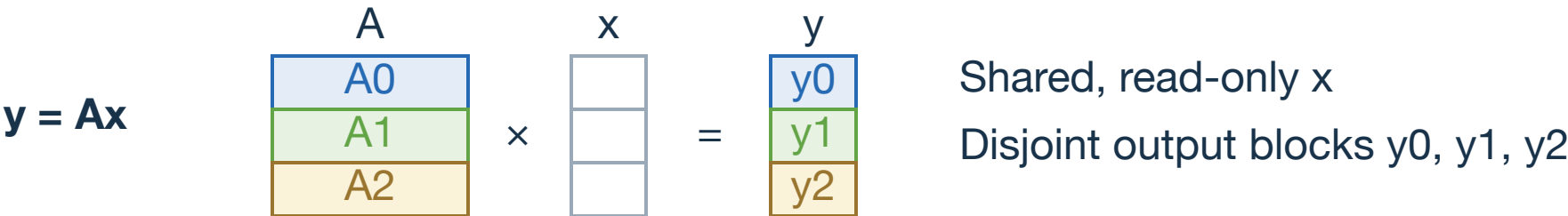
- We can run sub-tasks concurrently together to reduce total request processing time!



Sidenote: Parallelism can also reduce latency

Because one task can be split into multiple sub-tasks, e.g., split $y = Ax$ by rows.

- We can run sub-tasks concurrently together to reduce total request processing time!



Sidenote: Parallelism can also reduce latency

Because one task can be split into multiple sub-tasks, e.g., split $y = Ax$ by rows.

- We can run sub-tasks concurrently together to reduce total request processing time!

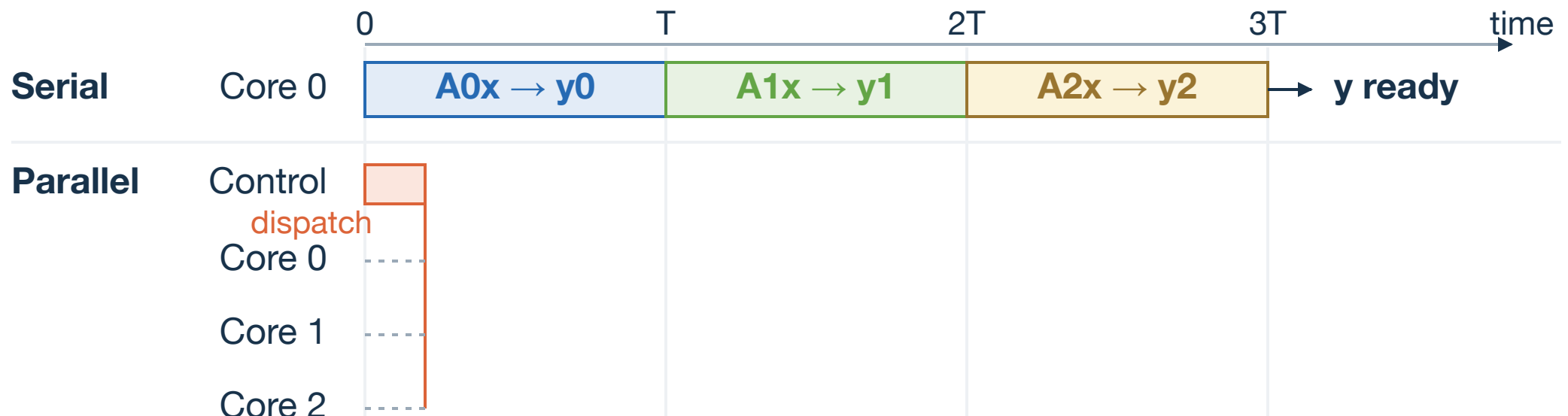
$$y = Ax$$

A	x	=	y
A0	x	=	y0
A1		y1	
A2		y2	

Shared, read-only x
Disjoint output blocks y0, y1, y2

3 Dispatch to 3 cores

Schematic coordination overhead



Sidenote: Parallelism can also reduce latency

Because one task can be split into multiple sub-tasks, e.g., split $y = Ax$ by rows.

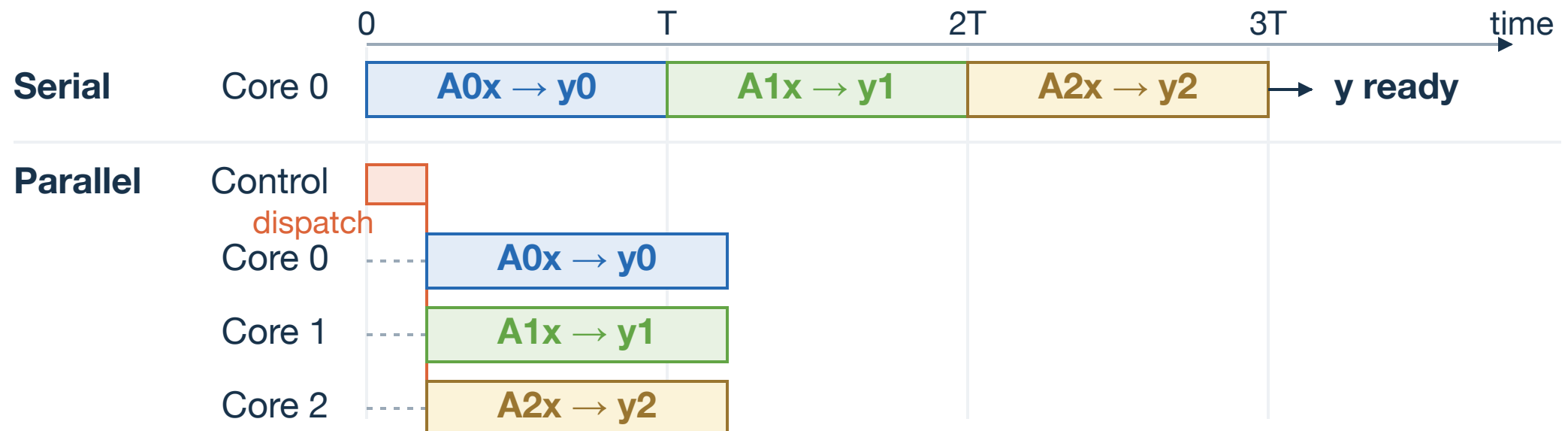
- We can run sub-tasks concurrently together to reduce total request processing time!

$$y = Ax$$

Shared, read-only x
Disjoint output blocks y_0, y_1, y_2

4 Compute in parallel

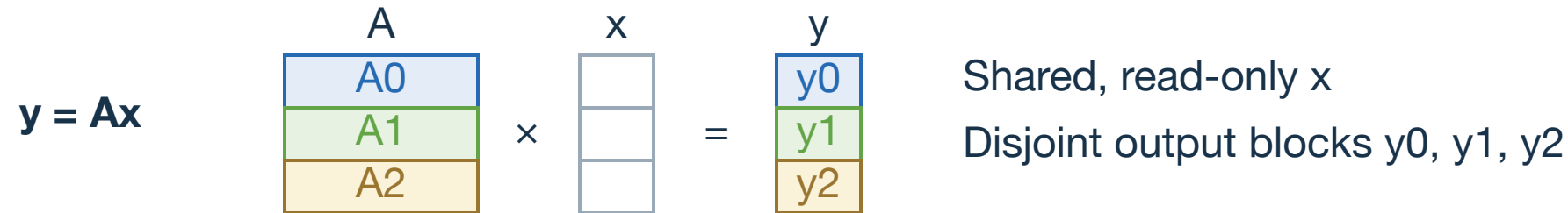
Schematic coordination overhead



Sidenote: Parallelism can also reduce latency

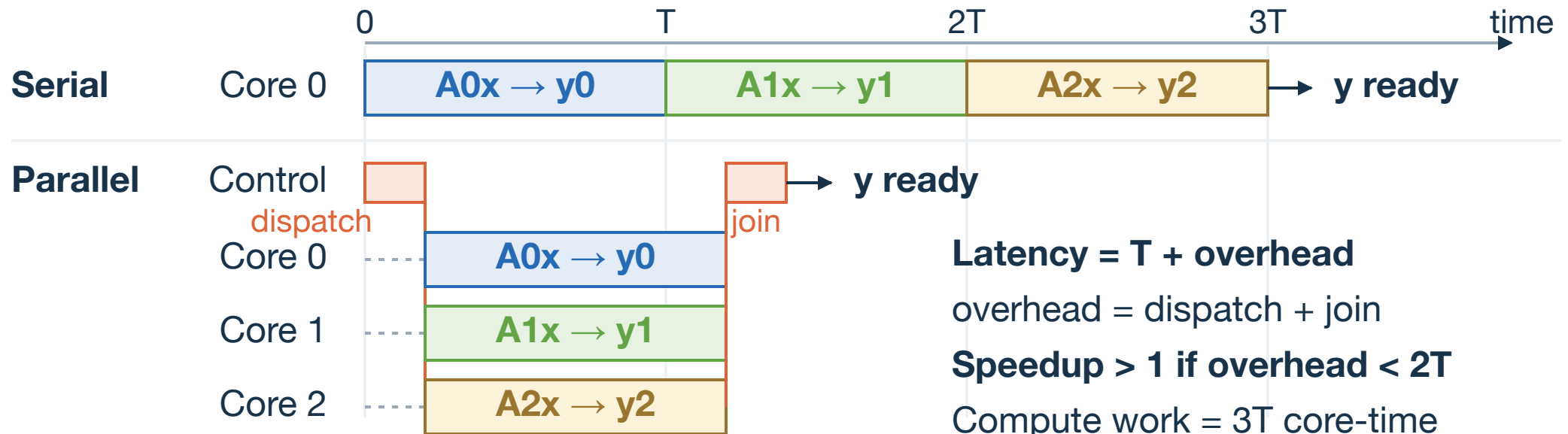
Because one task can be split into multiple sub-tasks, e.g., split $y = Ax$ by rows.

- We can run sub-tasks concurrently together to reduce total request processing time!



5 Join, then return y

Schematic coordination overhead



Example: Parallelism in AI – Training & Inference

Recall: the scaling law requires $\text{FLOP} = 6 \times B \times W \times S$ for the transformer to train

- But a single chip's compute has physical limits → to scale without scale the time, parallelism is the only way

The AI workloads run full with parallelism:

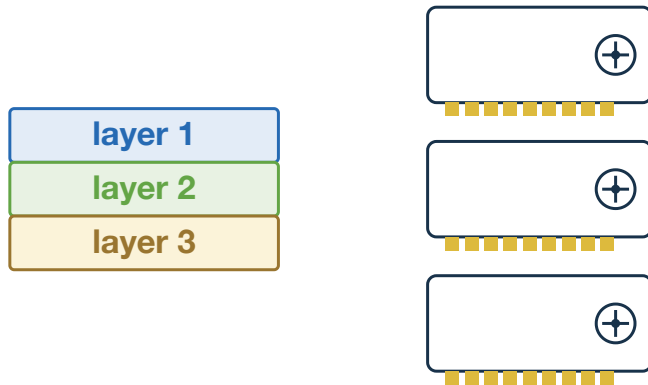
- When doing the computation on one request (e.g., training a model, serving one request)
- When serving multiple requests concurrently (e.g., at API service providers)

Both of them is non-trivial

Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

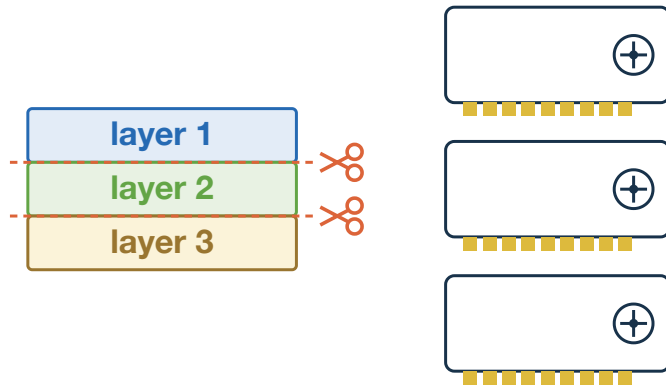
- Thus, we can spread the layer across GPUs to use parallelism



Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

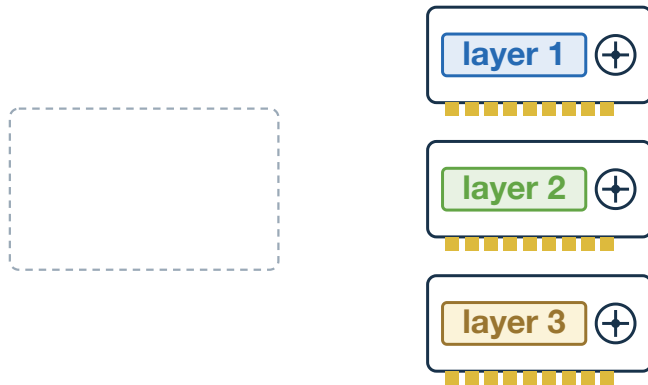
- Thus, we can spread the layer across GPUs to use parallelism



Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

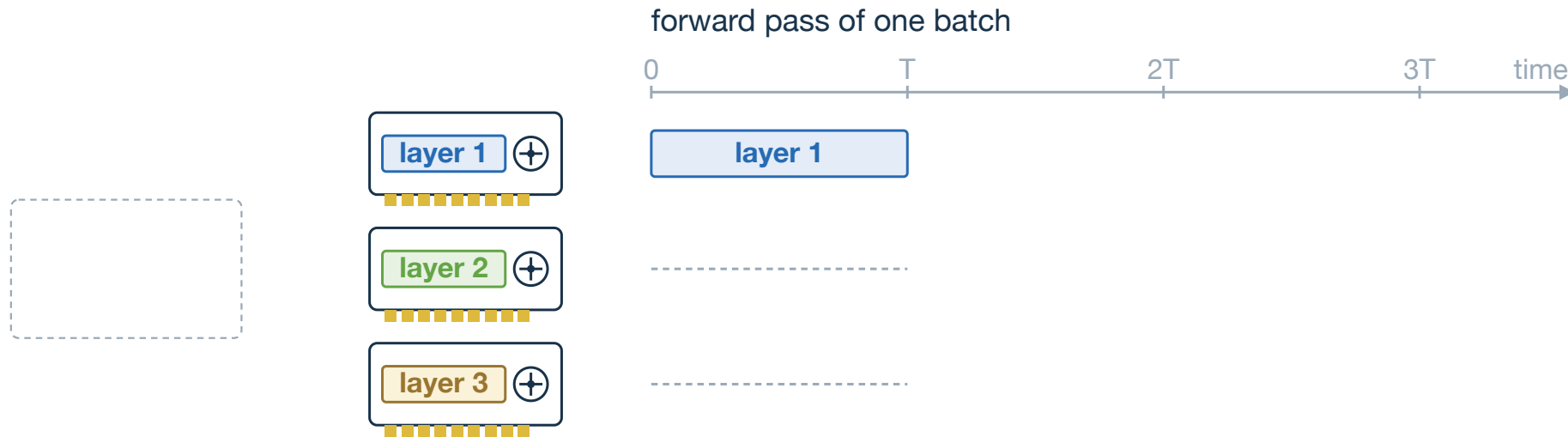
- Thus, we can spread the layer across GPUs to use parallelism



Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

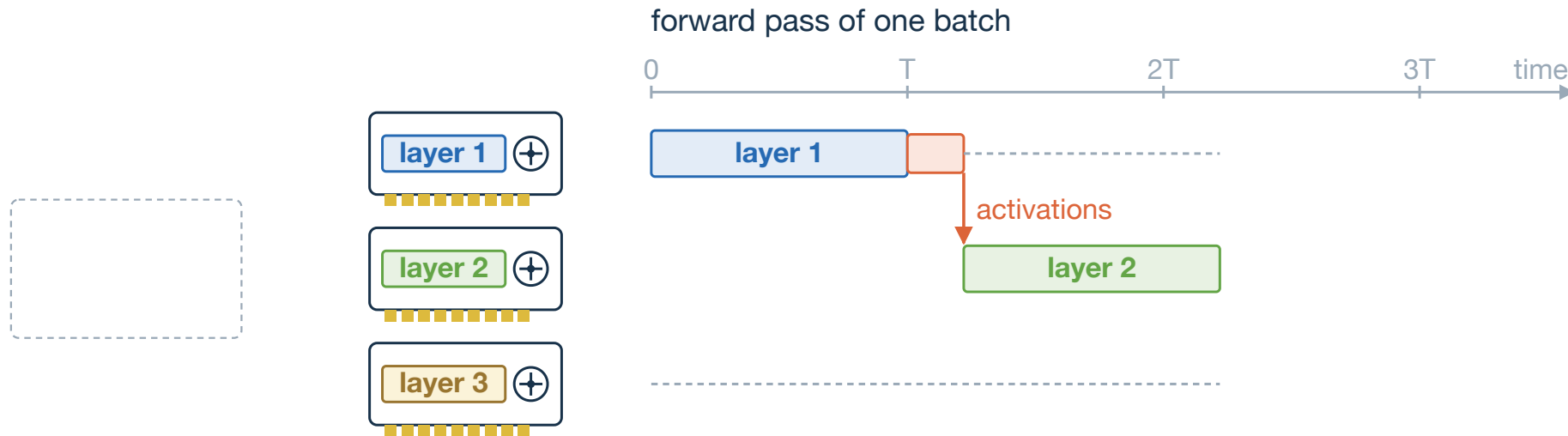
- Thus, we can spread the layer across GPUs to use parallelism



Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

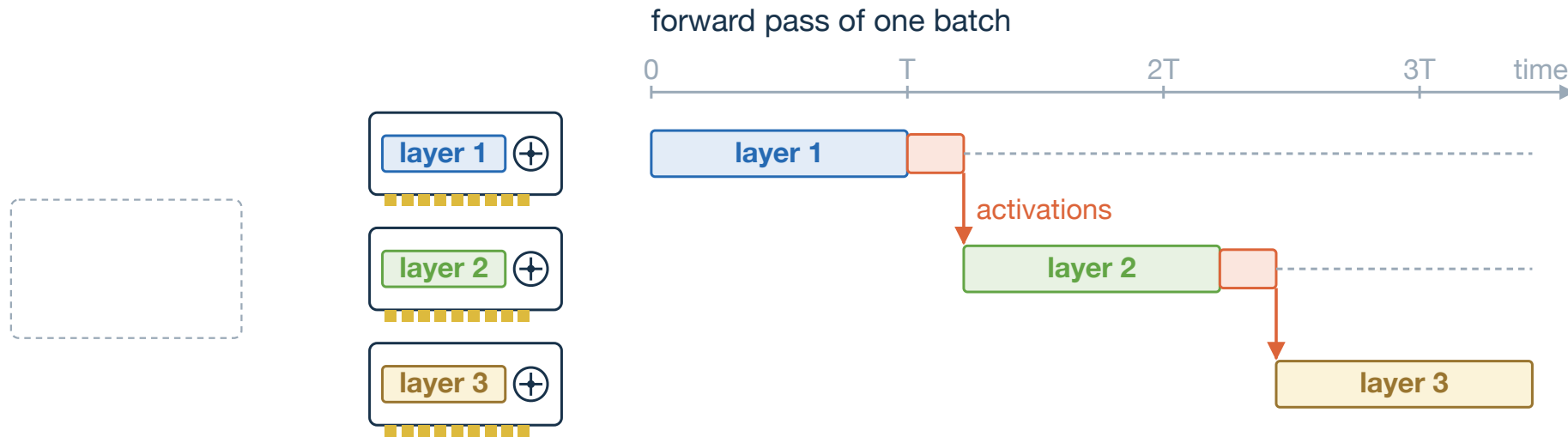
- Thus, we can spread the layer across GPUs to use parallelism



Training: spread the tasks across GPUs (machines)

For example, model follows a layer-by-layer computation, where each layer can be viewed as a dependent sub-task

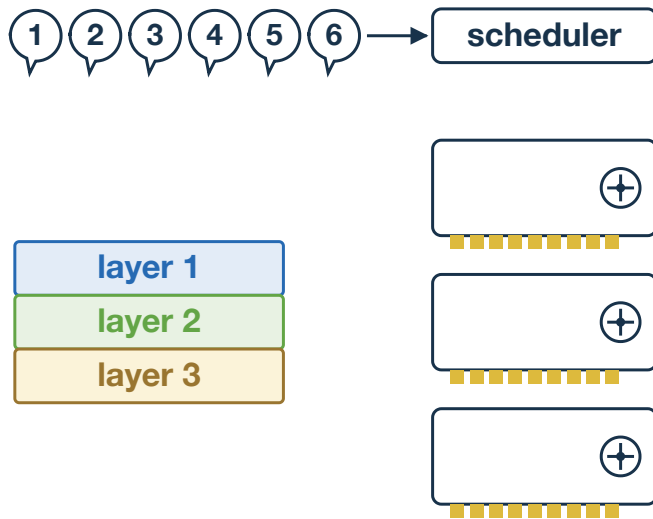
- Thus, we can spread the layer across GPUs to use parallelism



It is non-trivial: you can clearly see that naively doing so has many "bubble"s that affect machine utilizations

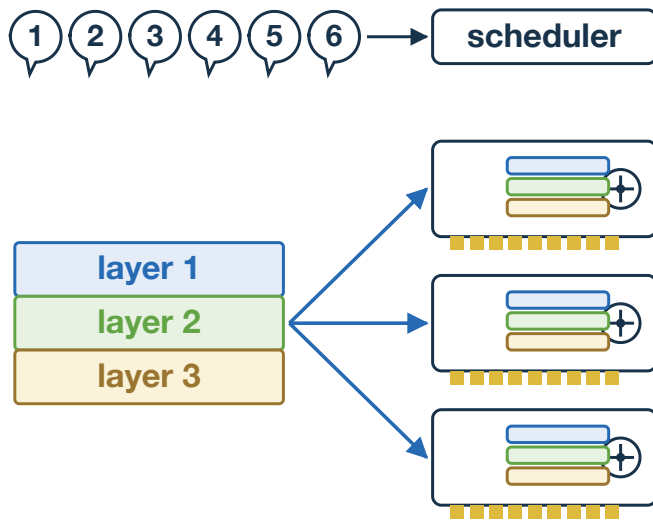
Inference (or normal request processing in traditional systems): we need to serve many requests at the same time

A scheduler sends each request to one GPU, which runs the full model



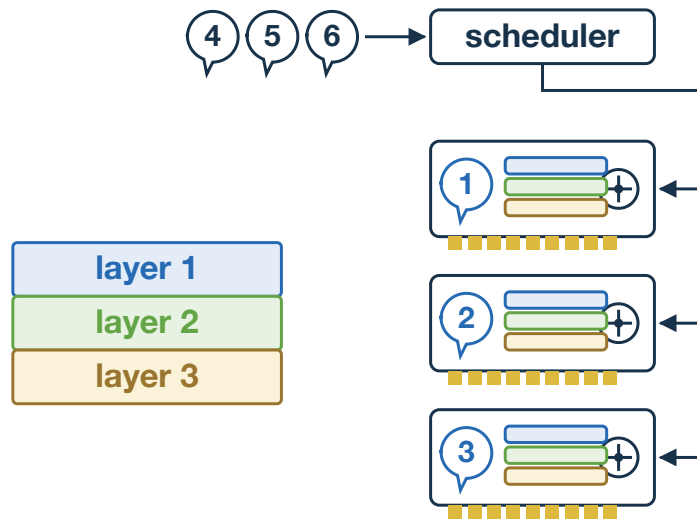
Inference (or normal request processing in traditional systems): we need to serve many requests at the same time

A scheduler sends each request to one GPU, which runs the full model



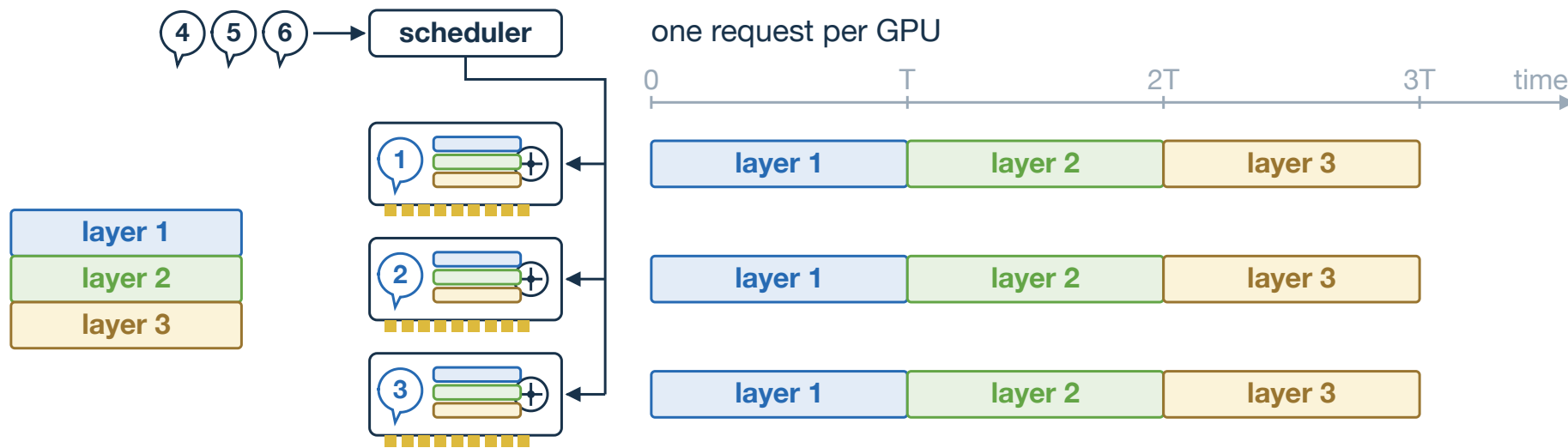
Inference (or normal request processing in traditional systems): we need to serve many requests at the same time

A scheduler sends each request to one GPU, which runs the full model



Inference (or normal request processing in traditional systems): we need to serve many requests at the same time

A scheduler sends each request to one GPU, which runs the full model



The problem with parallelism: Consistency (the data can still go wrong!)

Intra-task: **sub-task** have **dependency** that must enforce

- e.g., in training, there is `Forward → Backward → Update`
- Each step depends on the previous one; even within a task, not everything can run in parallel

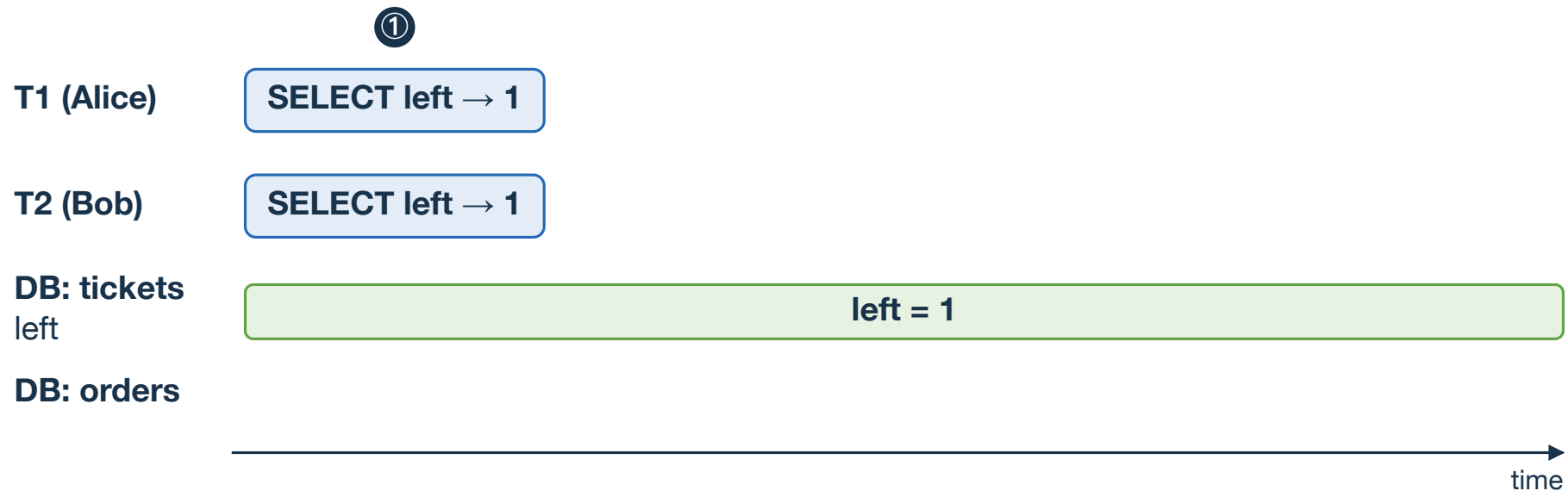
Inter-task: can access the shared data with **race condition**

- e.g., Two requests both see "1 ticket left" → 2 tickets may be sold
- Different tasks touch the same data; their execution order must be coordinated

Inter-task example: two transactions, one ticket (Isolation)

Ticket table `left = 1`; Alice (T1) and Bob (T2) run the same transaction:

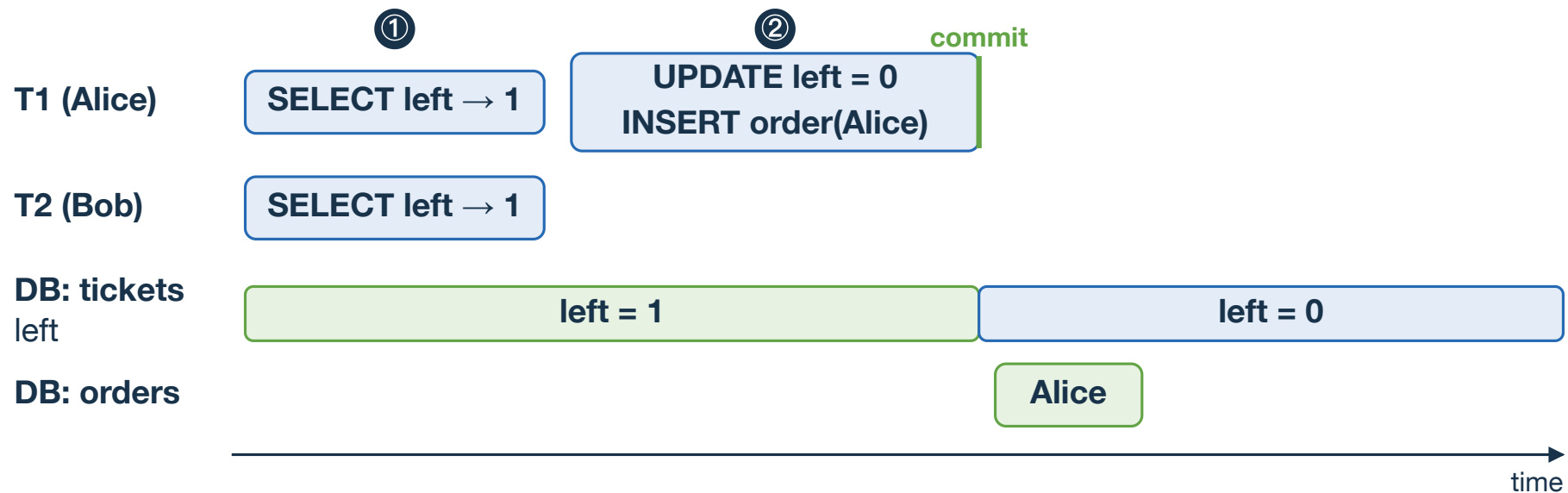
- `SELECT left` → if `left > 0`: `UPDATE left = 0`, `INSERT order`, `COMMIT`



Inter-task example: two transactions, one ticket (Isolation)

Ticket table `left = 1`; Alice (T1) and Bob (T2) run the same transaction:

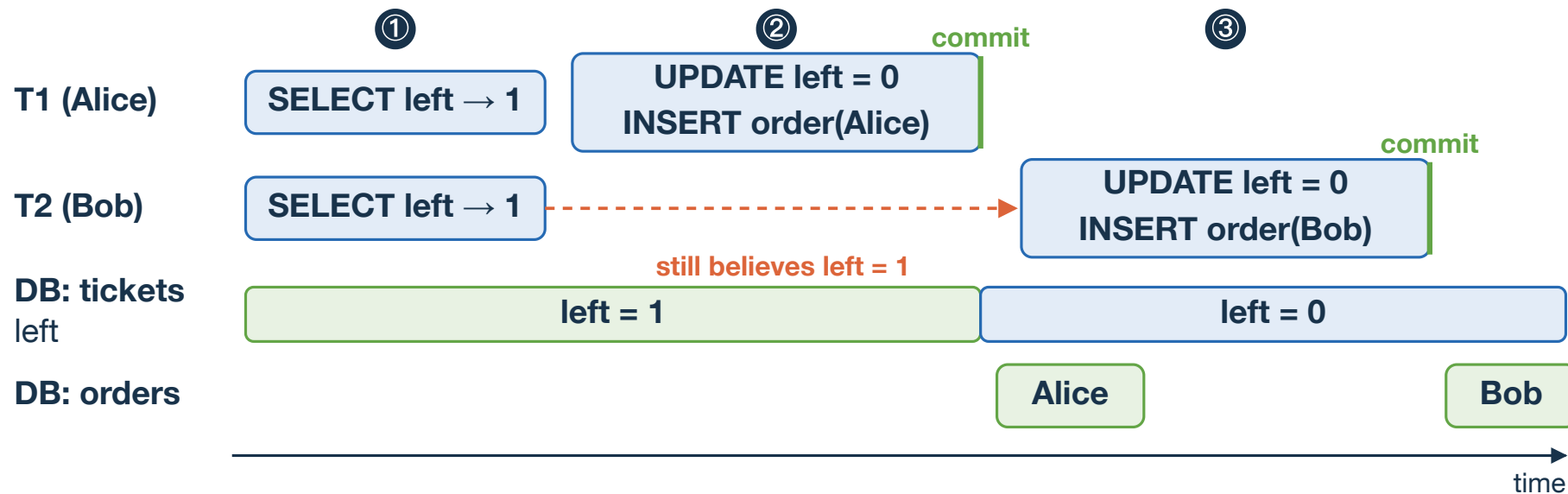
- `SELECT left` → if `left > 0`: `UPDATE left = 0`, `INSERT order`, `COMMIT`



Inter-task example: two transactions, one ticket (Isolation)

Ticket table `left = 1`; Alice (T1) and Bob (T2) run the same transaction:

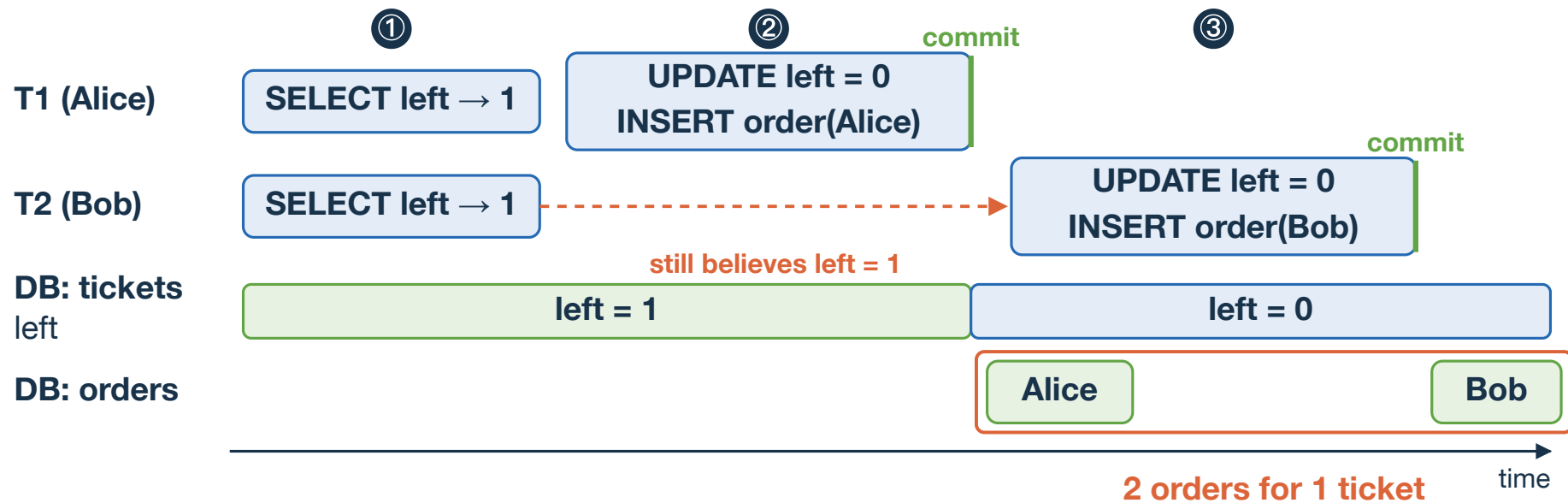
- `SELECT left` → if `left > 0`: `UPDATE left = 0`, `INSERT order`, `COMMIT`



Inter-task example: two transactions, one ticket (Isolation)

Ticket table `left = 1`; Alice (T1) and Bob (T2) run the same transaction:

- `SELECT left` → if `left > 0`: `UPDATE left = 0`, `INSERT order`, `COMMIT`



Each task run alone is correct; but when **interleaved**, there is a problem

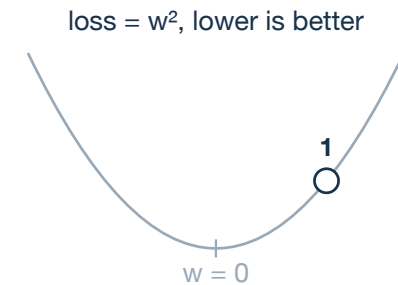
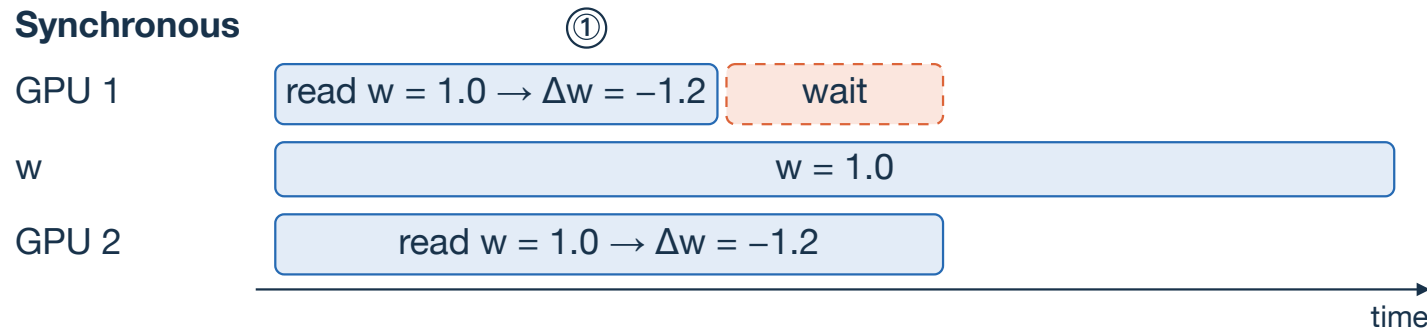
Isolation: concurrent transactions must appear to run one after another

Intra-task example: synchronous vs. asynchronous training

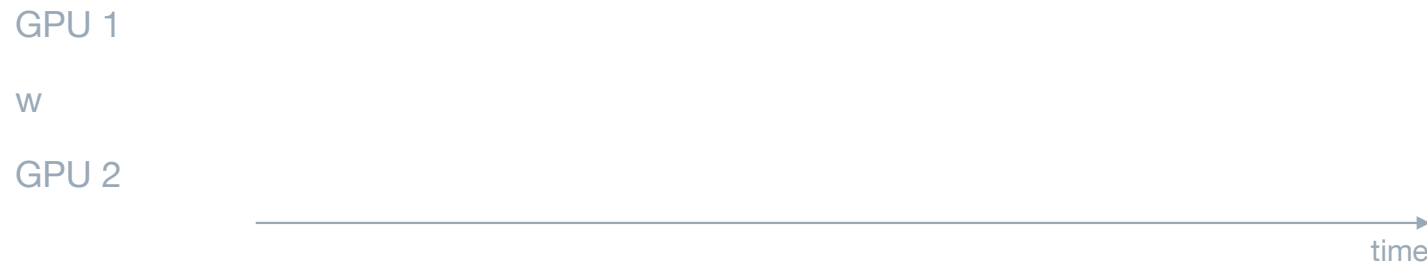
Compute the update of the Δw of the average of all Δw computed on all GPUs

- **Sync**: wait for all GPUs finish their job, and do the update
- **Async**: any GPU finished can update the w

Synchronous



Asynchronous

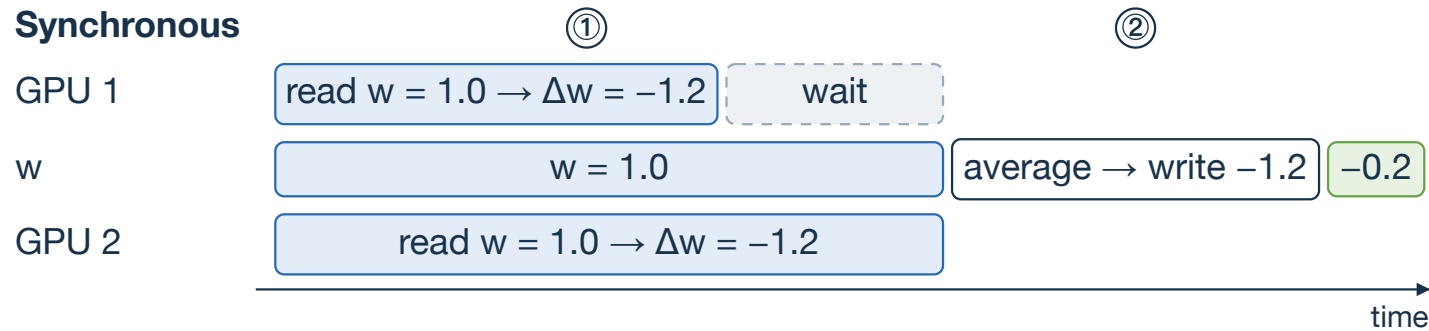


Intra-task example: synchronous vs. asynchronous training

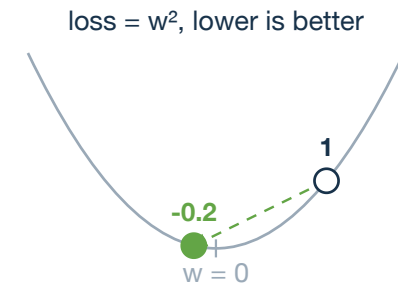
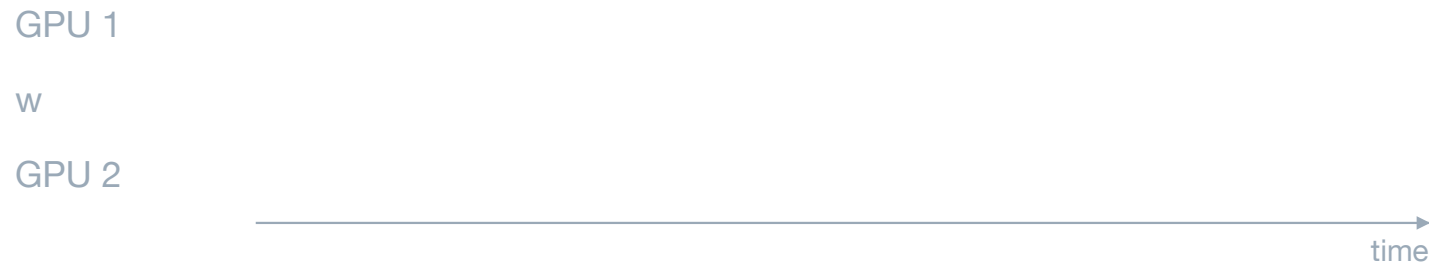
Compute the update of the Δw of the average of all Δw computed on all GPUs

- **Sync**: wait for all GPUs finish their job, and do the update
- **Async**: any GPU finished can update the w

Synchronous



Asynchronous

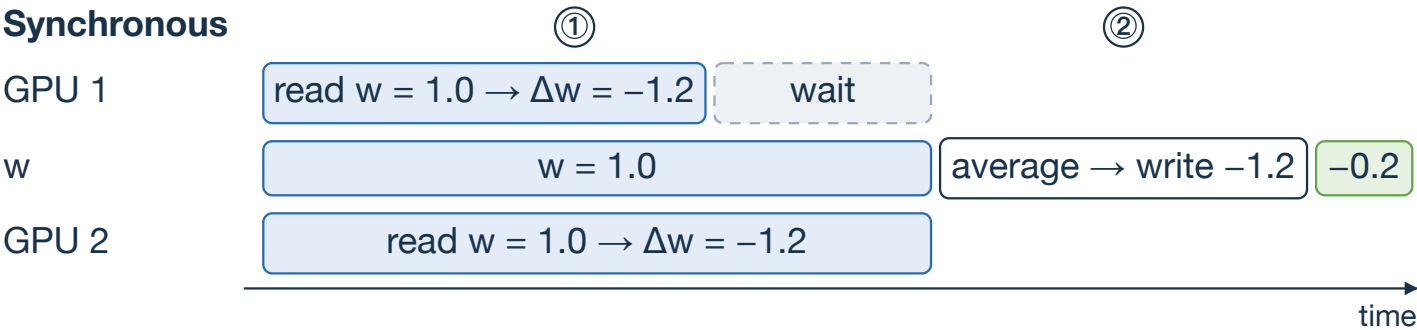


Intra-task example: synchronous vs. asynchronous training

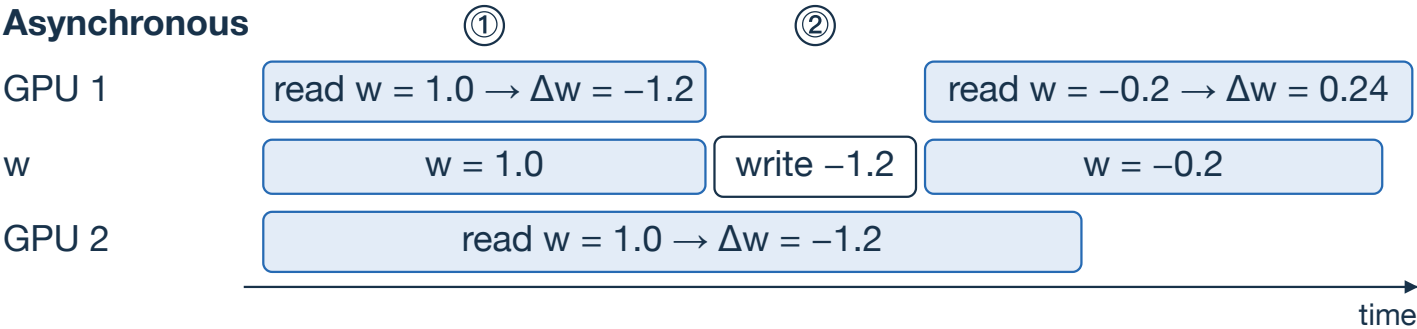
Compute the update of the Δw of the average of all Δw computed on all GPUs

- **Sync**: wait for all GPUs finish their job, and do the update
- **Async**: any GPU finished can update the w

Synchronous



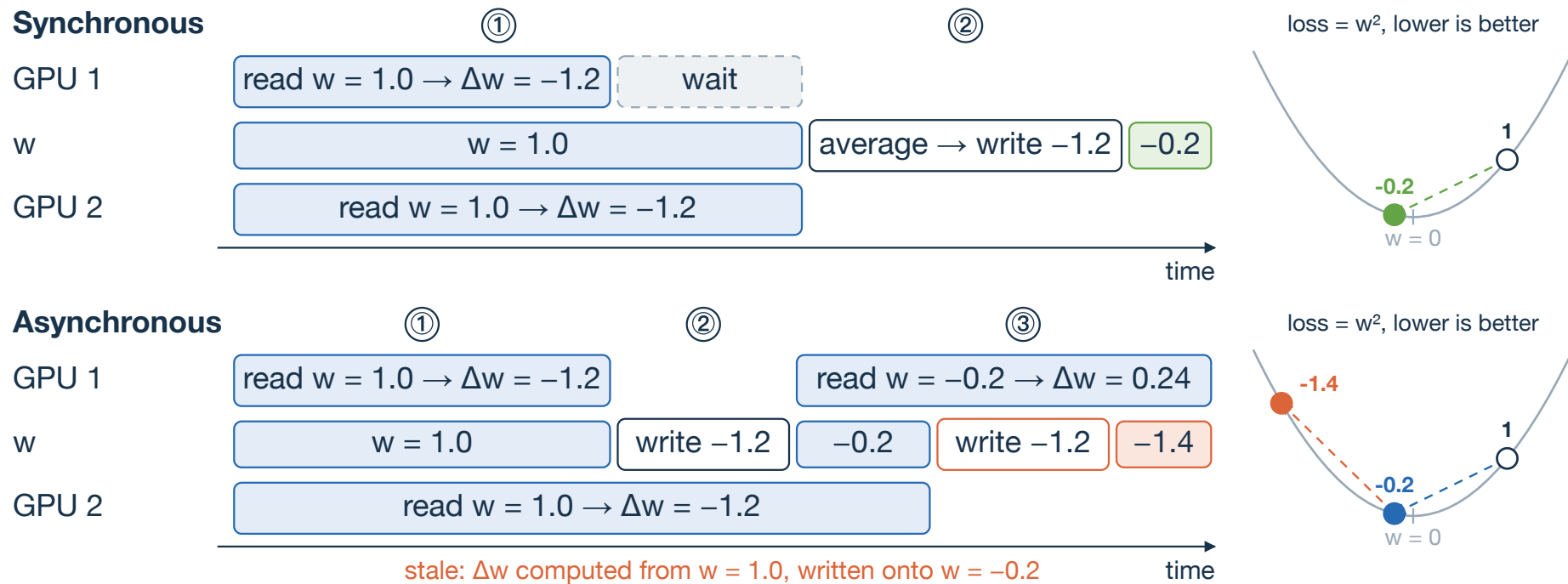
Asynchronous



Intra-task example: synchronous vs. asynchronous training

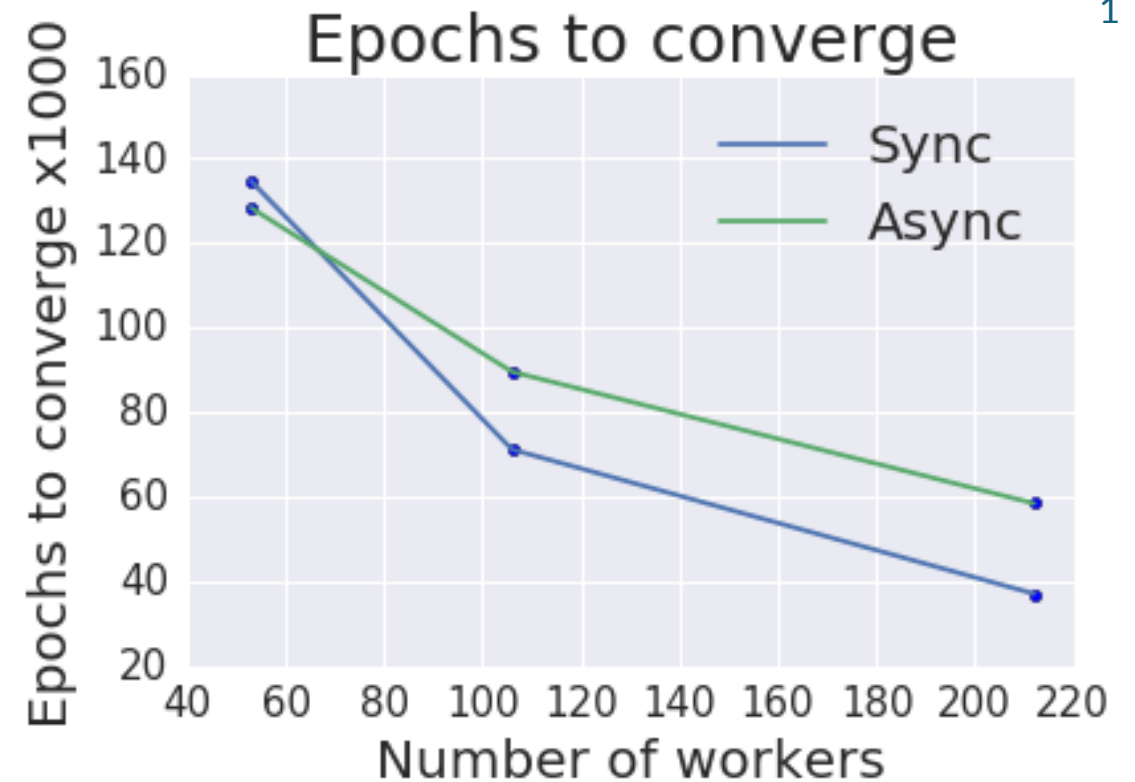
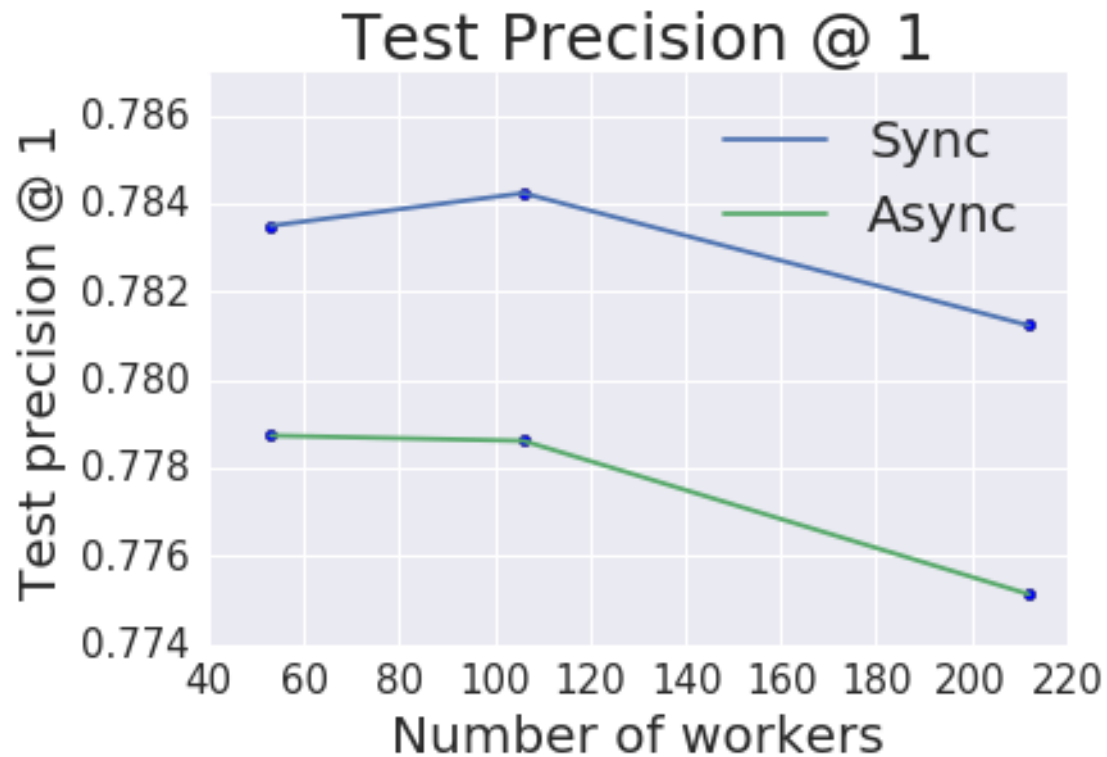
Compute the update of the Δw of the average of all Δw computed on all GPUs

- **Sync**: wait for all GPUs finish their job, and do the update
- **Async**: any GPU finished can update the w



A stale update leads to a slower coverage (not mentioning harder to debug)

Asynchronous training in practice: lower accuracy, slower convergence



¹ Chen, Monga, Bengio, Jozefowicz, [Revisiting Distributed Synchronous SGD](#), arXiv 2016 (v3), Figure 8(b)(c)

Why concurrent consistency is hard: Inter-task Dependency Tracking

Inter-task dependencies are often unknown:

- Before execution, the system does not know which tasks will touch the same data.

Take ticket buying as an example: two users both request "buy any one ticket"

- When the requests start, the system does not know which ticket each user will end up with
- The two requests look independent; the dependency only appears if they compete for the same ticket

Hence we need concurrency control methods: 2PL, OCC, etc.

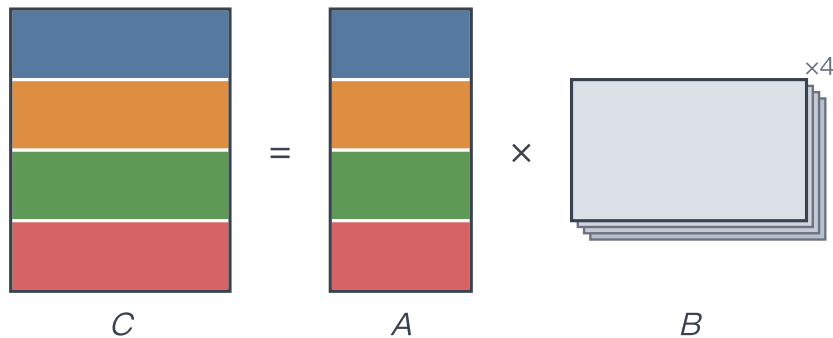
Another parallelism challenge: Intra-task Workload Partition

Intra-task dependencies are usually known, so consistency is not that hard

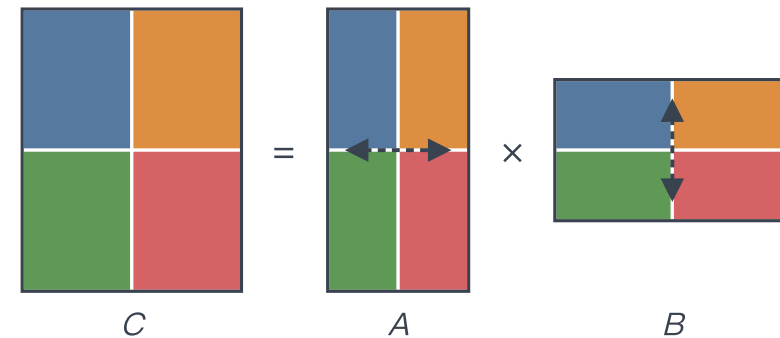
- E.g., one training step: `Forward → Backward → Update` dependency is well-known

But another challenge is **workload partition**: how do we partition a task's to subtasks?

- Example: a big matrix multiply `C = A × B`, computed by 4 machines together
- Different partitioning methods have different computation communication **trade-offs**!



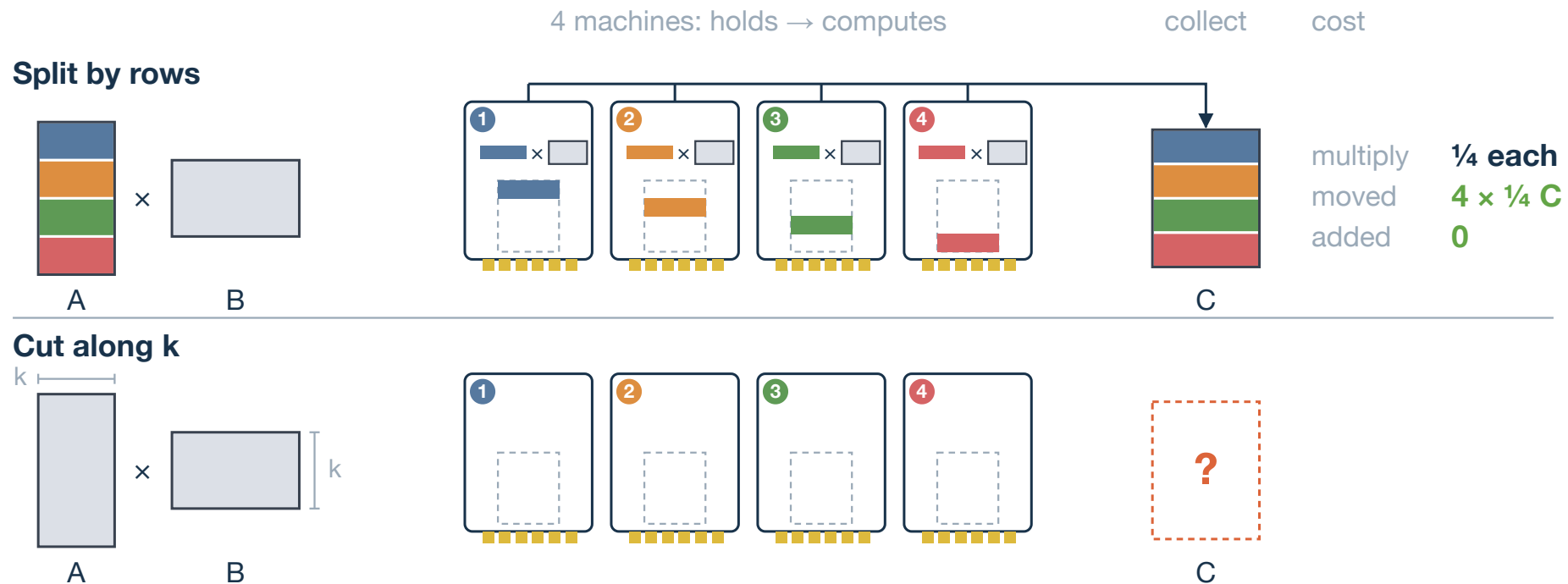
Split by rows: zero communication, but B is redundant



Split into blocks: saves memory, but needs communication

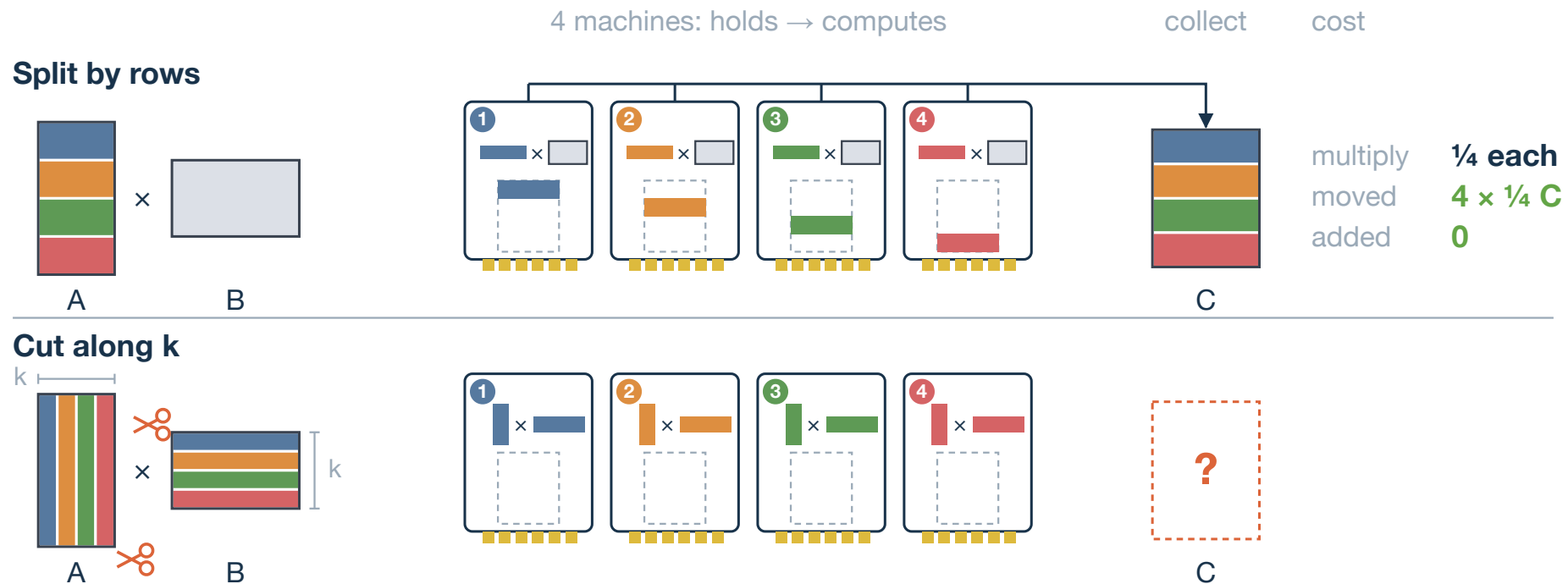
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



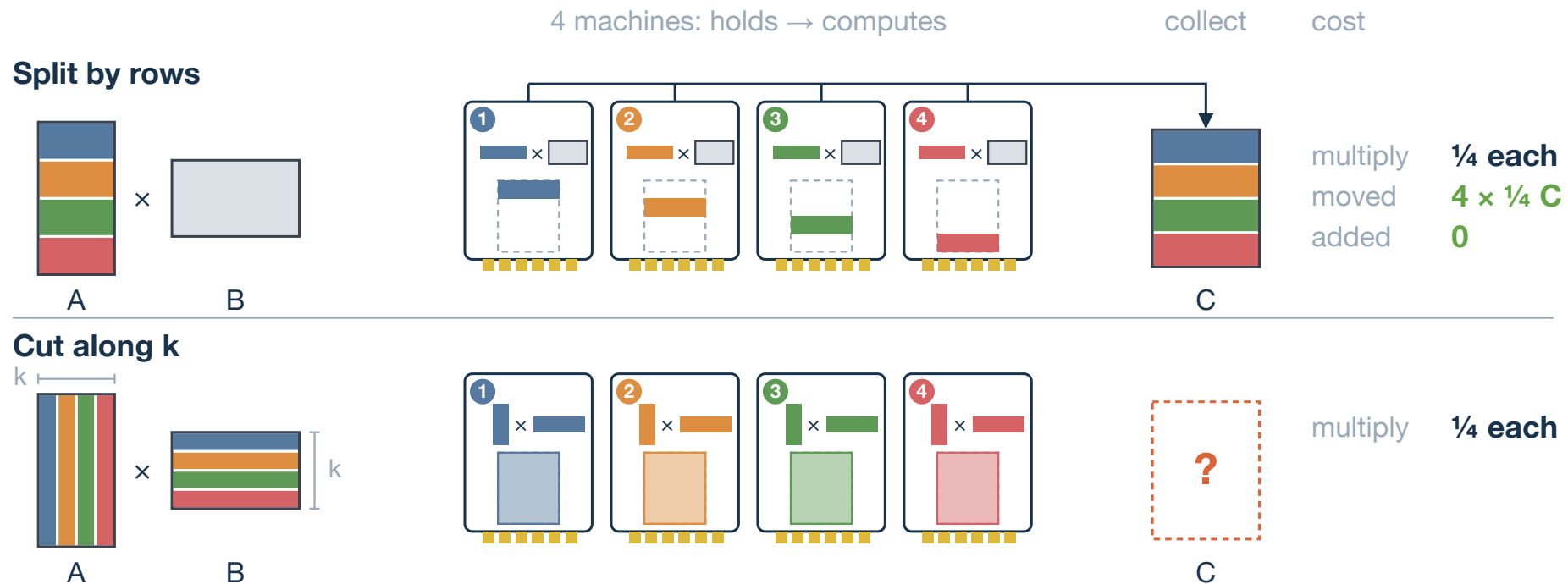
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



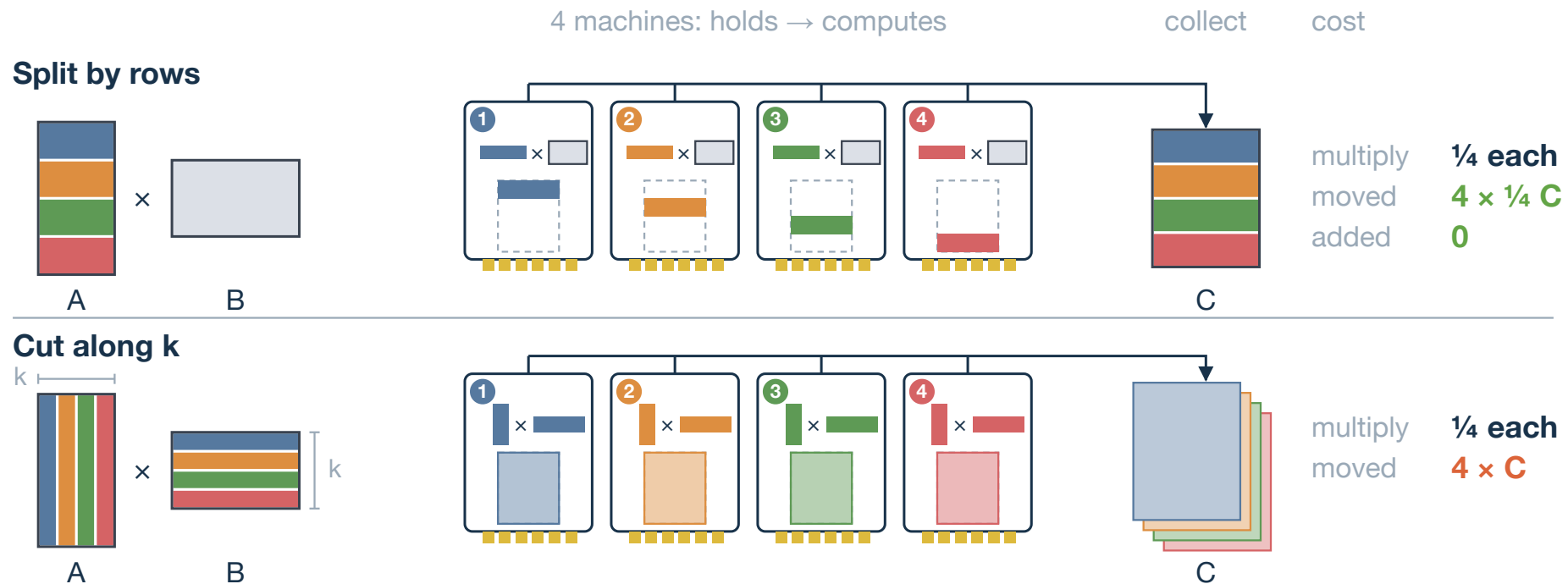
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



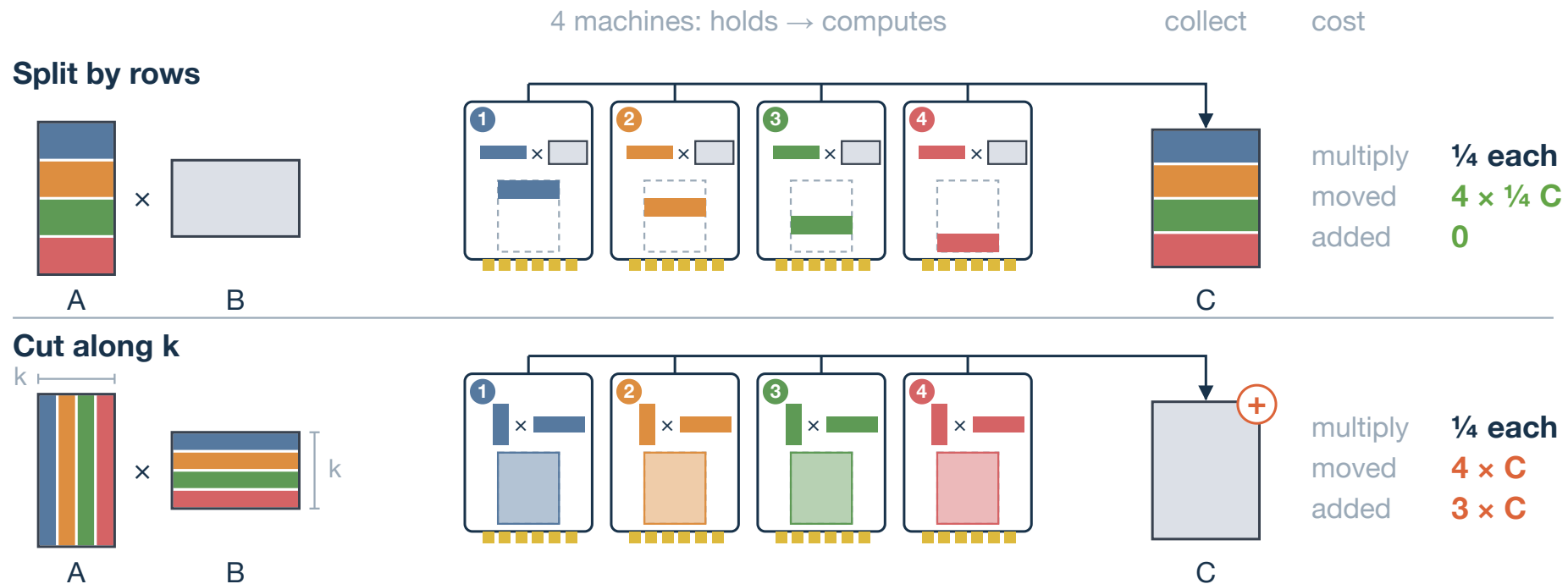
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



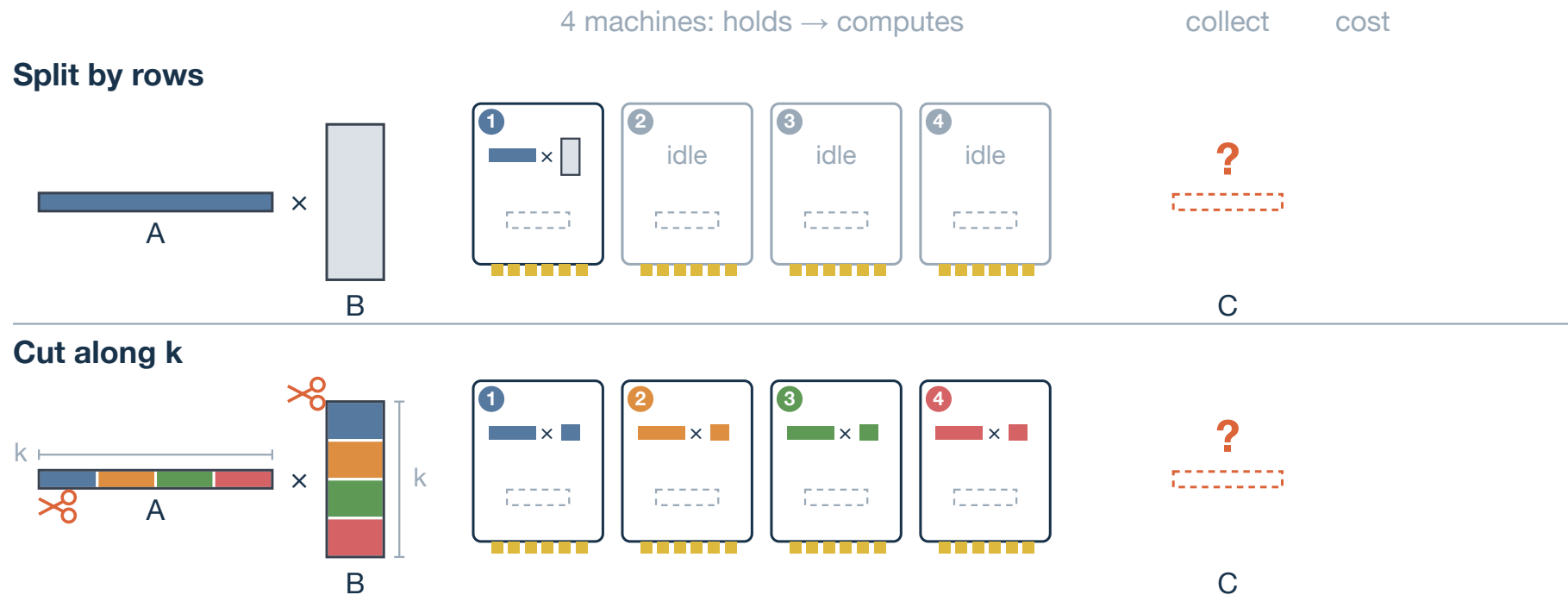
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



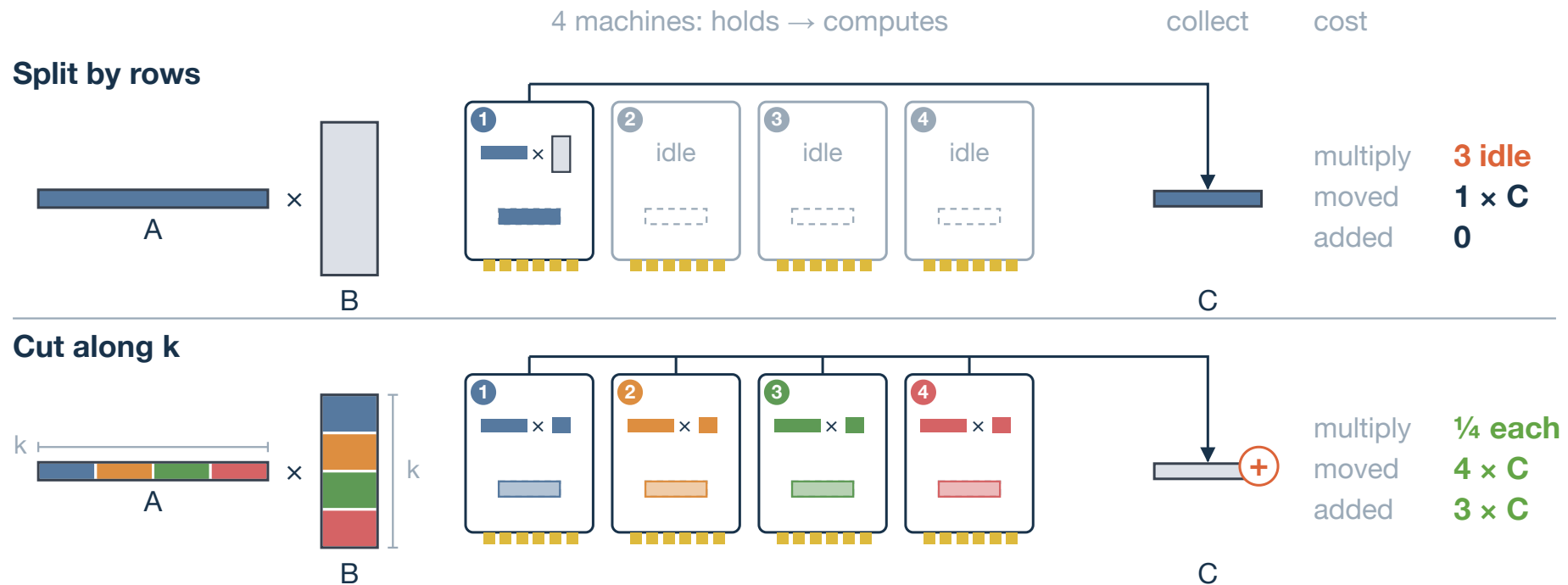
Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over



Cut along k instead: even ignoring memory, the extra work differs

Same $C = A \times B$ on 4 machines, inputs in place. Top: split by rows (last page). Bottom: cut along k , the dimension summed over

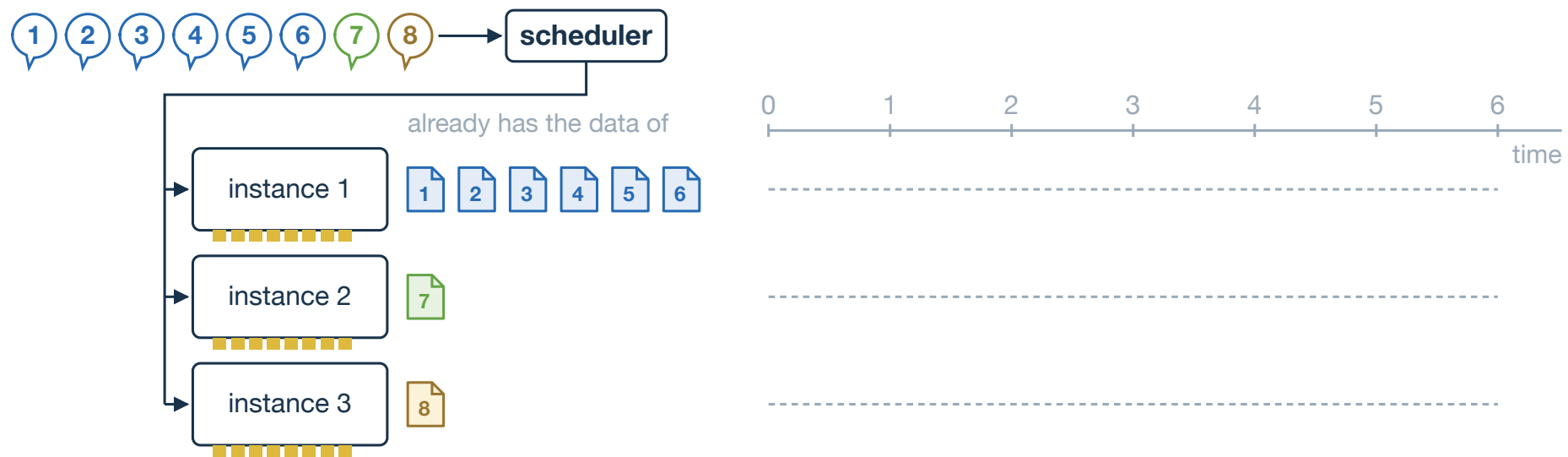


The detailed partition method depends on the application "shape"

Inter-task Partition has challenges too: Load Balance is not that simple

Inter-task partition: each request is a task, and we only decide **who goes where**

- **Load:** throughput is set by the **busiest** machine, not by the average one
- **Cost:** a request is cheaper on the instance that **already has the data it needs**

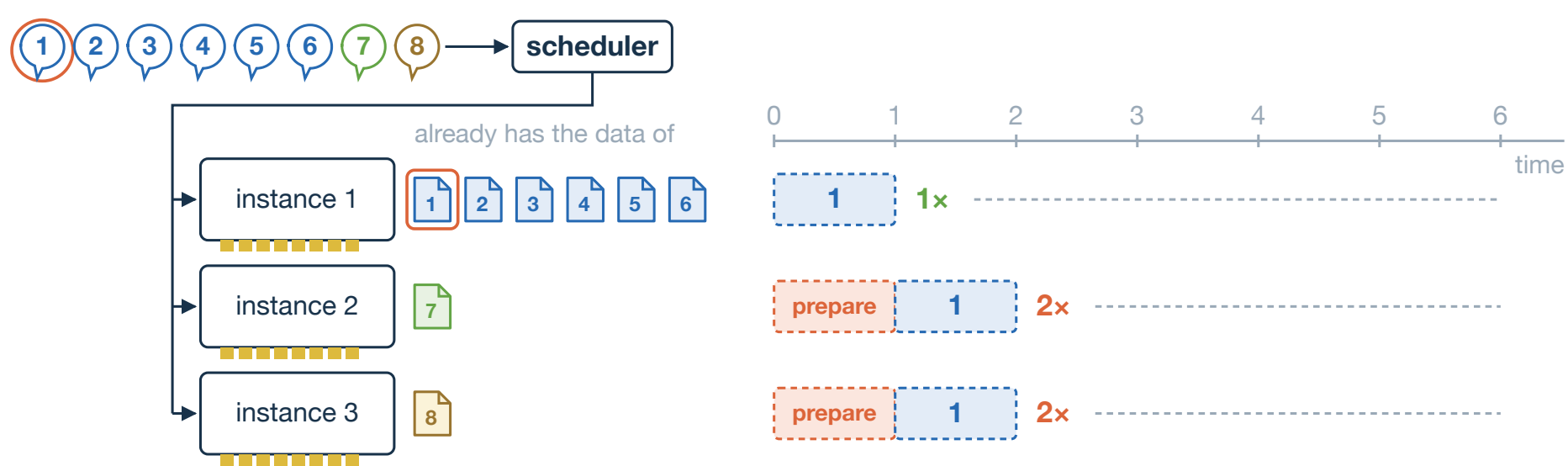


¹ Example after [Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling](#), arXiv:2603.15202

Inter-task Partition has challenges too: Load Balance is not that simple

Inter-task partition: each request is a task, and we only decide **who goes where**

- **Load:** throughput is set by the **busiest** machine, not by the average one
- **Cost:** a request is cheaper on the instance that **already has the data it needs**

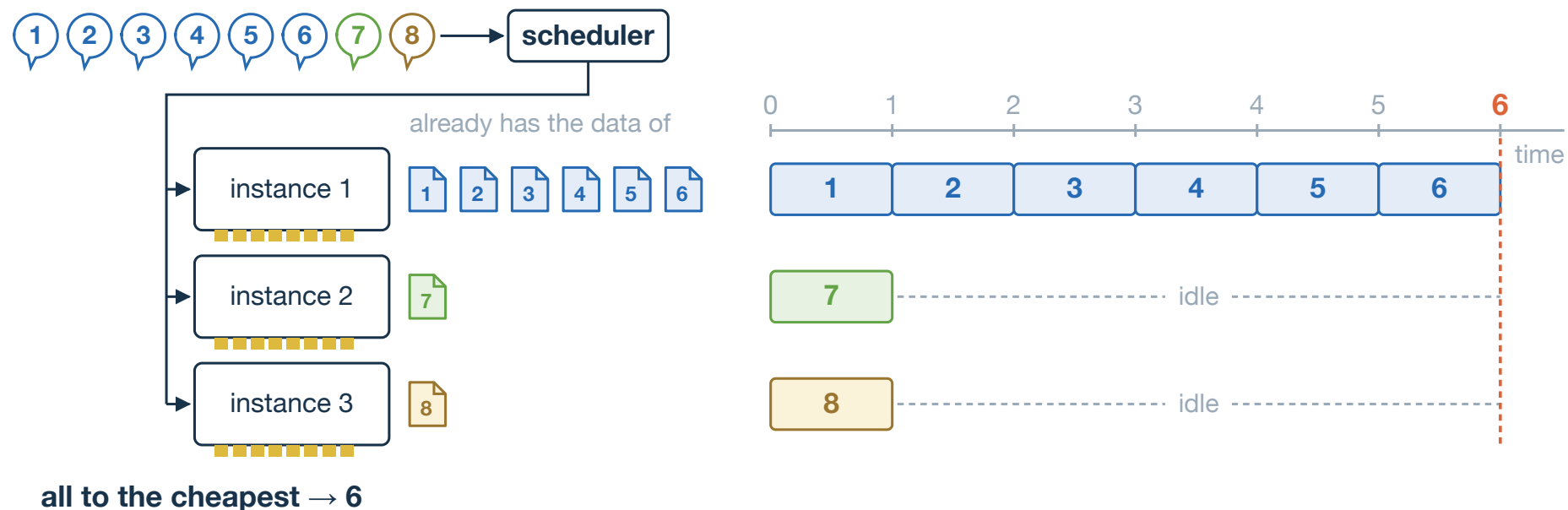


¹ Example after [Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling](#), arXiv:2603.15202

Inter-task Partition has challenges too: Load Balance is not that simple

Inter-task partition: each request is a task, and we only decide **who goes where**

- **Load:** throughput is set by the **busiest** machine, not by the average one
- **Cost:** a request is cheaper on the instance that **already has the data it needs**



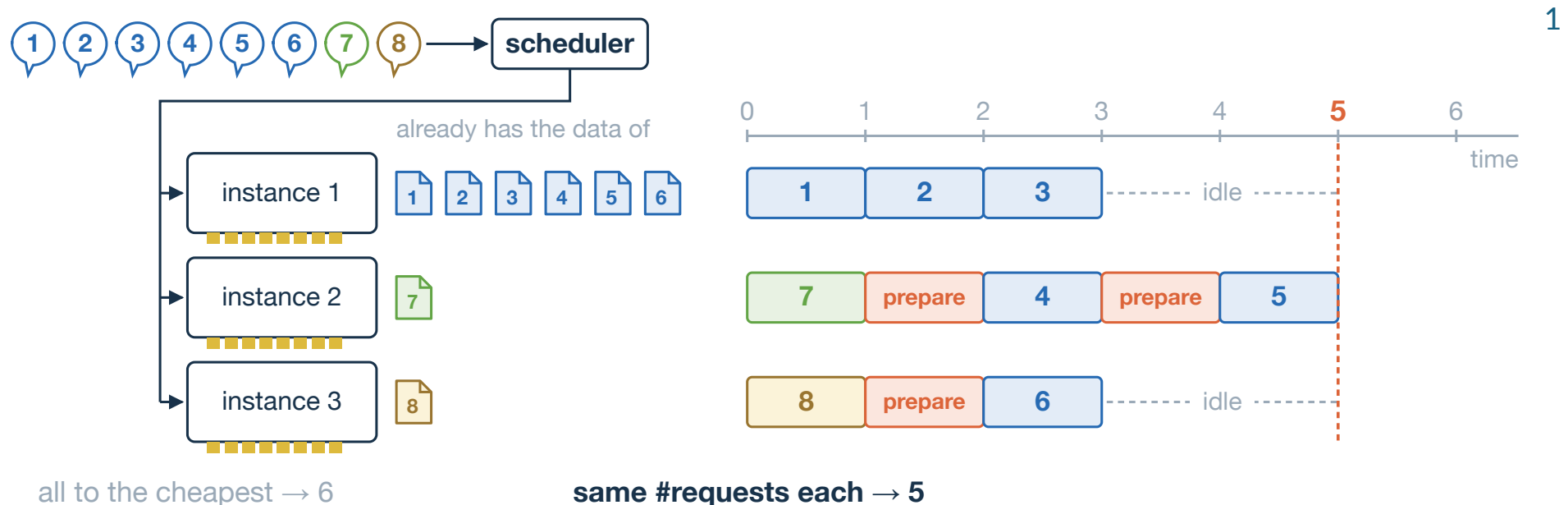
1

¹ Example after [Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling](#), arXiv:2603.15202

Inter-task Partition has challenges too: Load Balance is not that simple

Inter-task partition: each request is a task, and we only decide **who goes where**

- **Load:** throughput is set by the **busiest** machine, not by the average one
- **Cost:** a request is cheaper on the instance that **already has the data it needs**

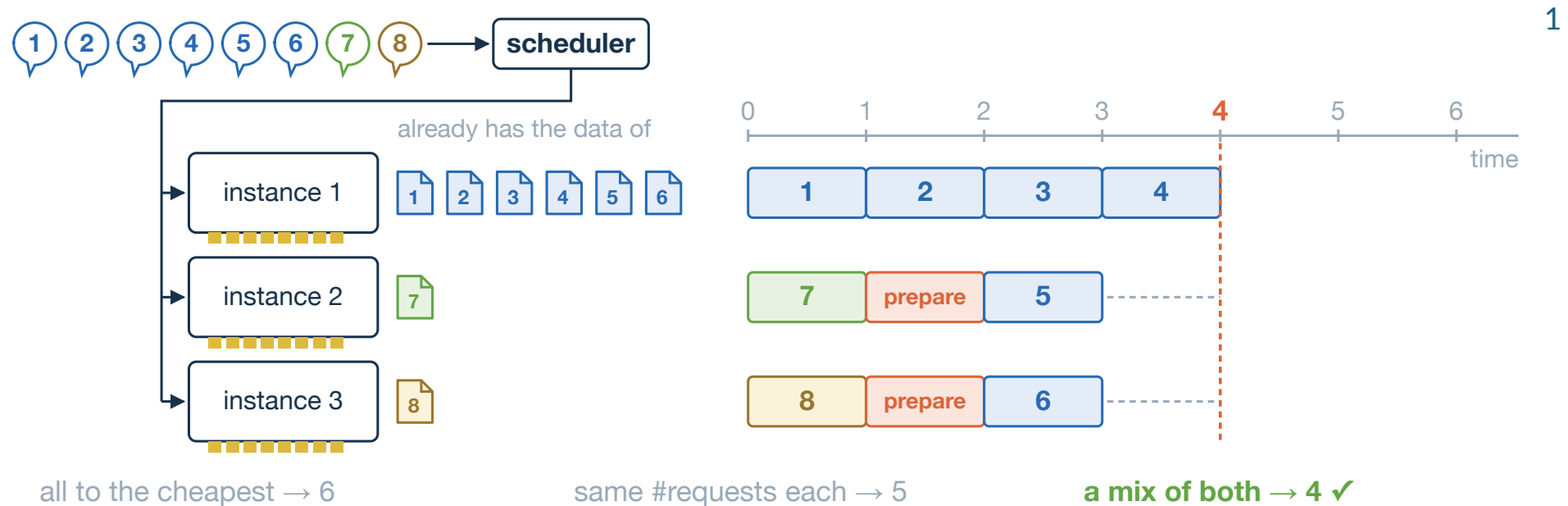


¹ Example after [Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling](#), arXiv:2603.15202

Inter-task Partition has challenges too: Load Balance is not that simple

Inter-task partition: each request is a task, and we only decide **who goes where**

- **Load:** throughput is set by the **busiest** machine, not by the average one
- **Cost:** a request is cheaper on the instance that **already has the data it needs**



Chasing either one alone loses, and the sweet spot **moves with the workload**

¹ Example after [Simple is Better: Multiplication May Be All You Need for LLM Request Scheduling](#), arXiv:2603.15202

Scaling is not that easy, there are walls!

Along the way, we have already hit several walls of the physical world:

- **The compute wall:** a chip's compute and power are limited (compute is bounded by today's devices)
- **The data wall:** data access speed (latency, bandwidth) has hard limits

We need systems methods to deal with these walls!

- But there are also walls that cannot be addressed

Recall: scaling a system is mostly about solving "consistency" + "reliability" + "performance"

- Can we have them all? The CAP theorem tells us we cannot.



Takeaway: Scale the System with Three Easy Pieces — **Consistency, Reliability & Performance**

Digitalization and AI both stand on a scalable compute system infrastructure

A scalable compute system must deliver three pieces:

- **Consistency:** is the data correct?
- **Reliability:** can it operate under failures? Can data be lost? Can it still be accessed (atomicity / durability / availability)?
- **Performance:** is it fast enough (latency)? Can it handle more (throughput)?

And today we only covered part of each piece; much more is left:

- **Consistency:** e.g., transactions — many reads and writes must look like one
- **Reliability:** e.g., security — data must not be stolen or tampered with; that is also part of being reliable
- **Performance:** e.g., networking performance, elasticity, ...
- We will cover them piece by piece in later lectures